

러닝플랫폼 학습 가이드 문서

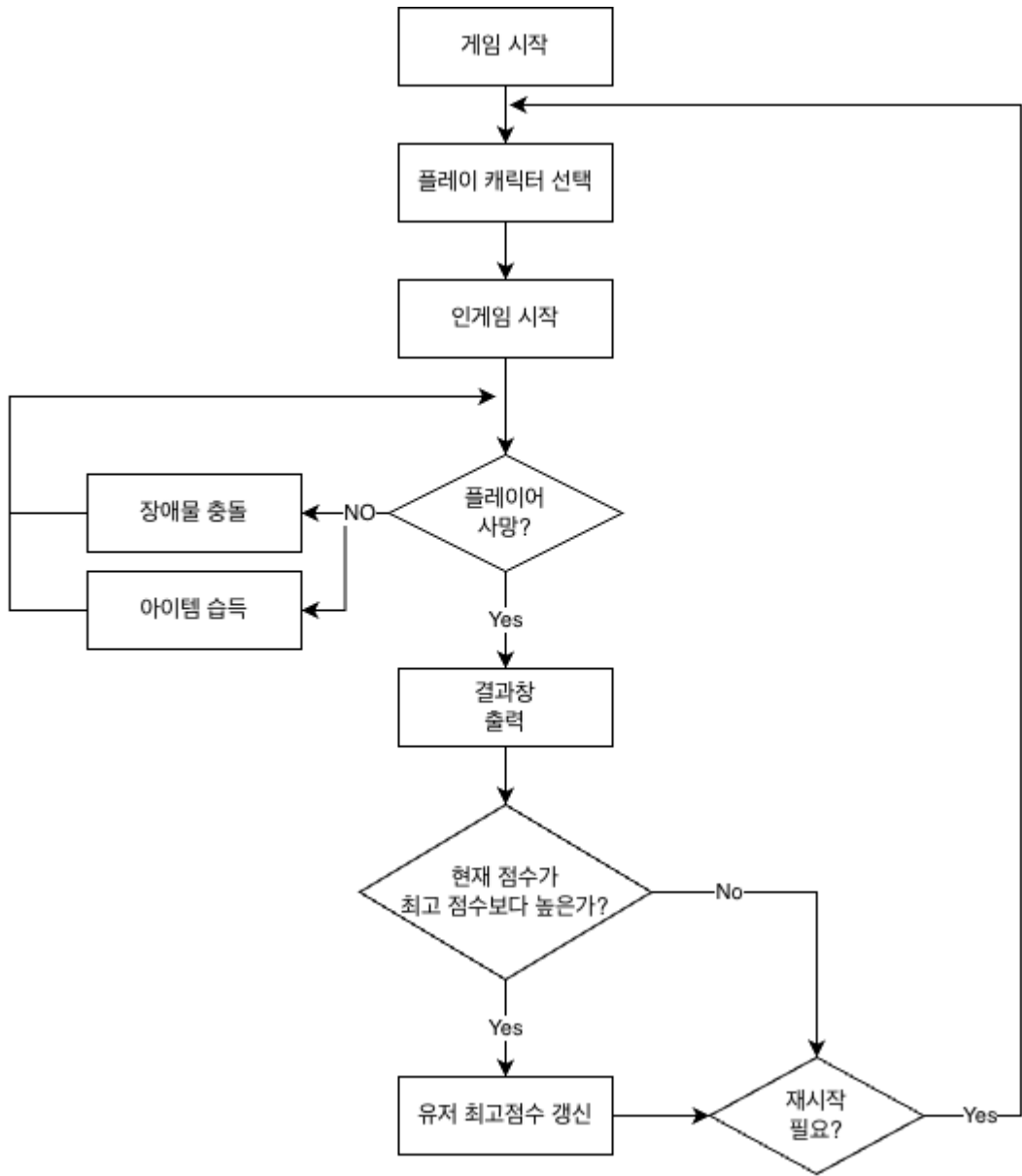
학습 목표

- 가이드 문서와 샘플 월드를 통하여 러닝플랫폼 게임의 핵심 기능들을 구현하는 방법에 대해 학습합니다.
 - 메이플스토리 월드에서 제작하려는 게임의 방식에 따라 맵을 세팅하는 방법에 대해 학습합니다.
 - Logic에 대해 이해하고, 이를 활용하여 게임 구조를 설계하는 방법을 학습합니다.
 - 학습한 내용을 바탕으로 월드를 원하는 형태로 가공하고 응용할 수 있습니다.
-

학습 핵심 기능

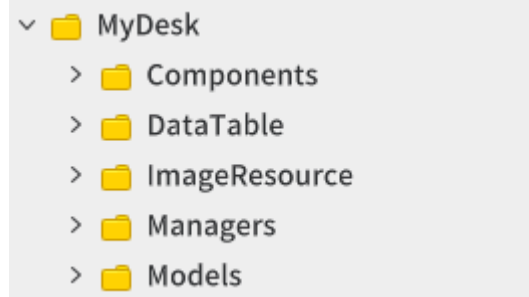
- **Physics Component:** 기본 제공되는 Body 컴포넌트(Rigid, Kinematic, Sideview)외에도, 물리 엔진이 적용된 PhysicsBody와 PhysicsCollider 컴포넌트의 사용법을 학습합니다.
- **절차적 맵 생성:** 캐릭터가 달릴 수 있는 발판을 단순히 무작위로 무작위로 생성하는 것을 넘어, 타일이 자연스럽게 연결 되도록 생성하는 방법을 학습합니다.
- **오브젝트 풀:** 다수의 발판 타일, 아이템을 관리하기 위해 오브젝트 풀 기법의 원리를 학습하고, 제작할 수 있습니다.

핵심 게임플레이 루프



- 1. **게임 시작:** 타이틀 화면과 함께 게임 시작 또는 게임의 조작방법을 확인할 수 있습니다.
- 2. **플레이 캐릭터 선택:** 타이틀 화면에서 게임 시작 버튼을 눌렀을 경우 캐릭터를 선택하는 UI가 출력됩니다.
- 3. **인게임 시작:** 카운트다운을 진행하고 본격적인 게임을 시작합니다. 캐릭터별 스크롤 속도가 적용되어 타일, 배경요소가 캐릭터 반대 방향으로 이동되며 아이템, 장애물 요소와 상호작용이 일어납니다.
- 4. **캐릭터 체력 상태 검사:** 게임 종료 조건인 시간에 따라 시간 흐름에 따라 지속해서 감소하는 체력이 0이 되었는지 확인합니다.
- 5. **결과 확인:** 현재 점수와 기존 최고 점숫값을 게임 결과 창에 출력하며 만약 현재 점수가 기존 최고 점숫값보다 높다면 유저의 최고 점숫값을 새로 갱신합니다.
- 6. **재시작 요청 확인:** 게임 결과 창에서 타이틀 화면 복귀를 요청하면, 초기 타이틀 상태로 돌아갑니다.

폴더 구성



이름	설명
Components	샘플 월드 구현을 위해 엔티티에 부여한 컴포넌트들을 담은 폴더입니다.
DataTable	샘플 월드를 구성하는 데 필요한 DataSet을 담은 폴더입니다.
ImageResource	게임 제작에 필요한 ImageResource를 담은 폴더입니다. 현재는 바닥 타일, 장애물에 활용되는 Image Resource만 담겨 있습니다.
Managers	메이플스토리 월드의 기능인 Logic으로 제작된 스크립트를 담은 폴더입니다.
Models	Model화한 엔티티들을 담은 폴더입니다.

구현 순서

- 💡 **기초 학습:** 메이플스토리 월드의 기본 기능 및 해당 장르의 기초적인 기능을 담은 학습 항목입니다.
- 📖 **심화 학습:** 샘플 월드에 구현된 기능을 학습하고 구현하는 항목입니다. 해당 과정의 학습을 통해 장르의 기능 확장과 학습 이해도를 높일 수 있습니다.

[💡 기초 학습]1. 월드 환경 구성하기

- 메이플스토리 월드에서 제공하는 맵의 종류와 각 Body 컴포넌트의 관계에 대해 학습합니다.
- 러닝플랫폼 게임 제작할 때 어떠한 게임제작 환경이 필요한지 학습합니다.
- PhysicsRigidbody, PhysicsCollider 컴포넌트에 대해 학습합니다.
- 실제 월드에 적용해보며, 필요한 설정이 무엇인지 파악합니다.

[💡 기초 학습]2. Logic 이해하기

- Logic이 어떤 역할을 하는지에 대해 학습합니다.
- Manager에 대한 개념을 이해하고, 사용 방법을 학습합니다.
- GameLogic, DataSetLogic, UIManagerLogic 구성 이유와 목적을 이해합니다.

[💡 기초 학습]3. 스크롤 시스템 구성하기

- 러닝플랫폼 게임 제작할 때 캐릭터와 환경요소의 이동방식을 학습합니다.
- 월드에서 보이는 배경 요소를 제어하는 방법을 학습합니다.
- 게임 진행 시점 이후로 시간이 지남에 따라 환경요소의 외형을 변경하는 방법을 학습합니다.

[💡 기초 학습]4. 절차적 생성 방식을 통한 맵 관리하기

- 러닝플랫폼에서 절차적 생성 방식이 무엇인지, 필요한 이유에 대해 학습합니다.
- 게임에서 활용될 바닥 타일을 제작하며 이를 Model로 구성하는 방법을 학습합니다.
- 데이터 테이블을 기반으로 절차적 생성 방식으로 바닥 타일이 생성될 수 있도록 DataSet을 제작합니다.
- 절차적 생성 방식을 활용한 바닥 타일을 생성하며 무한한 스크롤링 환경을 구현합니다.

[💡 기초 학습]5. 오브젝트 풀 생성하기

- 오브젝트 풀에 대한 개념을 학습하고 오브젝트 풀의 목적을 이해합니다.
- 오브젝트 풀 Logic을 구현합니다.
- 오브젝트 풀의 생성과 반환을 통해 작동 방식을 직접 확인합니다.
- 바닥 타일이 파괴되었을 때, 타일을 새로 생성하는 방법에 대해 학습합니다.

[💡 기초 학습]6. 플레이어 캐릭터 구현하기

- 러닝플랫폼 게임에서 캐릭터에게 점프, 슬라이딩 동작을 수행 시키는 방법을 학습합니다.
- 다양한 캐릭터를 만들기 위해 필요한 DataSet을 구성하는 방법을 학습하고 제작해봅니다.
- 게임이 재시작 될 경우 캐릭터의 상태 및 정보를 되돌리는 방법에 대해 학습합니다.
- 플레이어가 스킬 키 입력 시 발동되는 캐릭터별 스킬을 수행 시키는 방법을 알아봅니다.
- 플레이어에게 다양한 캐릭터를 선택할 수 있는 선택창을 제공하는 방법과 캐릭터를 선택하는 기능을 구현합니다.

[💡 기초 학습]7. Life 시스템 구현하기

- 본격적인 게임이 시작된 이후, 지속적으로 체력이 감소되는 기능을 제작하는 방법을 학습합니다.
- 체력이 지속적으로 감소될 때 이를 유저가 확인할 수 있는 UI 요소를 제작하는 방법을 학습합니다.

[💡 기초 학습]8. UI 구성하기

- UIGroup이 무엇인지, 활용 및 편집 방법에 대해 학습합니다.
- 타이틀 화면에 활용되는 UI 요소를 제작하며 UI 편집 방법에 대해 학습합니다.
- 타이틀 화면에 나타나는 UI 요소를 Manager Logic을 통해 코드로 제어하는 방법을 학습합니다.
- 인게임 내에 표기되어야 하는 UI 요소를 제작하며 실시간으로 연동되도록 구현합니다.
- 게임이 종료되었을 때 각 점수를 나타내는 결과창을 나타내는 UI를 구성합니다.

[📖 심화 학습]9. 장애물 오브젝트 구현하기

- 지속적으로 움직이고 있는 바닥 타일에 장애물 요소를 어떻게 배치할지에 대해 학습합니다.
- 캐릭터와 장애물의 충돌을 감지할 수 있도록 설정하는 방법에 대해 학습합니다.
- 캐릭터와 장애물이 충돌되었을 때 캐릭터의 체력이 감소되는 기능을 반영합니다.

[📖 심화 학습]10. 아이템 구현하기

- 지속적으로 움직이고 있는 바닥 타일에 아이템 요소를 어떻게 배치할지에 대해 학습합니다.
- 다양한 효과를 갖는 아이템을 구성하기 위해 DataSet을 구성하는 방법에 대해 학습합니다.
- 저장된 아이템 데이터 테이블을 기반으로 효과와 고깃값을 설정하는 방법에 대해 학습합니다.
- 캐릭터와 아이템의 충돌 시 발생하는 다양한 아이템 효과를 발동시키는 방법과 실제 동작을 구현합니다.

[📖 심화 학습]11. 데이터 저장 시스템 구현하기

- 플레이어의 캐릭터 사망 시, 현재 점수가 기존 최고 점수보다 높을 경우 이를 갱신하여 저장하는 방법에 대해 학습합니다.
- 플레이어가 월드를 실행했을 때 이전에 최고 점수가 있었다면 이를 불러와서 타이틀 화면에 보일 수 있도록 제작합니다.

[💡 기초 학습]월드 환경 구성하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 월드 환경 구성하기: 00:00:01

학습 목표

- 메이플스토리 월드에서 제공하는 다양한 타일맵 모드의 특징을 이해하고, 각 맵에 적합한 Body 컴포넌트의 상관관계를 파악합니다.
- 러닝 플랫폼터 장르의 특성을 분석하고, 이에 최적화된 게임 제작 환경을 스스로 구성할 수 있습니다.
- 캐릭터의 움직임과 충돌 감지를 구현하기 위해 PhysicsRigidbody와 PhysicsCollider 컴포넌트의 핵심 기능과 속성을 학습합니다.
- 학습한 물리 시스템을 실제 엔티티에 적용해 보고, 테스트 플레이를 통해 게임 기획에 맞는 물리 설정값을 조정하는 방법을 익힙니다.

해당 영상 시청 후 수행해 볼 내용

- 러닝 플랫폼터에 맞는 맵 환경 구성하기
- 캐릭터와 바닥 타일의 상호작용을 위한 컴포넌트 등록하기
- 직접 만든 바닥 타일 위를 정상적으로 밟고 이동할 수 있는지 테스트해 보기

메이플스토리 월드 맵 모드

Key Point

Map 형식마다 게임의 시점이 변경!

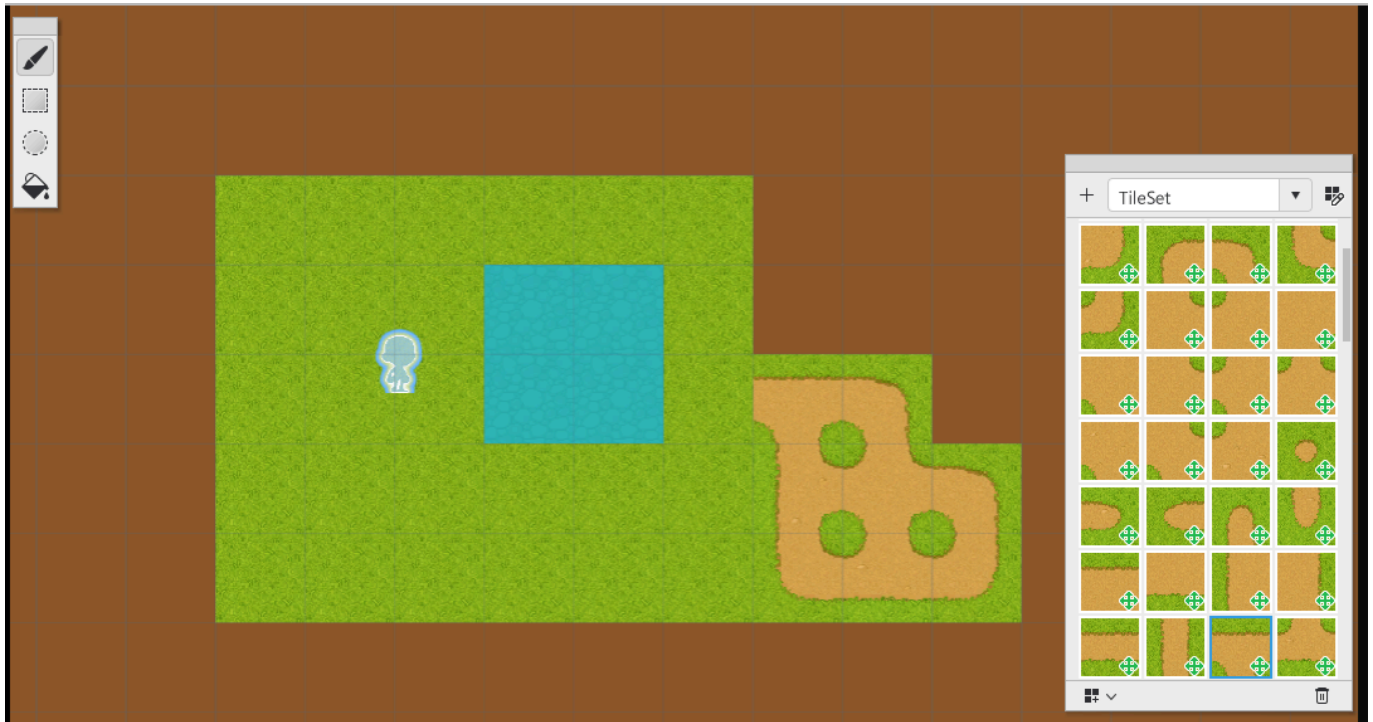
Map 형식	사용하는 Component	적용되는 게임의 시점
TileMap	Rigidbody Component	메이플스토리 느낌의 횡스크롤
RectTileMap	Kinematicbody Component	탐다운 뷰
SideviewRectTileMap	Sideviewbody Component	바둑판 형태의 Tile을 이용한 횡스크롤

TileMap



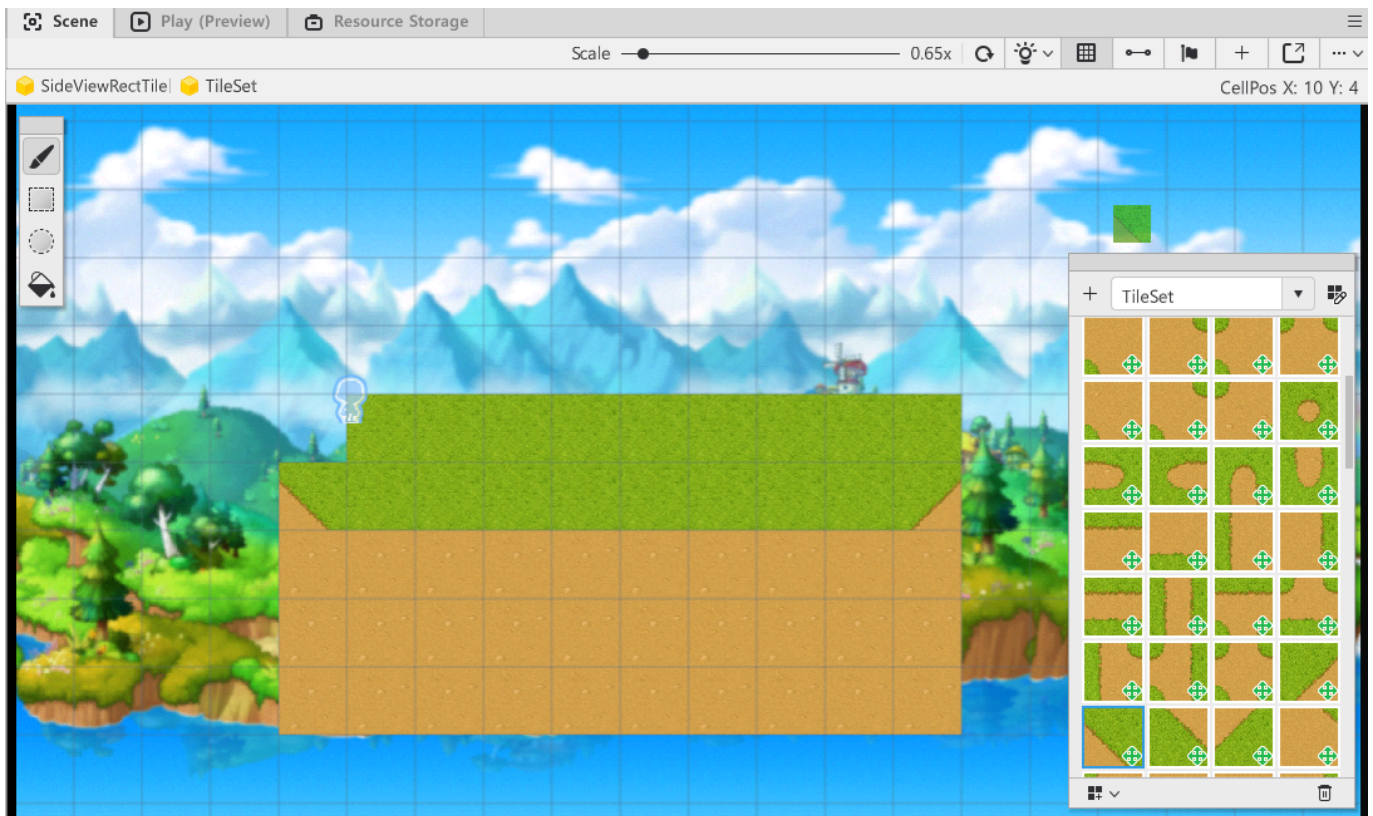
- 메이플스토리 월드에서 처음으로 월드를 생성하면 기본 지정되는 맵 형식입니다.
- 메이플스토리의 느낌을 가장 잘 살릴 수 있는 방식의 월드입니다.
- 메이플스토리 월드가 제공하는 Tile을 활용하여 발판을 생성하고, 엔티티를 배치하여 맵을 형성할 수 있습니다.
- 타일의 배치에 따라 경사로를 제작할 수 있습니다.
- 횡스크롤 형태의 게임을 제작할 때, 메이플스토리의 느낌을 살리고 싶을 때 유용한 맵 방식입니다.

RectTileMap



- 탑다운 뷰 형태의 게임을 제작할 때 효과적인 방식의 맵 형식입니다.
- 직접 타일셋을 제작하거나, 메이플스토리 월드에서 제공하는 타일셋 리소스를 활용하여 맵을 형성할 수 있습니다.
- 바둑판 형태로 타일을 배치하여 맵을 형성할 수 있습니다.
- 타일마다 이동 가능과 이동 불가능한 성질을 부여할 수 있습니다.
- TileMap, SideViewRectTileMap 방식과 다르게 4방향, 8방향으로 이동하는 형태의 게임을 제작할 수 있습니다.

SideviewRectTileMap



- 횡스크롤 방식의 게임을 제작할 때 효과적인 방식의 맵 형식입니다.
- 직접 타일셋을 제작하거나, 메이플스토리 월드에서 제공하는 타일셋 리소스를 활용하여 맵을 형성할 수 있습니다.

- 바둑판 형태로 타일을 배치하여 맵을 형성할 수 있습니다.
- RectTileMap과는 다르게 이동 불가능한 타일셋이 아닐 경우, 캐릭터가 타일을 **통과**합니다.
- 따라서 SideViewRectTileMap으로 맵을 형성할 경우, 이동 불가능한 타일셋을 **발판**으로 제작해야 합니다.

BodyComponent

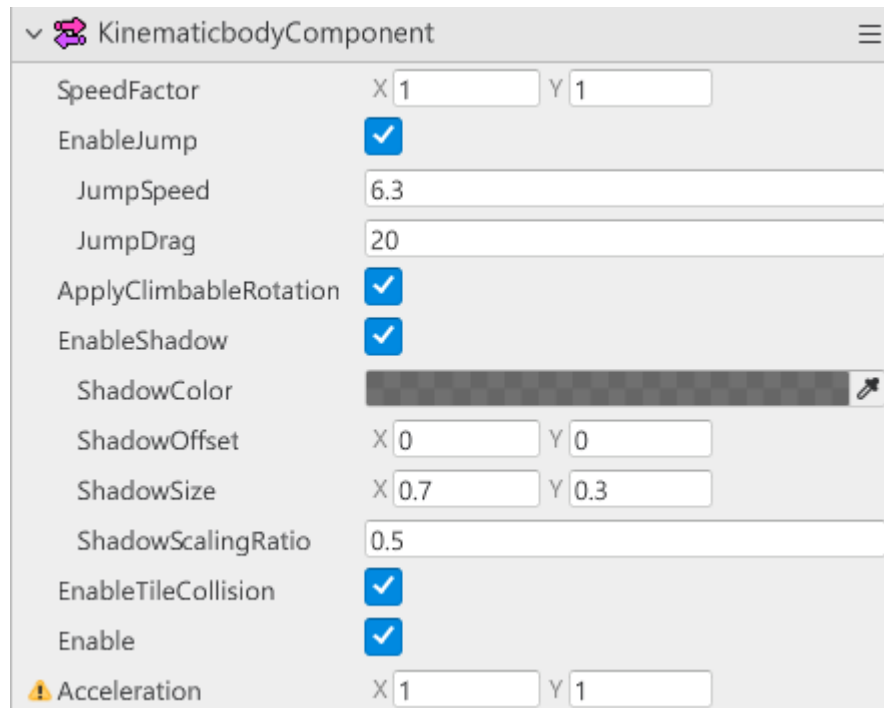
- 메이플스토리 월드에서 맵의 발판인 타일들과 상호작용하기 위해서는 BodyComponent가 필요합니다.
- BodyComponent는 맵의 형식에 따라 다른 BodyComponent를 요구합니다.
- BodyComponent는 크게 3종류의 BodyComponent가 존재합니다.

RigidbodyComponent

RigidbodyComponent	
Mass	100
Gravity	1
JumpBias	0.03
WalkJump	1.23
DownJumpSpeed	2
WalkSpeed	1.4
WalkAcceleration	1
WalkDrag	1
WalkSlant	0.9
AirAccelerationX	1
AirDecelerationX	1
FallSpeedMaxX	1
FallSpeedMaxY	1
LayerSettingType	All ▼
IsBlockVerticalLine	☐
IgnoreMoveBoundary	☐
ApplyClimbableRotation	☒
IsolatedMove	☐
KinematicMove	☐
Enable	☒

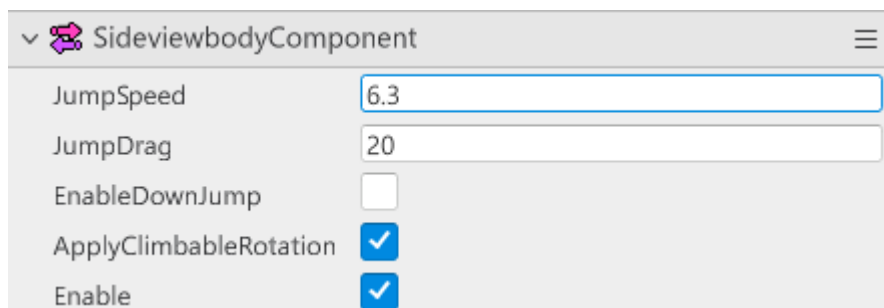
- 메이플스토리 월드의 TileMap 방식에서 캐릭터의 물리를 제어합니다.
- 맵이 TileMap인데 RigidbodyComponent가 없으면, 캐릭터가 타일과 상호작용을 할 수 없습니다.
- RigidbodyComponent내에 캐릭터의 점프 제어, 중력 제어 등 다양한 물리 프로퍼티값을 제공합니다.
- 해당 값을 변경하여 물리 방식을 변경할 수 있습니다.

KinematicComponent



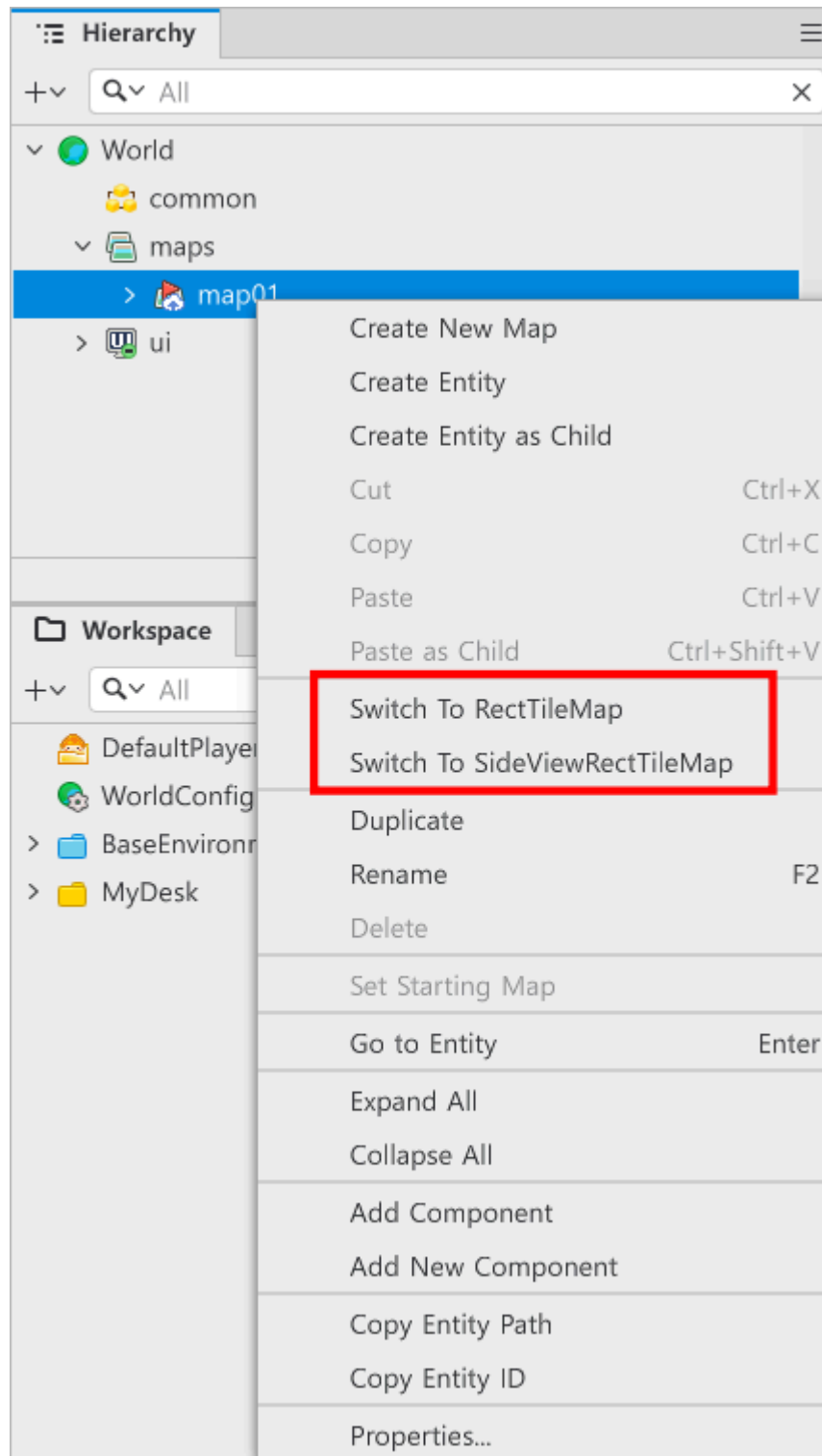
- 메이플스토리 월드의 RectTileMap 방식일 때 캐릭터의 물리를 제어합니다.
- 맵이 RectTileMap일 때 KinematicBodyComponent가 없으면, RectTileMap의 타일들과 상호작용이 일어나지 않습니다.
- KinematicBodyComponent에서 캐릭터의 이동 속도, 점프 여부, 그림자 여부 등을 조절할 수 있습니다.

SideviewbodyComponent



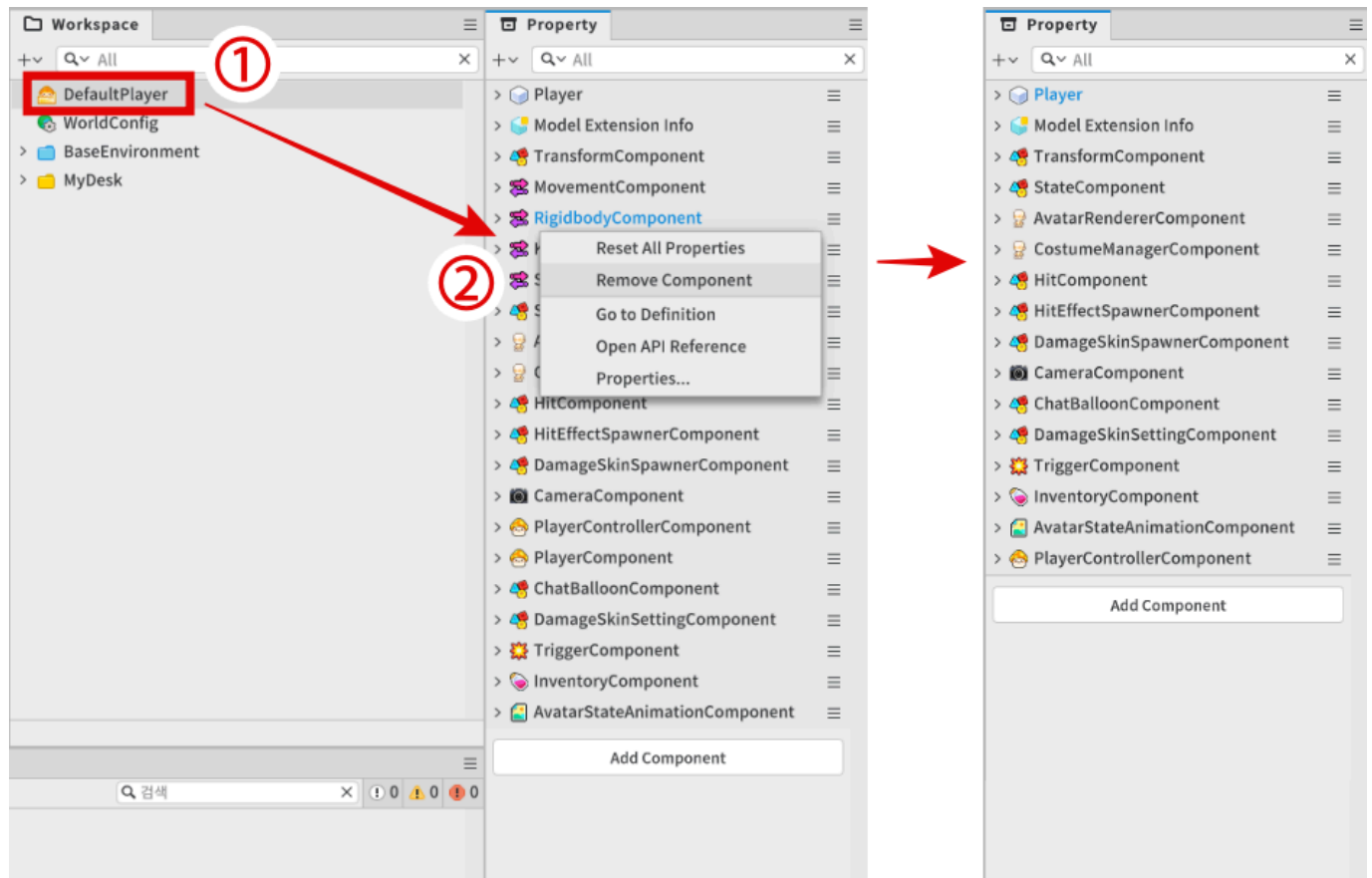
- 메이플스토리 월드에서 SideViewRectTileMap 방식을 사용할 경우, 캐릭터의 물리를 제어합니다.
- 해당 컴포넌트가 존재하지 않을 경우, 캐릭터는 맵의 타일들과 상호작용을 할 수 없습니다.
- 캐릭터의 점프 속도, 중력 가속도 등을 조절할 수 있습니다.

맵 형식 변경하기



- Hierarchy 창에서 변경하려는 맵을 우클릭합니다.
- Switch To RectTileMap, Switch To SideViewRectTileMap중 하나를 클릭합니다.
 - 만약 현재 맵이 TileMap이 아닐 경우, Switch To TileMap이 출력되며, 현재 맵 상태로의 전환이 숨김 처리됩니다.
- 러닝플랫폼어 게임 제작을 위해, Switch To SideViewRectTileMap을 클릭하여 맵을 SideViewRectTileMap으로 전환합니다.

DefaultPlayer의 기존 Component제거하기

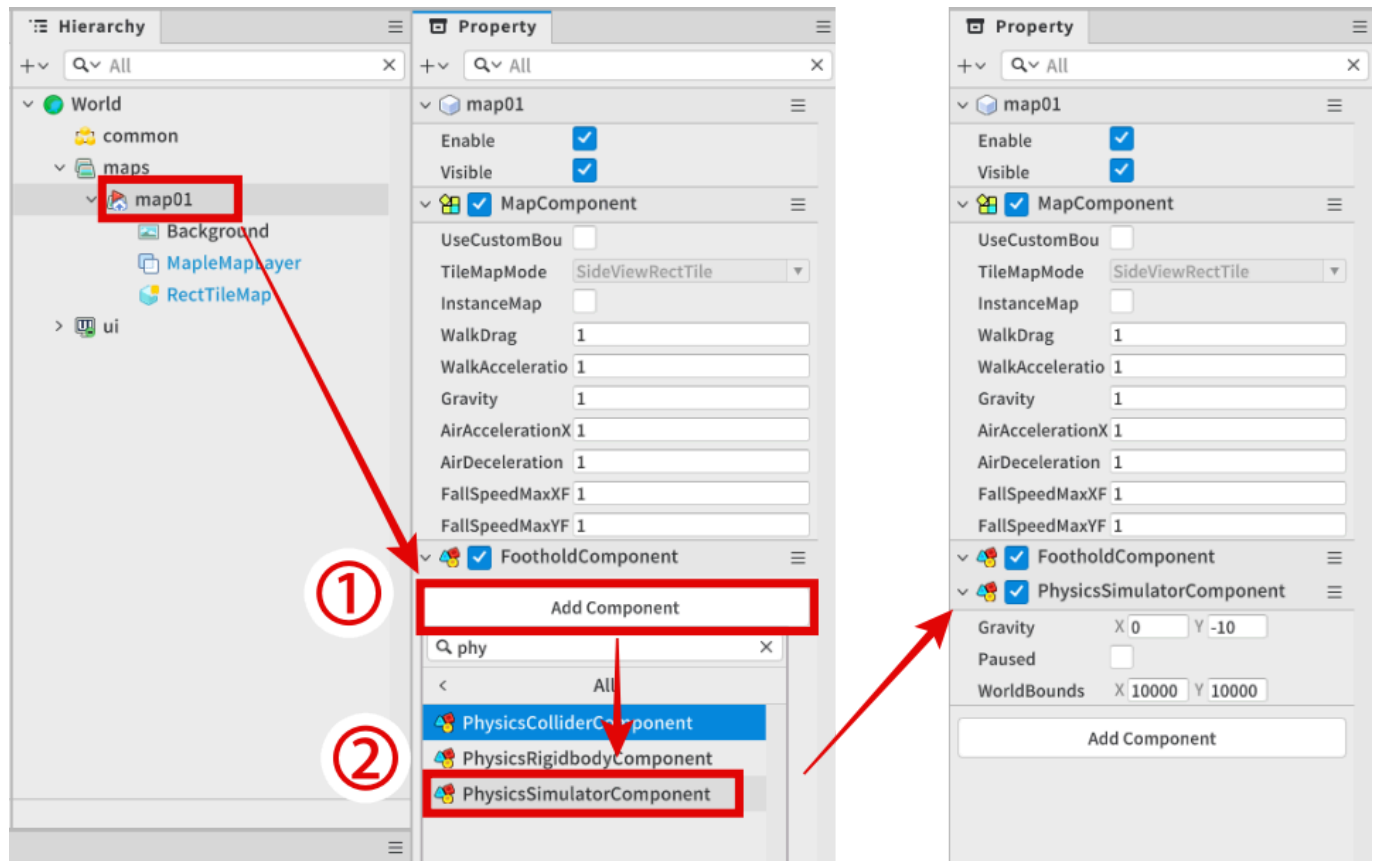


1. 플레이어가 조작할 캐릭터의 템플릿이 될 Workspace에서 DefaultPlayer(Model)를 선택합니다.
2. Property탭에서 새로운 물리 및 상호작용 기능의 간섭을 방지하기 위해 기존 대상 컴포넌트를 Remove Component로 제거합니다.(Rigidbody, Kinematicbody, Sideviewbody, Movement, Player)

Physics 컴포넌트를 활용한 물리 시스템 적용

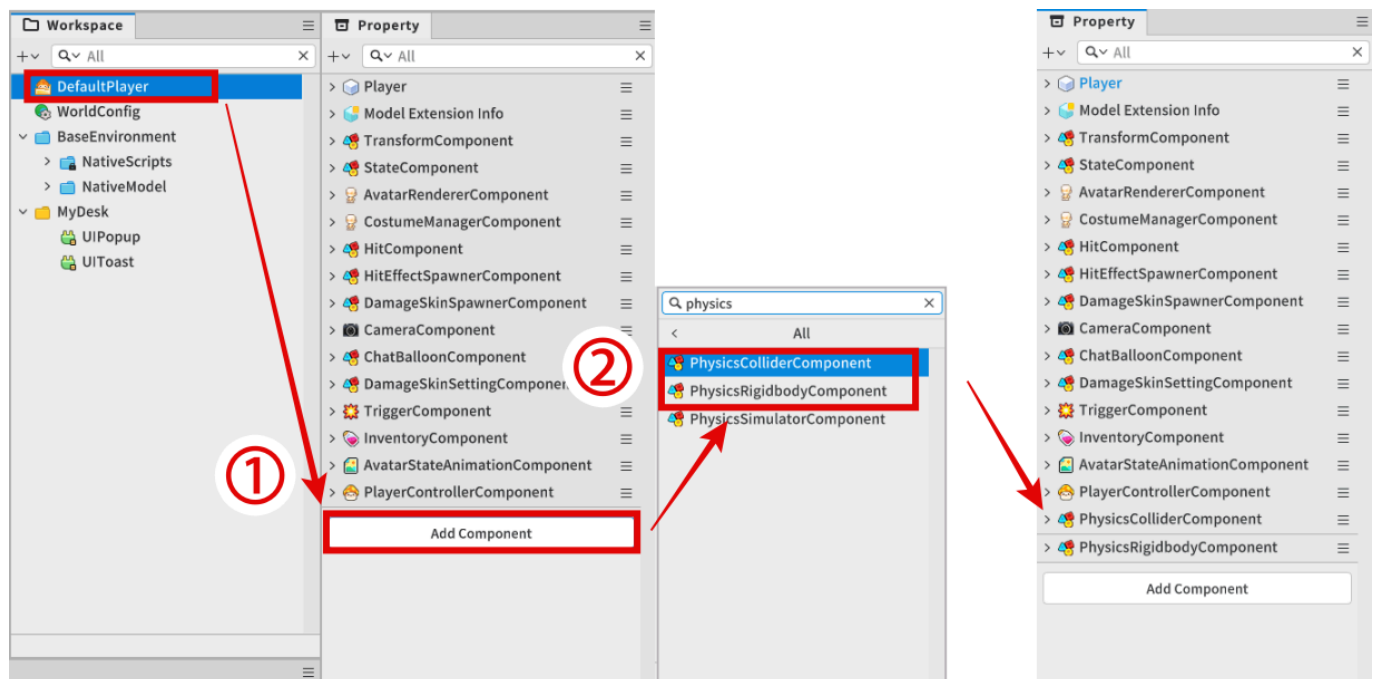
- 러닝플랫폼 게임특성상 바닥 타일을 끊임없이 생성하고 파괴해야 합니다. 따라서 관리를 원활하게 하고 독립적인 기능을 부여하기 위해, 바닥 타일 자체를 하나의 온전한 엔티티로 구현합니다.
- TileSet으로 제작된 지형이 아니므로 별도의 Physics 설정이 필요합니다.

맵 엔티티에 PhysicsSimulator 등록하기



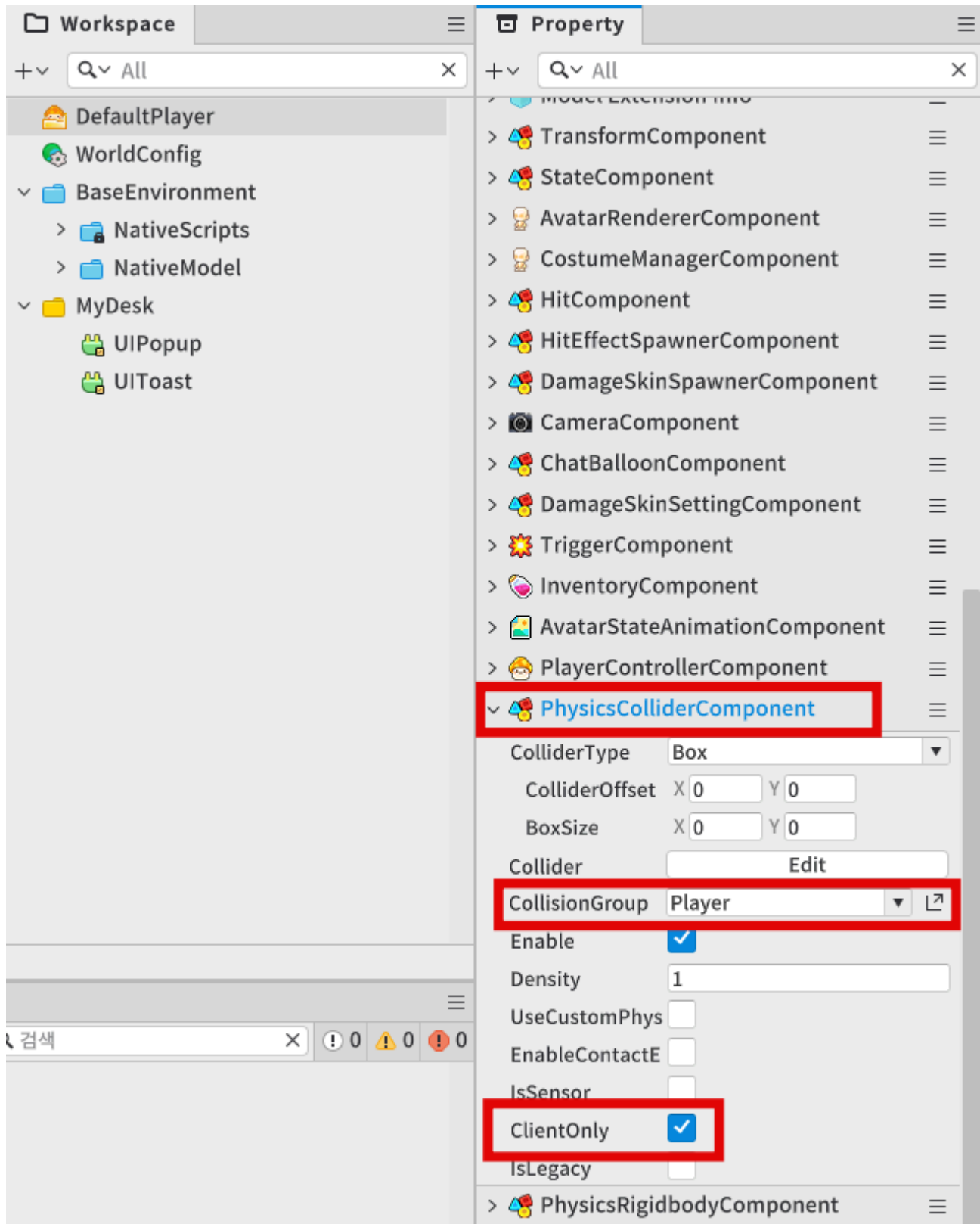
1. Hierarchy탭에서 Map엔티티를 선택한 후 AddComponent버튼을 클릭합니다.
2. PhysicsSimulatorComponent를 등록합니다.

캐릭터 PhysicsComponent 등록하기



1. 원본 DefaultPlayer를 선택하고 Property탭에서 AddComponent버튼을 클릭합니다.
2. PhysicsCollider, PhysicsRigidbodyComponent를 등록합니다.

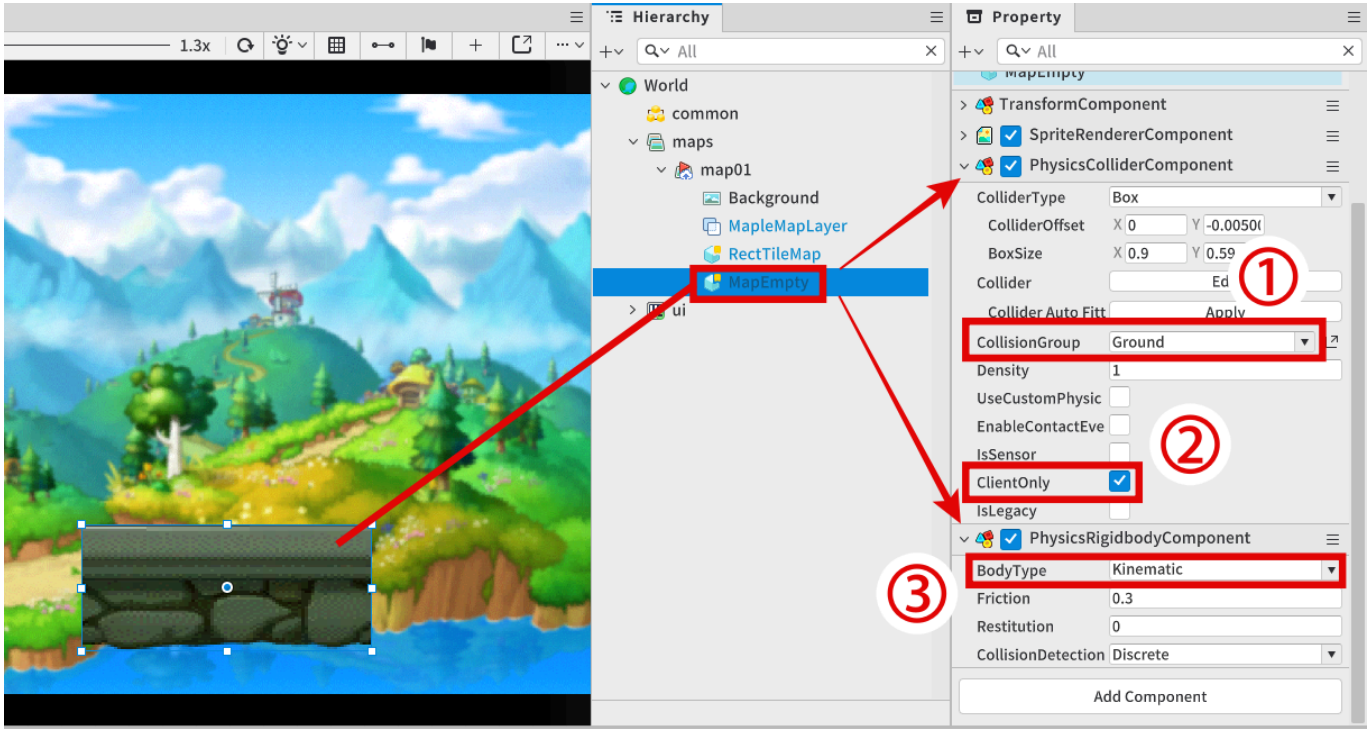
PhysicsColliderComponent 설정하기



- 충돌 대상을 설정하기 위해 CollisionGroup을 'Player'그룹을 추가해 설정합니다.
- 다른 유저와 상호작용하지 않는 게임이므로 PhysicsColliderComponent의 내부 속성 중에서 ClientOnly항목을 체크해줍니다.

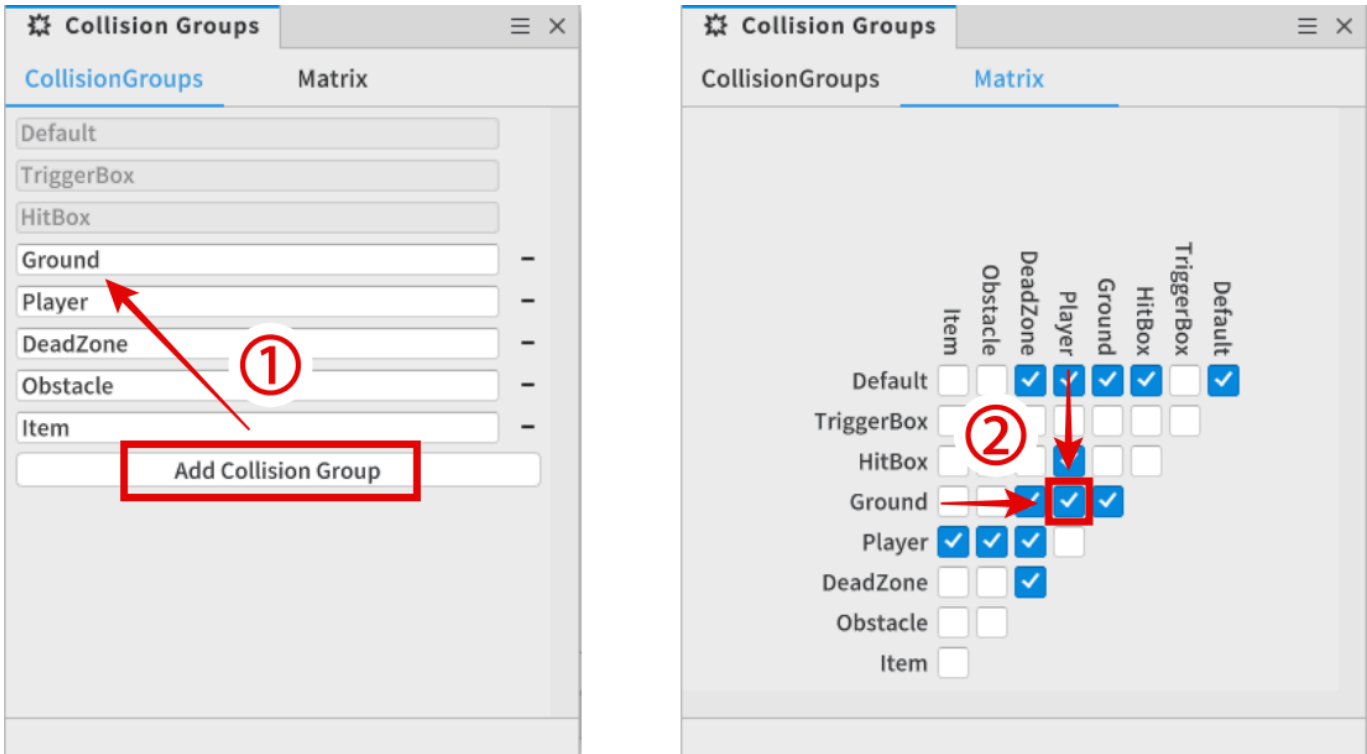
바닥 타일 엔티티 구성 및 상호작용 테스트

테스트 바닥 타일 설정하기



- 테스트로 진행할 엔티티를 생성하고 캐릭터와 마찬가지로 PhysicsRigidbody, PhysicsColliderComponent를 등록합니다.
1. CollisionGroup을 캐릭터와 상호작용하기 위한 Ground로 설정합니다.
 2. 캐릭터와 마찬가지로 로컬환경에서만 동작되므로 PhysicsColliderComponent 내부 속성 중에서 ClientOnly항목을 체크해줍니다.
 3. 바닥으로 활용할 엔티티는 정적인 상태로 이동처리만 수행되어야 하므로 PhysicsRigidbodyComponent 내부 속성 중에서 BodyType을 Kinematic으로 설정합니다.

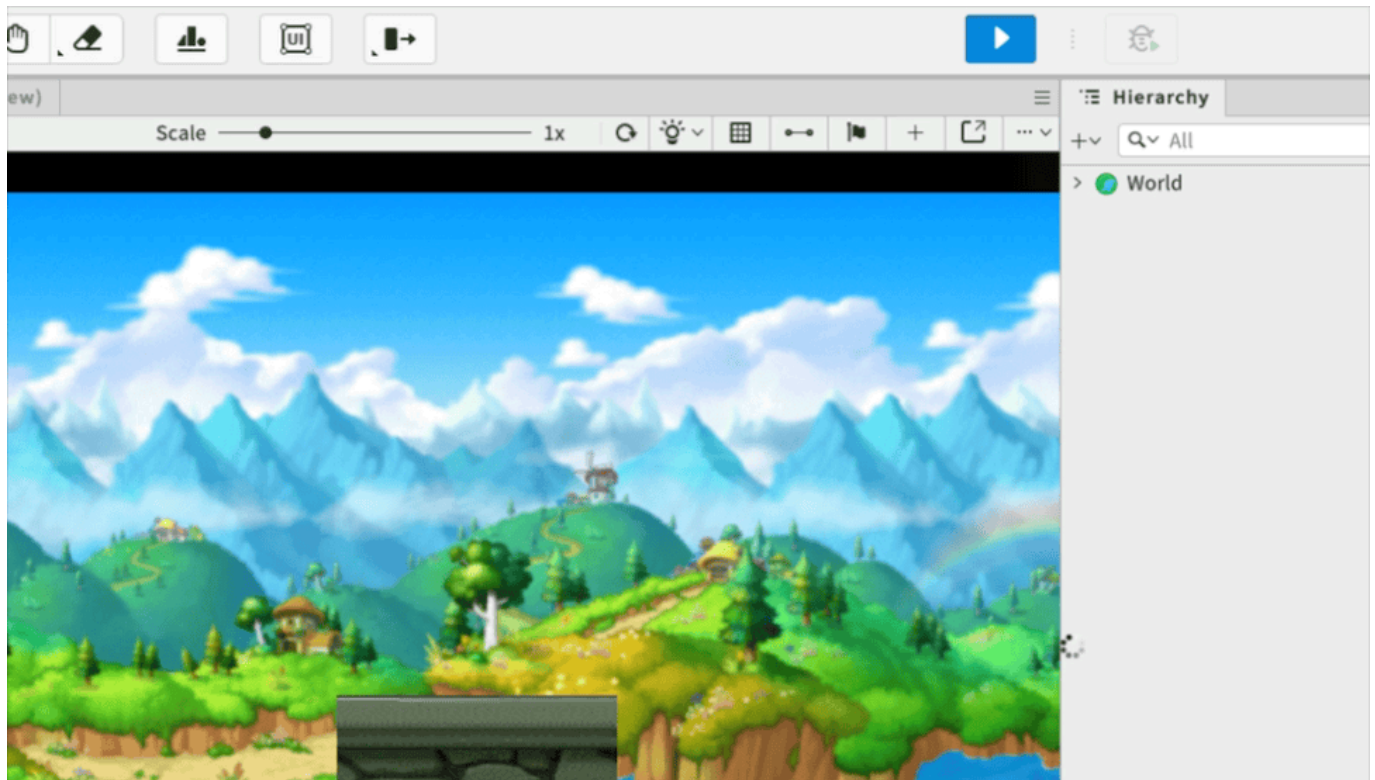
CollisionGroup 추가 및 설정하기



1. CollisionGroup의 Add Collision Group을 통해 새로운 CollisionGroup 항목을 생성할 수 있습니다.

2. Player와 Ground의 충돌을 감지하기 위해선, Matrix 탭에서 Player와 Ground의 선상에 체크표시가 활성화되어 있는지 확인합니다.

상호작용 테스트하기



- 에디터 상단의 플레이어튼 클릭을 통해 실행되는 월드 환경에서 캐릭터가 정상적으로 중력의 영향을 받으며 타일 엔티티의 위에 서있는지 확인합니다.

최소 구현 기준

- 캐릭터가 정상적으로 달릴 수 있는 러닝플랫폼 게임의 맵을 구성합니다.
 - 캐릭터가 허공에 떠있거나, 바닥 타일을 뚫고 지나가지 않아야 합니다.

응용 및 확장 방향

- 플레이할 캐릭터의 TransformComponent의 Scale값을 조절하여 캐릭터의 크기를 조절하고, PhysicsCollider의 collider Edit 버튼을 통해 콜라이더 사이즈를 수정할 수 있습니다. 이러한 설정을 통해 게임난이도를 직관적으로 설정할 수 있습니다.

[💡 기초 학습]Logic 이해하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- Logic 이해하기: 00:13:54

학습 목표

- 메이플스토리 월드에서 전역(Global)으로 동작하는 Logic의 핵심 개념을 이해하고, 일반 엔티티에 종속되는 컴포넌트와의 차이점을 명확히 구분할 수 있습니다.
- 게임 내 다양한 시스템을 중앙 집중식으로 통제하는 '매니저'의 필요성을 파악하고, 이를 통해 프로젝트의 코드 복잡도를 낮추고 유지보수성을 높이는 방법을 학습합니다.
- 러닝 플랫폼의 뼈대가 되는 GameLogic, DataSetLogic, UIManagerLogic을 설계하고, 각 로직이 유기적으로 데이터를 주고받는 구조를 구현할 수 있습니다.

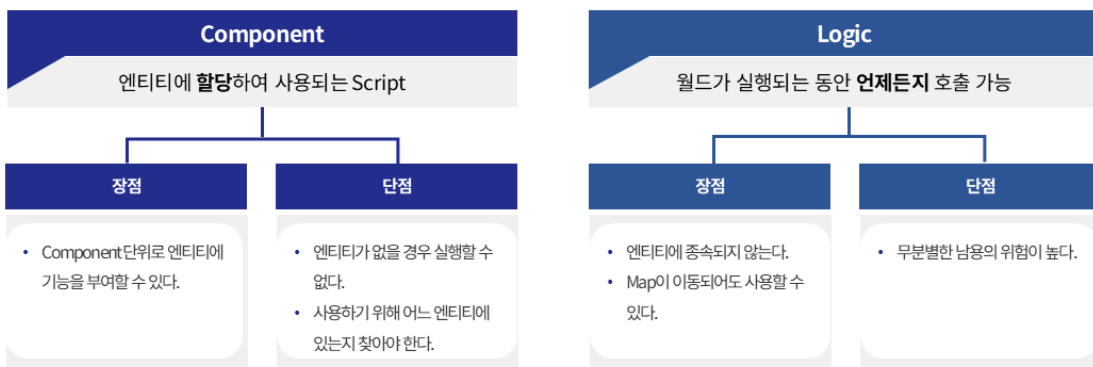
해당 영상 시청 후 수행해 볼 내용

- 집중적으로 관리할 수 있는 기능들이 무엇이 있는지 분석하고, 이를 Logic 스크립트로 제작해 봅니다.
- 제작된 Logic 스크립트를 외부(Logic 외 다른 컴포넌트)에서 호출이 정상적인지 테스트해 봅니다.

Logic이란?

Key Point

엔티티의 종속 여부에 따른 Script 차이점

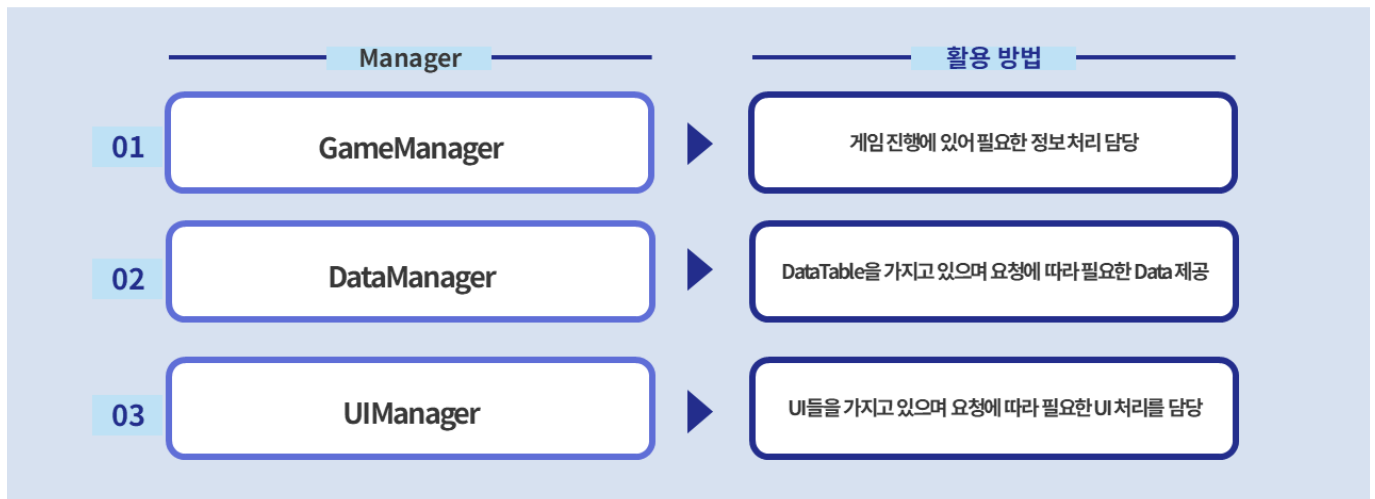


- 메이플스토리 월드가 제공하는 스크립트의 형태 중 하나입니다.
- Logic을 생성할 경우, Workspace 내에서 초록색 플러그 형태의 아이콘으로 표기됩니다.
- 또한 Component와 달리, 엔티티에 부여하지 않아도 스크립트에 접근할 수 있습니다.

Manager 개념

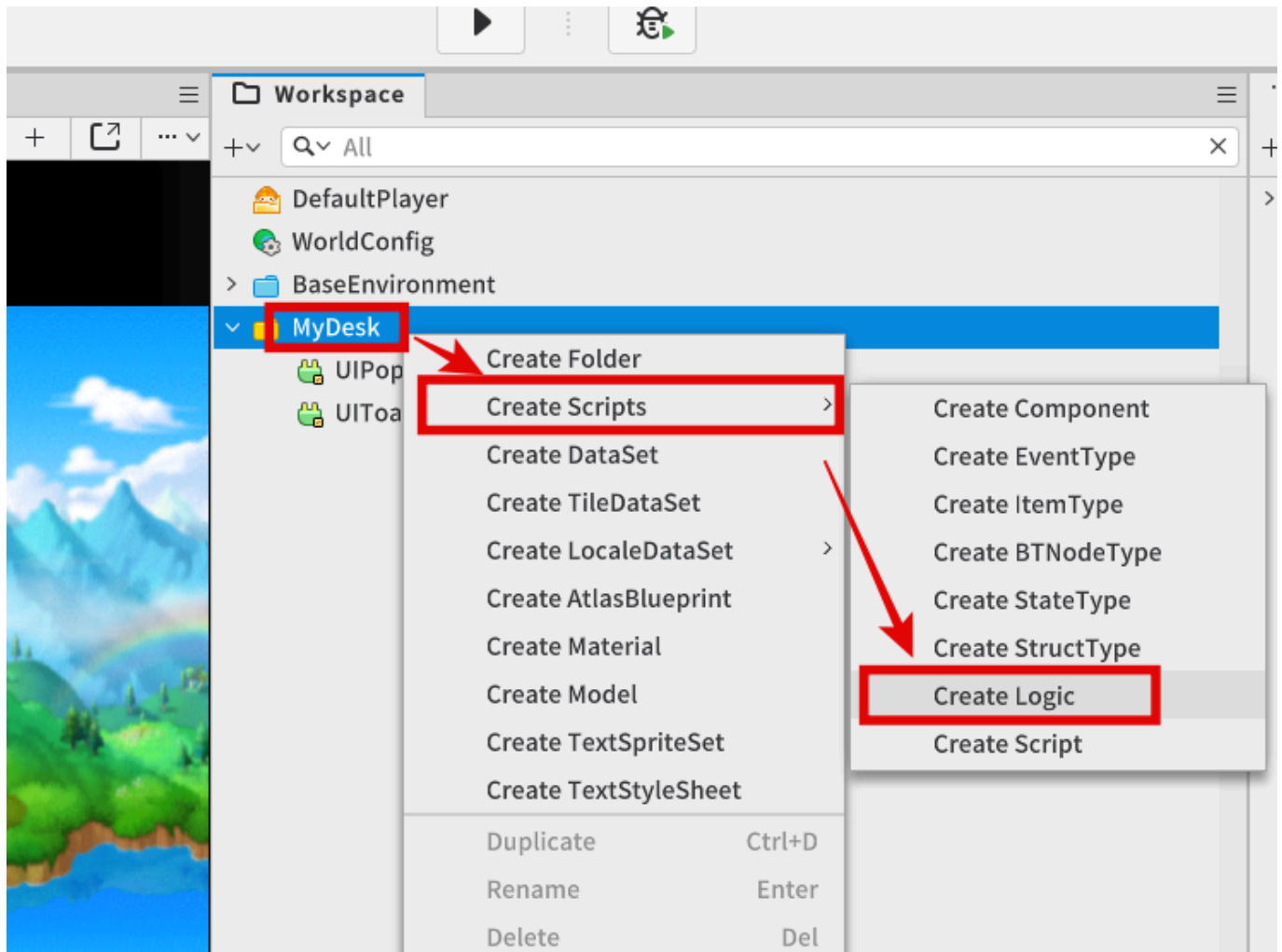
Key Point

Manager를 통해 특정 기능을 담당하도록 구성



- Logic은 특정한 기능이나 역할을 지정하고, 해당하는 기능을 수행하도록 담당자 형태로 제작할 수 있습니다.
- 하나의 기능이나 역할을 담당하는 담당자를 Manager라고 칭합니다.
- 만약 특정 기능에 문제가 생겼을 경우, Manager의 내용만 수정하여, 기능을 빠르고 편하게 해결할 수 있습니다.
 - Case 1: 캐릭터가 데미지 계산을 완료하고 몬스터에게 전달하는 경우
 - 데미지 계산식이 변경될 때마다, 데미지 계산에 신규 요소가 추가될 때마다 계산식을 변경해야 합니다.
 - 모든 캐릭터의 코드를 일일이 찾아 수정해야 합니다.
 - Case 2: 데미지 계산을 Logic으로 제작한 Manager에게 요청하는 경우
 - Logic 내에 있는 데미지 계산식을 변경합니다.
 - 모든 캐릭터, 몬스터의 데미지 계산을 Logic이 담당하므로 더 이상 수정할 요소가 없습니다.
 - 추후 데미지 계산식에 문제가 생겨도, Manager를 수정하면 해결할 수 있습니다.

Logic 제작 방법



- Workspace에서 MyDesk를 우클릭합니다.
- 메뉴 화면에서 Create Scripts를 클릭합니다.
- Create Logic을 클릭하여 새로운 Logic을 생성합니다.

샘플 월드 내 Manager Logic의 사용 용도

이름	사용 용도
GameLogic	게임의 진행에 따라 처리해야 하는 기능들을 관리하기 위해 제작된 Manager Logic입니다. 체력 및 점수제어, 게임시작, 게임종료, 유저 데이터 저장 등 게임 진행에 있어 필요한 처리를 담당합니다.
DataSetLogic	DataSet을 관리하고 사용하기 위한 Logic입니다. 데이터가 필요할 때마다 필요한 DataSet을 일일이 로드하기엔 번거롭습니다. 게임이 시작되면 DataTableLogic이 모든 DataSet을 캐싱하며 필요할 때마다 캐싱된 데이터를 반환합니다.
UIManagerLogic	UI를 관리하기 위한 Manager Logic입니다. 컴포넌트 또는 GameLogic이 UI와 관련된 처리가 필요할 경우, 해당 Manager Logic에게 요청을 전달합니다. UIMangerLogic은 모든 UI들의 컴포넌트를 가지고 있으므로, UI와 관련된 모든 요청을 전담합니다.

- 위와 같은 형태로 샘플 월드는 Logic을 생성하고, 사용하고 있습니다.
- 추가적인 기능 제작 또는 확장이 필요할 때, 여러분들만의 Logic을 선언하고 제작하여 사용할 수 있습니다.

샘플 월드 내 Logic 사용 예시

GameLogic의 경우

- 게임이 시작되면, 지속적해서 체력을 감소하며 점수를 증가시키는 기능을 처리하고 있습니다.
- 관리되고 있는 체력이 0이 될 경우 결과 화면을 요청하며 현재 점수와 기존 최고 점수를 비교하여 유저의 최고 점수를 갱신합니다.
- 본격적인 게임이 진행될 때, 일정시간마다 스테이지의 전환을 위한 타이머 기능을 수행합니다.

DataSetLogic의 경우

- 메이플스토리 월드의 내장 기능인 DataService를 활용해 아이템, 타일, 캐릭터 정보 등의 원본 데이터를 읽어옵니다.
- 읽어온 데이터를 코드에서 이름만 대면 바로 찾을 수 있도록 꼬리표(Key)를 달아, 알아보기 쉬운 Lua 테이블 변수 형태로 정리해 두는 역할을 합니다.

UIManagerLogic의 경우

- 타이틀 화면, 캐릭터 선택 창, 체력 및 점수 표시, 결과 팝업 등 게임 내 존재하는 UI요소의 참조 경로를 중앙에서 통합하여 관리합니다.
- 각 UI요소를 찾기 위한 복잡한 경로를 하나로 모아두어, 다른 스크립트에서 특정 UI 조작이 필요할 때 간편하게 접근할 수 있도록 돕습니다.
- 엔티티 참조로 설정한 상태에서 게임 상태(시작,플레이,종료)에 따라 필요한 UI 그룹만 일괄적으로 켜고 끄는 팝업 제어 기능을 수행합니다.

GameLogic 코드 작성하기

```
@Logic
script GameLogic extends Logic

property number discript = 0 -- 게임의 전체적인 흐름을 제어하기 위한 로직입니다.

property ScrollingController scrollingController = nil

property number currentScore = 0 -- 현재 게임 점수값을 담기 위한 변수입니다.

@Sync
property number bestScore = 0 -- 플레이어의 최고 점수를 담기 위한 변수입니다.

property number scoreIncreaseTimer = 0 -- 주기적으로 점수를 갱신하기 위한 타이머 변수입니다.

property number currentStage = 1 -- 현재 스테이지 번호를 저장하기 위한 변수입니다.

property number stageTimer = 0 -- 스테이지가 전환 될 타이밍을 측정하기 위한 타이머 변수입니다.

property number hp = 0 -- 플레이어의 체력값을 저장하기 위한 변수입니다.
```

property boolean isEnd = true -- 게임이 종료되었음을 기록하는 상태 변수입니다.

property boolean isMobile = true -- 구동환경이 모바일 환경인지 상태를 저장하는 변수입니다.

```
@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 월드 시작 시 호출되는 메서드입니다.
```

```
-- 게임 로직의 초기화를 진행합니다.
self:Init()
```

```
-- 월드 진행이 모바일로 진행되는지 상태변수로 저장합니다.
self.isMobile = Environment:IsMobilePlatform()
```

```
-- 월드 시작 시 배경음악을 재생합니다.
_SoundService:PlayBGM("f5bb92d91e134c699e7bb2cf1aaca8e6",0.4)
end
```

```
@ExecSpace("ClientOnly")
method void Init()
-- 전반적인 게임을 관리하는 메인 로직의 초기화를 진행합니다.
```

```
-- 내부 멤버 참조를 연결합니다.
self:ConnectRefs()
```

```
-- 다른 핵심 로직들의 초기화를 진행합니다.
self:OrderInit()
-- 점수값을 초기화합니다.
self:SetScore(0)
-- 현재 체력값을 초기화합니다.
self:SetHP(100)
end
```

```
@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 내부 멤버의 참조를 연결하는 메서드입니다.
```

```
-- 스크롤링 컨트롤러 엔티티를 연결합니다.
self.scrollingController =
_EntityService:GetEntityByPath("/maps/MainGame/Scrolling").ScrollingController
end
```

```
@ExecSpace("ClientOnly")
method void OrderInit()
-- 게임 로직으로부터 각 핵심 로직, 컴포넌트들의 초기화 명령을 진행합니다.
```

```
-- 스크롤링 컨트롤러 초기화를 진행합니다.
self.scrollingController:Init()
-- 데이터셋 로직 초기화를 진행합니다.
_DataSetLogic:Init()
-- UI매니저로직 초기화를 진행합니다.
```

```

_UIManagerLogic:Init()
-- 오브젝트 풀 로직 초기화를 진행합니다.
_ObjectPoolLogic:Init()
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 매 프레임마다 호출되는 메서드입니다.

-- 게임이 진행중이 아니라면
if self.isEnd == true then
    -- 메서드를 종료합니다.
    return
end

-- delta 값만큼 자동으로 체력을 감소합니다.
self.AutoDecreaseHp(delta)
-- delta 값만큼 자동으로 점수를 증가합니다.
self.AutoIncreaseScore(delta)

-- /분기1/ 스테이지 타이머의 남은 시간이 있다면
if self.stageTimer > 0 then
    -- 스테이지 타이머의 값에 delta값을 감소합니다.
    self.stageTimer -= delta
-- /분기2/ 스테이지 타이머에 남은 시간값이 없다면
else
    -- 스테이지 전환을 호출합니다.
    self.StageChange()
    -- 스테이지 타이머 값을 Config에 설정한 값으로 초기화합니다.
    self.stageTimer = _Config.stageChangeTime
end

end

@ExecSpace("ClientOnly")
method void AutoIncreaseScore(number delta)
-- increaseScoreIntervalSecond(주기) 마다 점수를 증가시키는 메서드입니다.

-- /분기1/ 측정 타이머의 값이 기준이 되는 Interval 값보다 작다면
if self.scoreIncreaseTimer < _Config.increaseScoreInterval then
    -- 타이머에 delta값을 추가시킨뒤 메서드 종료합니다.
    self.scoreIncreaseTimer += delta
    return
end

-- /분기2/ 측정 타이머의 값이 기준이 되는 Interval 값보다 크거나 같다면 (특정 주기 도달시점입
니다.)

-- 점수를 추가합니다.
self.AddScore(_Config.increaseScore)
-- 측정 타이머를 초기화합니다.
self.scoreIncreaseTimer = 0
end

```

```

@ExecSpace("ClientOnly")
method void AutoDecreaseHp(number delta)
-- 자동으로 체력을 감소시키는 메서드입니다.

-- 현재 체력값에 감소되어야하는 값을 전달합니다.
self:AddHP(delta * _Config.decreaseHpValue)
end

@ExecSpace("ClientOnly")
method void SetScore(number value)
-- 매개변수로 전달받은 값을 Score 값으로 설정시키는 메서드입니다.

-- 매개변수 1 number value : 점수값으로 설정되는 값입니다.

-- 현재 점수값을 특정값으로 설정합니다.
self.currentScore = value
-- UIManager에 등록된 Text에 특정값으로 설정합니다.
_UIManagerLogic:SetScoreText(value)
end

@ExecSpace("ClientOnly")
method void AddScore(number addValue)
-- 매개변수로 전달받은 값을 Score 값에 추가시키는 메서드입니다.

-- 매개변수 1 number addValue : 기존 점수값에 추가되는 값을 전달받습니다.

-- 현재 점수에 전달받은 값을 추가시킵니다.
self.currentScore += addValue
-- UIManager 에 등록된 Text에 현재 점수값을 전달합니다.
_UIManagerLogic:SetScoreText(self.currentScore)
end

@ExecSpace("ClientOnly")
method void SetHP(number value)
-- 매개변수로 전달받은 값을 Hp 값으로 설정시키는 메서드입니다.

-- 매개변수 1 number value : Hp 값으로 설정되는 값을 전달받습니다.

-- 체력값을 특정값으로 설정합니다. (clamp 를 활용해 미만, 초과를 제한합니다.)
self.hp = math.clamp(value,0,_Config.maxHp)

-- UIManager에 등록된 슬라이더에 체력값을 반영합니다.
_UIManagerLogic:SetHpSliderValue(self.hp)
end

@ExecSpace("ClientOnly")
method void AddHP(number addValue)
-- 매개변수로 전달받은 값을 체력값에 추가시키는 메서드입니다.

-- 매개변수 1 number addValue : 추가되는 체력값을 전달받습니다.

-- 현재 체력에 매개변수값을 추가합니다.
self.hp += addValue

```

```

-- clamp 를 활용해 미만, 초과를 제한합니다.
self.hp = math.clamp(self.hp,0,_Config.maxHp)

-- UIManager에 등록된 슬라이더에 체력값을 반영합니다.
_UIManagerLogic:SetHpSliderValue(self.hp)

-- 체력값이 0보다 작거나 같다면
if self.hp <= 0 then
    -- 게임 종료 로직을 수행합니다.
    self:EndMainGame()
end
end

@ExecSpace("Client")
method void EndMainGame()
-- 게임을 종료하는 메서드입니다. (클라이언트전용 메서드)

-- 게임이 종료되었음 상태로 전환합니다.
self.isEnd = true

-- 캐릭터를 보이지 않게 설정합니다.
_UserService.LocalPlayer.Visible = false

-- 게임종료 시 구동환경이 모바일이라면 조이스틱을 비활성화합니다.
_UIManagerLogic:ShowMobileJoyStick(false)

-- 결과창 팝업내 설정될 값을 지정하고 화면 내 출력합니다. (최고 점수와, 현재 점수를 전달합니다.)
_UIManagerLogic:ShowResultPopup(tostring(math.floor(self.bestScore)),tostring(math.floor(self.currentScore)))

-- 스크롤링 컨트롤러에게 모든타일셋 오브젝트 반환(비활성화)을 진행합니다.
self.scrollingController:ReturnAllTiles()

-- 배경 스크롤링 속도를 감소시킵니다.
self.scrollingController.background.ScrollRate = 0.5

-- 현재 점수가 클라이언트 내에서 관리되고 있는 최고 점수보다 높다면
if self.currentScore > self.bestScore then
    -- 최고 점수값을 현재 점수로 설정합니다.
    self.bestScore = self.currentScore
    -- 최고 점수가 변경된 시점으므로 서버측의 최고 점수값을 갱신합니다.
    self:UpdateScore(self.bestScore)
end

-- 메인타이틀의 최고 점수값을 반영합니다.
_UIManagerLogic:UpdateTitleBestScore(self.bestScore)
end

@ExecSpace("ClientOnly")
method void GameStart(number selectCharacterId)
-- 메인게임을 시작할때 호출되는 메서드입니다.

-- CharacterInfoTable에서 캐릭터 선택 창에서 선택했던 캐릭터의 속도값을 가져옵니다.

```

```
local scrollSpeed =
_DataSetLogic.characterInfoTable[selectCharacterId].Speed
-- 기본 속도값을 등록합니다.
self:SetDefaultScrollSpeed(scrollSpeed)

-- 현재(시작) 스테이지를 1로 초기화합니다.
self.currentStage = 1

-- 스테이지 변경을 요청합니다.
self:StageChange()

self:SetScore(0)

-- 점수 타이머값을 초기화합니다.
self.scoreIncreaseTimer = 0

-- 체력값을 초기화합니다.
self:SetHP(100)

-- 스테이지 타이머를 config에 설정된 전환시간으로 설정합니다.
self.stageTimer = _Config.stageChangeTime

-- 모바일환경인 경우 모바일 조이스틱을 표기합니다.
_UIManagerLogic:ShowMobileJoyStick(true)

-- 스크롤링 컨트롤러의 초기화를 진행합니다.
self.scrollingController:Reset()

-- 플레이어 캐릭터의 초기화를 진행합니다.
_UserService.LocalPlayer.CustomPlayerController:ResetCharacter()

-- 유저에게 제공되어야할 UI가 보이도록 설정합니다.
_UIManagerLogic:ShowPlayerInfo(true)

-- 스킬 로직의 초기화를 진행합니다.
_SkillLogic:Reset()

-- 스킬 이펙스 리소스 버퍼링을 방지하기 위해 리소스 이펙트만 재생합니다.
_SkillLogic:PlaySkillEffect()

-- 캐릭터 마다 가진 고유 스킬 아이콘으로 이미지를 적용합니다.
_UIManagerLogic:RegistSkillIcon()

-- 아이템 로직의 초기화를 수행합니다.
_ItemLogic:Reset()

-- 본게임 시작전 3,2,1 카운트를 수행합니다. (Wait 메서드가 포함되어 있습니다.)
_UIManagerLogic:CountTimerStart()

-- 게임이 진행중인 상태로 전환합니다.
self.isEnd = false

end
```

```

@ExecSpace("ClientOnly")
method void SetDefaultScrollSpeed(number speed)
-- 캐릭터의 기본 스크롤링 속도값 설정합니다.
-- Defalut 와 Currrent를 나눈 이유 : 부스터 아이템 사용시 속도값이 변경되므로 캐릭터의 기본
속도값을 따로 저장하기 위함입니다.

-- 매개변수 1 number speed : 게임 진행시 적용될 스크롤링 속도값을 전달받습니다.

-- 스크롤링 컴포넌트의 기본 속도값을 매개변수 값으로 설정합니다.
self.scrollingController.defaultSpeed = speed
-- 스크롤링 컴포넌트의 현재 속도값을 매개변수 값으로 설정합니다.
self.scrollingController.currentScrollSpeed = speed
end

@ExecSpace("ClientOnly")
method void StageChange()
-- 스테이지를 변경시키는 메서드입니다.

-- 스테이지 번호를 1증가시키지만, % 연산자를 활용해 리소스 요소를 순회할 수 있게 설정합니다.
self.currentStage = (self.currentStage %
(_DataSetLogic.stageResourceCount+1)) + 1

-- 현재 설정된 스테이지 번호에 맞는 배경화면으로 전환합니다.
self.scrollingController:ChangeBackground()
end

@ExecSpace("Server")
method void LoadData()
-- 기록된 정보를 가져오는 메서드입니다. (서버전용 메서드)
-- 서버측 실행 : 서버에서 관리되고 있던 데이터를 가져오기 위함입니다.

-- 데이터스토리지 서비스를 활용해 'MyData'라는 데이터 스토리지를 가져옵니다.
local data = _DataStorageService:GetGlobalDataStorage("MyData")

-- 해당 데이터 스토리지 내에서 'BestScore' 키에 해당하는 Value 값을 가져옵니다.
local errorCode,value = data:GetAndWait("BestScore")

-- 임시 점수값 변수를 등록합니다.
local tempScore

-- 만약 저장된 데이터가 없다면
if value == nil then
-- 임시 점수값을 0으로 설정합니다.
tempScore = 0

-- 만약 저장된 데이터가 있다면
else
-- 임시 점수값을 기존 점수값으로 설정합니다.
tempScore = tonumber(value)
end

-- bestScore값을 갱신합니다.
-- Sync로 선언된 프로퍼티로써 값 변경이 일어날때,
-- 클라이언트쪽에서도 값이 동기화되며 OnSyncProperty 메서드가 자동으로 호출됩니다.

```

```

self.bestScore = tonumber(tempScore)
end

@ExecSpace("Server")
method void SaveScore(number score)
-- 유저의 정보를 저장하는 메서드입니다.

-- 데이터 스토리지에 점수를 저장하는 메서드입니다. (서버전용 메서드)

-- 데이터스토리지 서비스를 활용해 'MyData'라는 데이터 스토리지를 가져옵니다.
local data = _DataStorageService:GetGlobalDataStorage("MyData")

-- 해당 스토리지 내의 'BestScore' 키에 해당하는 Value 값에 'score' 값을 등록합니다.
local errorCode = data:SetAndWait("BestScore", tostring(score))
end

@ExecSpace("Server")
method void UpdateScore(number score)
-- 서버 측에서 관리되는 점수값을 갱신하는 메서드입니다. (서버전용 메서드)

-- 매개변수 1 number score : 클라이언트 측에서 관리되는 점수값을 전달받습니다.

-- 서버측 bestScore값을 매개변수 값으로 설정합니다.
self.bestScore = score
end

@ExecSpace("ClientOnly")
method void OnSyncProperty(string name, any value)
-- Sync로 선언된 프로퍼티 한정으로 서버 측에서 값 변경이 일어날경우 자동으로 호출되는 메서드입니다.
-- 단, 서버 측에서 변경된 값이 기존 클라이언트 값과 동일하다면 해당메서드는 호출되지 않습니다.

-- 변경된 프로퍼티 변수명이 bestScore라면
if name == "bestScore" then
    -- UIManagerLogic에게 메인 타이틀화면의 최고 점수 텍스트 갱신을 요청합니다.
    _UIManagerLogic:UpdateTitleBestScore(tonumber(value))
end
end

@EventSender("Service", "UserService")
handler HandleUserEnterEvent(UserEnterEvent event)
-- 유저 진입, 입장시 호출되는 메서드입니다.

-- 서버측의 데이터를 불러옵니다.
self:LoadData()

end

@EventSender("Service", "UserService")
handler HandleUserLeaveEvent(UserLeaveEvent event)
-- 유저 이탈시 호출되는 메서드입니다.

-- 서버측으로 최고 점수를 등록하는 메서드를 수행합니다.
self:SaveScore(self.bestScore)

```

```
end
```

```
end
```

DataSetLogic 코드 작성하기

```
@Logic
script DataSetLogic extends Logic

property string discript = "" -- 해당 월드에서 관리되는 데이터들을 관리하기 위한 로직입니다.

property table tileSetInfoTable = {} -- 타일셋들의 데이터들을 담기 위한 테이블입니다.

property number totalTileSetCount = -1 -- 타일 종류 개수를 저장하는 변수입니다.

property table itemInfoTable = {} -- 아이템들의 데이터를 담기 위한 테이블입니다.

property table tilesFloorInfoTable = {[1]={} , [2]={}} -- 타일셋들의 층수 정보를 담는 테이블입니다. / ex table[1] = {2,4} 2,4번 타일셋은 1층부터 시작합니다.

property table characterInfoTable = {} -- 캐릭터 정보를 저장하는 테이블입니다.

property table skillInfoTable = {} -- 스킬 정보를 저장하는 테이블입니다.

property table stageRenderInfoTable = {} -- 스테이지 별 렌더되어야 할 리소스정보를 저장하는 테이블입니다.

property number stageResourceCount = 0 -- 스테이지 리소스 종류 수입니다.

@ExecSpace("ClientOnly")
method void Init()
-- DataSetLogic 컴포넌트 초기화를 진행하는 메서드입니다.

-- 타일셋 테이블을 초기화합니다.
self:InitTileSetInfoTable()
-- 아이템 테이블을 초기화합니다.
self:InitItemInfoTable()
-- 캐릭터 테이블을 초기화합니다.
self:InitCharacterInfoTable()
-- 스킬 테이블을 초기화합니다.
self:InitSkillInfoTable()
-- 스테이지 렌더 테이블을 초기화합니다.
self:InitStageRenderInfoTable()
end

@ExecSpace("ClientOnly")
method void InitTileSetInfoTable()
-- 타일 오브젝트와 관련된 데이터셋을 저장하기 위한 메서드입니다.
```

```

-- 월드스페이스에 저장된 데이터셋의 이름을 로컬로 접근하기 위한 변수입니다.
local dataSetName = "TileSetInfo"

-- 대상 데이터셋의 Row수입니다.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 타일셋 종류 수를 저장합니다.
self.totalTileSetCount = rowCount

-- 각 줄의 데이터를 담기 위한 변수입니다.
local rowData

-- 1 부터 데이터셋의 Row수만큼 순회를 시작합니다.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 타일셋 오브젝트의 첫번째 타일의 층수입니다.
        FirstFloor =
tonumber(_DataService:GetCell(dataSetName,i,"FirstFloor")),
        -- 타일셋 오브젝트의 마지막 타일의 층수입니다.
        LastFloor =
tonumber(_DataService:GetCell(dataSetName,i,"LastFloor")),
        -- 타일셋 오브젝트를 생성하기 위한 원본 엔티티 모델(프리팹)Id입니다.
        EntryID = _DataService:GetCell(dataSetName,i,"EntryID")
    }

    -- rowData로 저장한 데이터들을 테이블에 저장합니다.
    self.tileSetInfoTable[i] = rowData

    -- 지금 등록한 타일셋의 층수를 변수로 저장합니다.
    local firstTileFloor = self.tileSetInfoTable[i].FirstFloor

    -- /분기1/ 만약 첫번째 타일 층수가 1층이라면
    if firstTileFloor == 1 then
        -- 1층 테이블 목록에 저장합니다.
        table.insert(self.tilesFloorInfoTable[1],i)
    -- /분기2/ 첫번째 타일 층수가 1층이 아니라면
    else
        -- 2층 테이블 목록에 저장합니다.
        table.insert(self.tilesFloorInfoTable[2],i)
    end
end

end

@ExecSpace("ClientOnly")
method number GetDataSetRowCount(string Name)
-- 대상 데이터셋의 Row수를 반환합니다.

-- 매개변수1 string Name : 대상 데이터셋의 이름을 전달받습니다.

-- 확인한 데이터셋의 Row수를 반환합니다.

```

```

return _DataService:GetRowCount(Name)
end

@ExecSpace("ClientOnly")
method void InitItemInfoTable()
-- 아이템 데이터셋을 저장하기 위한 메서드입니다.

-- 월드스페이스에 저장된 데이터셋의 이름을 로컬로 접근하기 위한 변수입니다.
local dataSetName = "ItemInfo"

-- 대상 데이터셋의 Row수입니다.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 각 줄의 데이터를 담기 위한 변수입니다.
local rowData

-- 1 부터 데이터셋의 Row수만큼 순회 시작합니다.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 아이템 이름을 저장합니다.
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- 아이템의 RUID 를 저장합니다.
        RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID")),
        -- 아이템이 가지는 효과 고윳값을 저장합니다.
        Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
        -- 아이템 생성 확률을 저장합니다.
        SpawnRate =
        tonumber(_DataService:GetCell(dataSetName,i,"SpawnRate")),
        -- 아이템의 모델(프리팸)의 EntryID 값을 저장합니다.
        EntryID = tostring(_DataService:GetCell(dataSetName,i,"EntryID"))
    }

    -- rowData로 저장한 데이터들을 테이블에 저장합니다.
    self.itemInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitCharacterInfoTable()
-- 캐릭터 데이터셋을 저장하기 위한 메서드입니다.

-- 월드스페이스에 저장된 데이터셋의 이름을 로컬로 접근하기 위한 변수입니다.
local dataSetName = "CharacterInfo"

-- 대상 데이터셋의 Row수입니다.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 각 줄의 데이터를 담기 위한 변수입니다.
local rowData

-- 1 부터 데이터셋의 Row수만큼 순회 시작합니다.
for i = 1 , rowCount , 1 do
    rowData =

```

```

{
    -- 캐릭터 이름을 저장합니다.
    Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
    -- 캐릭터 고유 스크롤링 속도값을 저장합니다.
    Speed = tonumber(_DataService:GetCell(dataSetName,i,"Speed")),
    -- 캐릭터 고유 스킬 Id를 저장합니다.
    SkillId = tonumber(_DataService:GetCell(dataSetName,i,"SkillId")),
    -- 캐릭터에 부착된 Costume 렌더러에 설정될 RUID값을 저장합니다.
    RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID"))
}

-- rowData로 저장한 데이터들을 테이블에 저장합니다.
self.characterInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitSkillInfoTable()
-- 스킬 데이터셋을 저장하기 위한 메서드입니다.

-- 월드스페이스에 저장된 데이터셋의 이름을 로컬로 접근하기 위한 변수입니다.
local dataSetName = "SkillInfo"

-- 대상 데이터셋의 row수를 확인합니다.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 내부 멤버 리소스 종류 수를 저장합니다.
self.stageResourceCount = rowCount

-- 각 줄의 데이터를 담기 위한 변수입니다.
local rowData

-- 1 부터 데이터셋의 Row수만큼 순회를 시작합니다.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 스킬 명을 저장합니다.
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- 스킬 사용 효과 곱값을 저장합니다.
        Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
        -- 스킬 이펙트 RUID값을 저장합니다.
        EffectRUID =
        tostring(_DataService:GetCell(dataSetName,i,"EffectRUID")),
        -- 인게임 UI RUID값을 저장합니다.
        UI_RUID = tostring(_DataService:GetCell(dataSetName,i,"UI_RUID"))
    }

    -- rowData로 저장한 데이터들을 테이블에 저장합니다.
    self.skillInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitStageRenderInfoTable()

```

```

-- 스테이지 별 리소스 정보 데이터셋을 저장하기 위한 메서드입니다.

-- 월드스페이스에 저장된 데이터셋의 이름을 로컬로 접근하기 위한 변수입니다.
local dataSetName = "StageRenderInfo"

-- 대상 데이터셋의 row수를 확인합니다.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 각 줄의 데이터를 담기 위한 변수입니다.
local rowData

-- 1 부터 데이터셋의 Row수만큼 순회 시작합니다.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 스테이지 별 적용할 바닥 타일의 RUID값을 저장합니다.
        TileRUID =
        tostring(_DataService:GetCell(dataSetName,i,"TileRUID")),
        -- 스테이지 별 적용할 장애물 오브젝트의 RUID값을 저장합니다.
        ObstacleRUID =
        tostring(_DataService:GetCell(dataSetName,i,"ObstacleRUID")),
        -- 스테이지 별 적용할 배경화면 RUID값을 저장합니다.
        BackgroundRUID =
        tostring(_DataService:GetCell(dataSetName,i,"BackgroundRUID"))
    }

    -- rowData로 저장한 데이터들을 테이블에 저장합니다.
    self.stageRenderInfoTable[i] = rowData
end
end

end

```

UIManagerLogic 코드 작성하기

```

@Logic
script UIManagerLogic extends Logic

property number discript = 0 -- 전반적인 UI, 객체, 내부 값을 관리하기 위한 로직입니다.

property SliderComponent hpSlider = nil

property Entity toastUIGroup = nil

property TextComponent inGameCurrentScoreText = nil

property Entity resultPopup = nil

property Entity resultBestScoreText = nil

```

```

property Entity currentScoreText = nil

property Entity resultToMenuButton = nil

property Entity mainTitlePanel = nil

property Entity characterSelectPopup = nil

property SpriteGUIRendererComponent skillRenderer = nil

property TextComponent titleBestScore = nil

property Entity inGameScoreText = nil

property TextComponent counterText = nil

property Entity titleGameStartButton = nil

property Entity defaultUIGroup = nil

property Entity mobileSlideButton = nil

property Entity mobileJumpButton = nil

@ExecSpace("ClientOnly")
method void Init()
-- UIManagerLogic의 초기화를 진행하는 메서드입니다.

-- 내부 멤버들의 참조를 연결합니다.
self.ConnectRefs()

-- 초기화 진행시 UI 객체들이 담긴 메인 캔버스 활성화합니다.
self.toastUIGroup.Enable = true
-- 결과창 내 메인메뉴 버튼 클릭시 게임실행 첫 상태로 되돌리는 메서드를 연결합니다.
self.resultToMenuButton.ConnectEvent(ButtonClickEvent, self.ReturnToMenu)
-- 타이틀 화면에서 게임시작 버튼 입력 시 캐릭터 선택창을 띄우는 메서드를 연결합니다.
self.titleGameStartButton.ConnectEvent(ButtonClickEvent, self.ShowCharacterSelectPopup)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 내부에 필요한 멤버들의 참조를 연결하는 메서드입니다.

-- toastUIGroup 엔티티를 연결합니다.
self.toastUIGroup = _EntityService.GetEntityByPath("/ui/ToastGroup")
-- defaultUIGroup 엔티티를 연결합니다.
self.defaultUIGroup = _EntityService.GetEntityByPath("/ui/DefaultGroup")

-- 메인 타이틀 패널 엔티티를 연결합니다.
self.mainTitlePanel =
_EntityService.GetEntityByPath("/ui/ToastGroup/UITitle")
-- 메인 타이틀 화면 최고 점수 텍스트 컴포넌트를 연결합니다.

```

```

self.titleBestScore =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UITitleBestScorePanel/UITitleBestScore").TextComponent
-- 메인 타이틀 화면에서 게임시작버튼 엔티티를 연결합니다.
self.titleGameStartButton =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UIImageStartBtn")

-- 캐릭터 선택창 팝업 엔티티를 연결합니다.
self.characterSelectPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect")

-- 인게임 화면상단의 UI hpSlider 참조를 연결합니다.
self.hpSlider =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UI_HPbar").SliderComponent
-- 인게임 화면상단의 UI ScoreText (점수값) 참조를 연결합니다.
self.inGameCurrentScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UIInGameScore").TextComponent
-- 인게임 화면상단의 UI ScoreText ('Score' 문구만 띄워주는) 참조를 연결합니다.
self.inGameScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UIScoreText")
-- 인게임 스킬 아이콘 컴포넌트를 연결합니다.
self.skillRenderer =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UI_Skill").SpriteGUIRendererComponent
-- 인게임 게임시작 시 시작 대기 타이머 텍스트를 연결합니다.
self.counterText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UICounterText").TextComponent
-- 인게임 , 모바일 환경에서 캐릭터의 점프 기능을 수행하기 위한 조이스틱 버튼 엔티티를 연결합니다.
self.mobileJumpButton =
_EntityService:GetEntityByPath("/ui/DefaultGroup/Button_Jump")
-- 인게임 , 모바일 환경에서 캐릭터의 슬라이드 기능을 수행하기 위한 조이스틱 버튼 엔티티를 연결합니다.
self.mobileSlideButton =
_EntityService:GetEntityByPath("/ui/DefaultGroup/Button_Slide")

-- 결과화면 팝업 엔티티를 연결합니다.
self.resultPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup")
-- 결과창 팝업내 최고 점수 텍스트 엔티티를 연결합니다.
self.resultBestScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UIBestScoreText/UIBestScore")
-- 결과창 팝업내 현재점수 텍스트 엔티티를 연결합니다.
self.currentScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UIScoreText/UIScore")
-- 결과창 팝업내 메뉴버튼 엔티티를 연결합니다.
self.resultToMenuButton =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UISMainMenuBtn")
end

```

```

@ExecSpace("ClientOnly")
method void UpdateTitleBestScore(number score)
-- 메인 타이틀 화면에서 표기되는 최고 점수값을 설정하는 메서드입니다.

-- 매개변수 1 number score : 전달받는 값으로 타이틀의 화면내 점수값을 설정합니다.

-- score 값이 없을 경우
if score == nil or score <= 0 then
    -- 메인 타이틀 화면의 최고 점수 텍스트값을 '0'으로 설정합니다.
    self.titleBestScore.Text = "0"

-- score 값이 있었다면
else
    -- 메인 타이틀 화면의 최고 점수를 전달받은 값으로 등록합니다. math.floor를 활용해 소수점
    이하 부분을 버립니다.
    self.titleBestScore.Text = toString(math.floor(score))

end
end

@ExecSpace("ClientOnly")
method void ShowCharacterSelectPopup()
-- 캐릭터 선택창을 띄우는 메서드입니다.

-- 엔티티 서비스를 활용해 캐릭터 선택 팝업을 가져옵니다.
local characterSelectPopup =
_EntityService.GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect")

-- 팝업 엔티티를 활성화합니다.
characterSelectPopup.Enable = true
end

@ExecSpace("ClientOnly")
method void ShowMobileJoyStick(boolean active)
-- 모바일 환경에서 캐릭터를 제어하기 위한 조이스틱 컨트롤러를 표기하는 메서드입니다.

-- 매개변수 1 boolean active : true 일 경우 활성화 , false 일 경우 비활성화합니다.

-- 만약 현재 플랫폼 환경이 모바일 환경이라면
if _GameLogic.isMobile == true then
    -- 조이스틱 컨트롤러가 담긴 UI 캔버스 활성화, 비활성화를 매개변수 값으로 제어합니다.
    self.defaultUIGroup.Enable = active
end
end

@ExecSpace("ClientOnly")
method void CountTimerStart()
-- 본 게임 실행시 시작전 카운트 다운을 진행하는 메서드입니다.

-- 카운트 텍스트 오브젝트를 활성화합니다.
self.counterText.Entity.Enable = true

-- 카운터 텍스트의 값을 '3'으로 설정합니다.

```

```

self.counterText.Text = "3"
-- 로직 수행을 1초 대기합니다.
wait(1)
self.counterText.Text = "2"
wait(1)
self.counterText.Text = "1"
wait(1)

-- 카운트 다운이 종료되었으므로 카운트 텍스트 오브젝트를 비활성화합니다.
self.counterText.Entity.Enable = false
end

@ExecSpace("ClientOnly")
method void ShowPlayerInfo(boolean active)
-- 인게임 내 플레이어에게 제공되어야할 UI 요소를 활성화, 비활성화 하는 메서드입니다.

-- 매개변수 1 boolean active : true 일 경우 활성화, false 일 경우 비활성화합니다.

-- 인게임 hp 슬라이더의 활성화 상태를 매개변수 값으로 설정합니다.
self.hpSlider.Entity.Enable = active
-- 인게임 현재점수값을 나타내는 텍스트 오브젝트의 활성화 상태를 매개변수 값으로 설정합니다.
self.inGameCurrentScoreText.Entity.Enable = active
-- 인게임 'score' 텍스트 오브젝트의 활성화 상태를 매개변수 값으로 설정합니다.
self.inGameScoreText.Entity.Enable = active
-- 인게임 스킬 아이콘 오브젝트의 활성화 상태를 매개변수 값으로 설정합니다.
self.skillRenderer.Entity.Enable = active
end

@ExecSpace("ClientOnly")
method void RegistSkillIcon()
-- 인게임 내의 현재 캐릭터의 스킬아이콘 등록하는 메서드입니다.

-- 스킬아이콘의 RUID 값을 skillLogic에 등록된 skill정보를 기반으로 테이블에서 RUID 값을 가져옵니다.
self.skillRenderer.ImageRUID =
_DataSetLogic.skillInfoTable[_SkillLogic.skillId].UI_RUID
-- 스킬 사용 가능한 상태로 표기하기 위해 알파값을 최대값으로 설정합니다.
self.skillRenderer.Color.a = 1
end

@ExecSpace("ClientOnly")
method void ShowResultPopup(string bestScore, string currentScore)
-- 결과창 팝업을 띄우는 메서드입니다.

-- 최고 점수 텍스트를 등록합니다.
self.resultBestScoreText.TextComponent.Text = bestScore

-- 현재 점수 텍스트를 등록합니다.
self.currentScoreText.TextComponent.Text = currentScore

-- 결과창 팝업을 활성화합니다.
self.resultPopup.Enable = true;
end

```

```

@ExecSpace("ClientOnly")
method void ReturnToMenu()
-- 게임 종료 후 메인메뉴로 상태를 되돌리는 메서드입니다.

-- 인게임에서 표기되는 UI 요소들을 비활성화합니다.
self.ShowPlayerInfo(false)

-- 캐릭터 선택창 팝업을 비활성화합니다.
self.characterSelectPopup.Enable = false
-- 게임 종료 후 띄우는 결과창 팝업을 비활성화합니다.
self.resultPopup.Enable = false
-- 메인메뉴 타이틀에 띄워야할 패널 오브젝트를 활성화합니다.
self.mainTitlePanel.Enable = true
end

@ExecSpace("ClientOnly")
method void SetScoreText(number value)
-- 매개변수로 전달받은 값을 UI Score 텍스트로 설정시키는 메서드입니다.

-- 매개변수 1 number value : 점수값으로 설정되는 값을 전달받습니다.

self.inGameCurrentScoreText.Text = tostring(math.floor(value))
end

@ExecSpace("ClientOnly")
method number SetHpSliderValue(number value)
-- hp 슬라이더의 값을 매개변수의 값으로 설정하며 현재 슬라이더의 Value 값을 반환하는 메서드입니다.

-- 매개변수 1 number value : 슬라이더로 설정할 값을 전달받습니다.

-- 슬라이더의 값을 전달받은 값으로 설정하며, clamp를 활용해 초기 최대체력값을 초과하지 못하게 설정합니다.
self.hpSlider.Value = math.clamp(value, 0, _Config.maxHp)

-- 현재 슬라이더의 값을 반환합니다.
return self.hpSlider.Value
end

end

```

최소 구현 기준

- 앞으로 사용할 DataSet을 관리하고, 사용하기 위한 Logic을 제작합니다.
 - 샘플 월드 기준 DataSetLogic을 제작하여 DataSet을 관리합니다.
- 게임의 진행 및 전반적인 관리를 수행하는 Logic을 제작합니다.
 - 샘플 월드 기준 GameLogic을 제작하여 게임의 상태에 따른 처리를 구현합니다.

응용 및 확장 방향

- 매 초마다 점수를 추가하는 기능을 자신만의 점수 부여 공식으로 변경하여 게임의 흐름을 변경합니다.
- Logic을 활용하여 **변경 예정이 없는** 고정값을 관리하기 위한 'Config' 로직을 제작하여 사용합니다.
 - EX: 기존에 설정된 최대 체력값을 변경하여 게임의 난이도를 수정합니다.
 - EX: 시간에 따라 지속해서 감소하는 체력을 변경하여 게임의 난이도를 수정합니다.

[💡 기초 학습]스크롤 시스템 구성하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 스크롤 시스템 구성하기: 00:25:46

학습 목표

- 캐릭터가 계속 달리는 듯한 연출을 위해 배경과 타일을 반대 방향으로 이동시키는 스크롤 시스템의 핵심 원리를 이해합니다.
- OnUpdate메서드와 delta시간을 활용하여 프레임에 구애받지 않고 시간당 일정한 속도로 엔티티를 부드럽게 이동시키는 스크립트를 작성합니다.
- 배경화면과 바닥 타일의 속도를 제어해 게임의 난이도를 설정할 수 있습니다.
- 게임의 진행에 따라 스테이지 환경이 동적으로 어떻게 변경되는지 이해합니다.

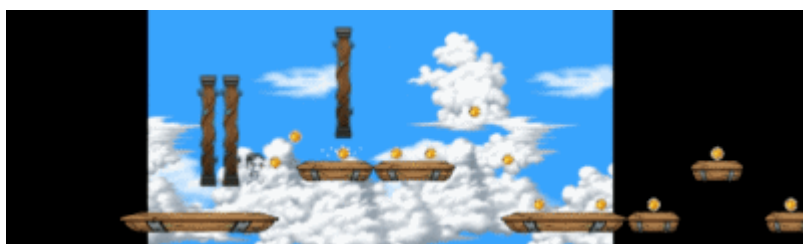
해당 영상 시청 후 수행해 볼 내용

- Map엔티티의 자식 Background엔티티의 BackgroundComponent의 ScrollRate값을 변경하며 적절한 배경 이동 속도로 설정합니다.

스크롤 시스템의 기본 구조 및 원리 이해

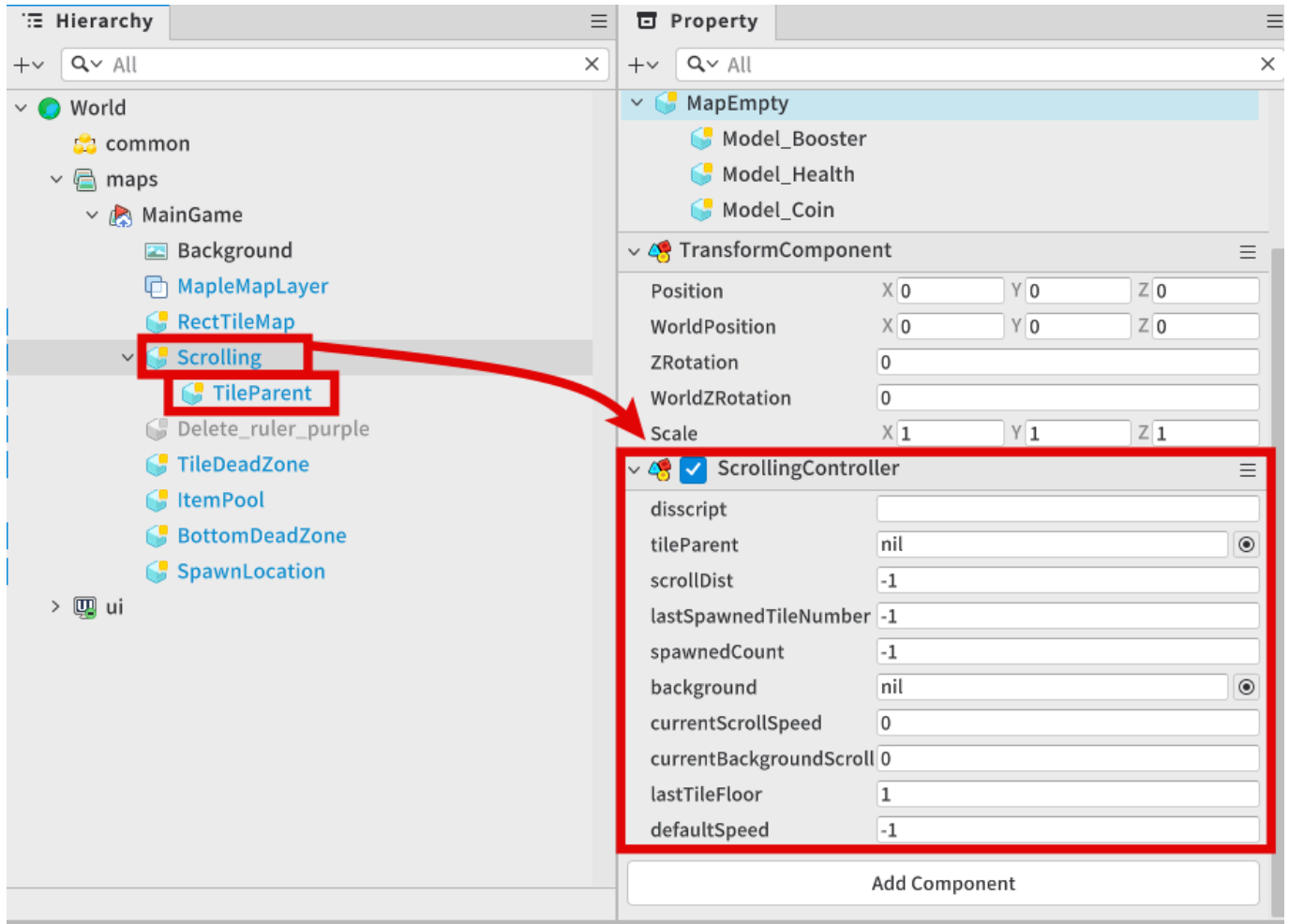
- 러닝 플랫폼어 게임에서 캐릭터가 끝없이 오른쪽으로 달려가게 만들면, 맵을 무한정 길게 제작해야 하는 한계에 부딪히게 됩니다.
- 캐릭터의 좌표값이 무한히 커질 경우 시스템상 치명적인 오류가 발생할 수 있습니다.
- 물리적 한계를 극복하고 무한한 맵을 안정적으로 연출하기 위해 스크롤 시스템의 도입이 필요합니다.

상대적 스크롤링



- 카메라와 플레이어 캐릭터는 이동시키지 않고 화면의 특정 위치에 고정해 둡니다.
- 대신 캐릭터가 밟고 있는 바닥 타일, 장애물, 배경 요소를 캐릭터의 진행 반대 방향인 왼쪽으로 지속해서 이동시킵니다.
- 이를 통해 플레이어의 시각에는 캐릭터가 앞으로 달려나가는 것처럼 보이는 착시 효과가 발생합니다.
- 결과적으로 맵의 크기나 끝을 걱정할 필요 없이, 제자리에서 끊임없이 달리는 환경을 구현할 수 있습니다.

ScrollingController 제작하기



- 빈 엔티티에 ScrollingController 컴포넌트를 등록합니다.
 - ScrollingController는 게임의 전반적인 스크롤 환경을 관리하는 주체입니다.
 - 생성된 엔티티의 TransformComponent의 Position 값을 (0,0,0)으로 설정합니다.
- 빈 엔티티를 생성하여 ScrollingController컴포넌트가 부착된 엔티티의 자식 엔티티로 설정합니다.
 - 해당 엔티티는 타일셋 전체를 이동시키기 위한 부모 엔티티 역할을 수행합니다.
 - 생성된 엔티티의 TransformComponent의 position 값을 (0,0,0)으로 설정해 부모와 동일선상에 위치시킵니다.

ScrollingController Component 구성하기

- 배경, 바닥 타일을 제어하기 위한 코드를 작성해야 합니다.
- 다음과 같이 코드를 작성합니다.
- 해당 코드는 Plain Text를 기준으로 작성되어 있습니다.

```

@Component
script ScrollingController extends Component

property string disscript = "" -- 러닝 플랫폼머 게임 내에 횡이동을 전반적으로 담당하기
위한 로직입니다.

property Entity tileParent = nil

property number scrollDist = -1 -- 현재 스크롤된 거리를 저장하는 변수입니다. //
config에 설정된 타일셋 하나당 거리는 10

property number lastSpawnedTileNumber = -1 -- 가장 마지막으로 생성된 타일번호입니다.

property number spawnedCount = -1 -- 몇개의 타일이 스폰되었는지에 대한 기록용 변수입니
다.

property BackgroundComponent background = nil

property number currentScrollSpeed = 0 -- 현재 스크롤링 속도값을 저장하기 위한 변수입
니다.

property number currentBackgroundScrollSpeed = 0 -- 현재 배경 스크롤링 속도값을 저
장하기 위한 변수입니다.

property number lastTileFloor = 1 -- 마지막으로 스폰된 마지막 타일층수를 기록하기 위한
변수입니다.

property number defaultSpeed = -1 -- 캐릭터 선택시 설정되는 스크롤 속도값 입니다.

@ExecSpace("ClientOnly")
method void Scroll(number moveDistance)
-- 현재 설정중인 스크롤을 전달받은 이동거리만큼 활성화된 TileSet 오브젝트 이동합니다.

-- 부모 오브젝트(TileParent)를 이동시킵니다.
self.tileParent.TransformComponent:Translate(moveDistance,0)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 내부에 필요한 멤버들의 참조를 연결하는 메서드입니다.

-- 모든 타일셋의 부모가 될 오브젝트의 Entity를 연결합니다.
self.tileParent =
_EntityService:GetEntityByPath("/maps/MainGame/Scrolling/TileParent")

-- Background 컴포넌트를 연결합니다.
self.background =
_EntityService:GetEntityByPath("/maps/MainGame/Background").BackgroundCompo
nent
end

@ExecSpace("ClientOnly")

```

```

method void Init()
-- Scrolling Controller 컴포넌트의 내부 초기화를 진행하는 메서드입니다.

-- 내부 참조연결을 진행합니다.
self:ConnectRefs()

-- 배경 화면 스크롤링 속도값을 Config에 설정한 배경 스크롤링 속도로 설정합니다.
self.background.ScrollRate = _Config.backgroundScrollSpeed

-- 실제 배경오브젝트가 스크롤링 되는 속도값을 지정합니다.
self.currentBackgroundScrollSpeed = _Config.backgroundScrollSpeed

-- 생성된 타일셋의 개수를 초기화합니다.
self.spawnedCount = 0
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 매 프레임마다 호출되는 메서드입니다.

-- 만약 메인게임이 종료되었다면
if _GameLogic.isEnd == true then
-- Update 메서드 동작을 중단합니다.
return
end

-- 설정된 스크롤링 속도로 바닥 오브젝트 이동을 수행합니다.
self:Scroll(-1 * delta * self.currentScrollSpeed)
end

@ExecSpace("ClientOnly")
method void SpawnRandomTile()
-- 무작위 타일셋을 생성하는 메서드입니다.

-- 마지막으로 생성한 타일 번호와 중복되지 않은 무작위 번호를 추출합니다.
local randomNumber = self:GetRandomTileNumber()

-- 랜덤으로 추출한 번호의 타일을 생성합니다.
self:SpawnTile(randomNumber)
end

@ExecSpace("ClientOnly")
method number GetRandomTileNumber()
-- 무작위 타일 번호를 추출하기 위한 메서드입니다.

-- 마지막으로 스폰된 타일셋의 끝나는 지점의 층수와 연결될 수 있는 타일셋 번호들이 담긴 테이블을 가져오는 과정입니다.
-- ex) 끝난 지점의 층수 : 2층 / 2층으로 시작하는 타일셋 번호가 담긴 테이블 0
-- ex) 끝난 지점의 층수 : 1층 / 2층으로 시작하는 타일셋 번호가 담긴 테이블 X
-- 연결된 타일셋을 생성하기 위함입니다.

-- 절차적 맵 생성의 핵심 로직입니다.

-- 마지막으로 스폰된 타일의 종료지점의 층수를 N 층으로 가정할때,

```

```

-- N 층으로 시작할 수 있는 타일셋 번호들이 담긴 테이블을 가져옵니다.
local tileSetFloorInfoTable =
_DataSetLogic.tilesFloorInfoTable[self.lastTileFloor]

-- 해당 테이블의 총 개수를 확인합니다.
local tileSetTableTotalCount = #tileSetFloorInfoTable

-- 1 부터 타일 종류 수까지를 범위로 무작위 번호를 추출합니다.
local randomNumber = math.random(1, tileSetTableTotalCount)

-- 대상 층수만 담긴 테이블에서 'randomNumber'번째 타일셋 번호를 추출합니다.
local selectedTileNumber = tileSetFloorInfoTable[randomNumber]

-- 연속된 타일 스폰을 방지하기 위한 로직입니다. (연속된 타일셋 생성을 허용하길 원한다면 아래 구문을
수행하지 않으면 됩니다.)

-- 임의로 뽑은 숫자의 번호가 이전에 스폰된 타일 번호와 같다면 (중복된 타일선택을 막기 위해 다시
랜덤 돌리기를 진행합니다.)
if selectedTileNumber == self.lastSpawnedTileNumber then

    -- 다시 임의의 무작위 숫자를 추출합니다. (해당 층수에 맞는 타일셋 요소들중 랜덤요소를 추출합
니다.)
    local tempNumber = math.random(1, tileSetTableTotalCount)

    -- 임시의 숫자가 또 다시 같은 번호라면
    if tempNumber == randomNumber then
        -- 임시로 뽑은 숫자의 값에 1 증가시킨 뒤 나머지 연산자를 활용합니다.
        tempNumber = (tempNumber + 1) % tileSetTableTotalCount + 1

        -- 데이터셋의 첫 시작값은 1부터 시작되는 이유로, 만약 나머지 연산처리를 한 결과가 0이
라면
        if tempNumber == 0 then
            -- tempNumber 값을 1로 설정합니다.
            tempNumber = 1
        end

    end

    -- 대상 층수만 담긴 테이블에서 임의로 뽑은 'tempNumber' 번째 요소를 선택합니다.
    selectedTileNumber = tileSetFloorInfoTable[tempNumber]
end

-- 해당 타일셋 번호를 반환합니다.
return selectedTileNumber
end

@ExecSpace("ClientOnly")
method void SetScrollSpeed(number speed)
-- 바닥 타일 오브젝트의 이동속도를 조절하는 메서드입니다.

-- 매개변수 1 number speed : 설정하고자 하는 속도값을 전달받습니다.

-- 현재 속도값을 매개변수로 전달받은 속도값으로 설정합니다.

```

```

self.currentScrollSpeed = speed
end

@ExecSpace("ClientOnly")
method void SetBackgroundScrollSpeed(number speed)
-- 배경화면 스크롤 속도를 조절하는 메서드입니다.

-- 매개변수 1 number speed : 설정하고자 하는 속도값을 전달받습니다.

-- 배경의 스크롤 속도를 매개변수 값으로 설정합니다.
self.background.ScrollRate = speed
end

@ExecSpace("ClientOnly")
method void ReturnAllTiles()
-- 생성된 모든 타일셋 오브젝트들을 전부 반환(비활성)하는 메서드

-- 'tileParent'의 자식들 중에서 재귀적으로 접근하여 'TileSet' 컴포넌트들을 테이블로 담습니
다.
local tileSetTable =
self.tileParent:GetChildComponentsByTypeName("TileSet", true)

-- 'TileSet'컴포넌트들이 담긴 테이블의 수만큼 순회를 진행합니다.
for i = 1, #tileSetTable, 1 do

    -- 만약 각 요소의 오브젝트가 켜져(활성화되어) 있는 상태라면 (켜져있는 오브젝트들만 반환을 진
    행하기 위함입니다.)
    if tileSetTable[i].Entity.Enable == true then

        -- 각 요소들의 타일셋 컴포넌트로 접근합니다.
        local tempTileset = tileSetTable[i].Entity.TileSet
        -- 타일셋의 자식으로 배치된 아이템들을 전부 회수합니다.
        tempTileset:ReturnAllItems()
        -- 해당 타일셋을 다시 오브젝트 풀로 반환합니다.
        _ObjectPoolLogic:ReturnTile(tempTileset.Entity)
    end
end

end

method void SpawnTile(number tileSetNumber)
-- 타일셋 오브젝트를 스폰하는 메서드입니다.

-- 매개변수 1 number tileSetNumber : 해당 매개변수 번호의 타일셋을 생성하기 위해 전달받습
니다.

-- 오브젝트 풀로부터 '매개변수'번의 타일 오브젝트를 전달받습니다.
local spawnedTile = _ObjectPoolLogic:GetTile(tileSetNumber)

-- 오브젝트 풀로부터 스폰받은 타일셋의 끝나는 지점의 층수를 기록합니다.
self.lastTileFloor =
_DataSetLogic.tileSetInfoTable[tileSetNumber].LastFloor

```

```

-- 타일셋의 기준이 되는 부모 오브젝트로부터의 x축 거리를 설정합니다.
-- spawnedCount(기존에 생성된 타일셋 수) * 10(타일셋 프리팹의 기준 거리단위)
spawnedTile.TransformComponent.Position = Vector3(self.spawnedCount *
10,0,0)

-- 생성된 타일셋 수를 1 증가시킵니다.
self.spawnedCount += 1
-- 마지막으로 생성된 타일셋 번호를 오브젝트 풀로부터 가져온 타일셋 번호로 교체합니다.
self.lastSpawnedTileNumber = tileSetNumber

-- 오브젝트 풀로부터 가져온 타일셋의 초기화를 진행합니다.
spawnedTile.TileSet:Init()
end

@ExecSpace("ClientOnly")
method void Reset()
-- 게임 진행에 필요한 멤버 요소를 초기화하는 메서드입니다.

-- 생성되었던 타일수를 0으로 초기화합니다.
self.spawnedCount = 0
-- 모든 타일셋 오브젝트들의 부모가 되는 'tileParent' 오브젝트의 위치를 원점으로 초기화합니다.
self.tileParent.TransformComponent.Position = Vector3(0,0,0)
-- 배경화면 스크롤링 속도를 Config에서 기준으로 설정했던 값으로 설정합니다.
self:SetBackgroundScrollSpeed(_Config.backgroundScrollSpeed)

-- 게임을 시작할때 1부터 5번까지 순서대로 타일셋을 생성합니다.

-- 1부터 5번까지 반복문을 수행하며
for i = 1, 5, 1 do
    -- i 번 타일을 스폰합니다.
    self:SpawnTile(i)
end
end

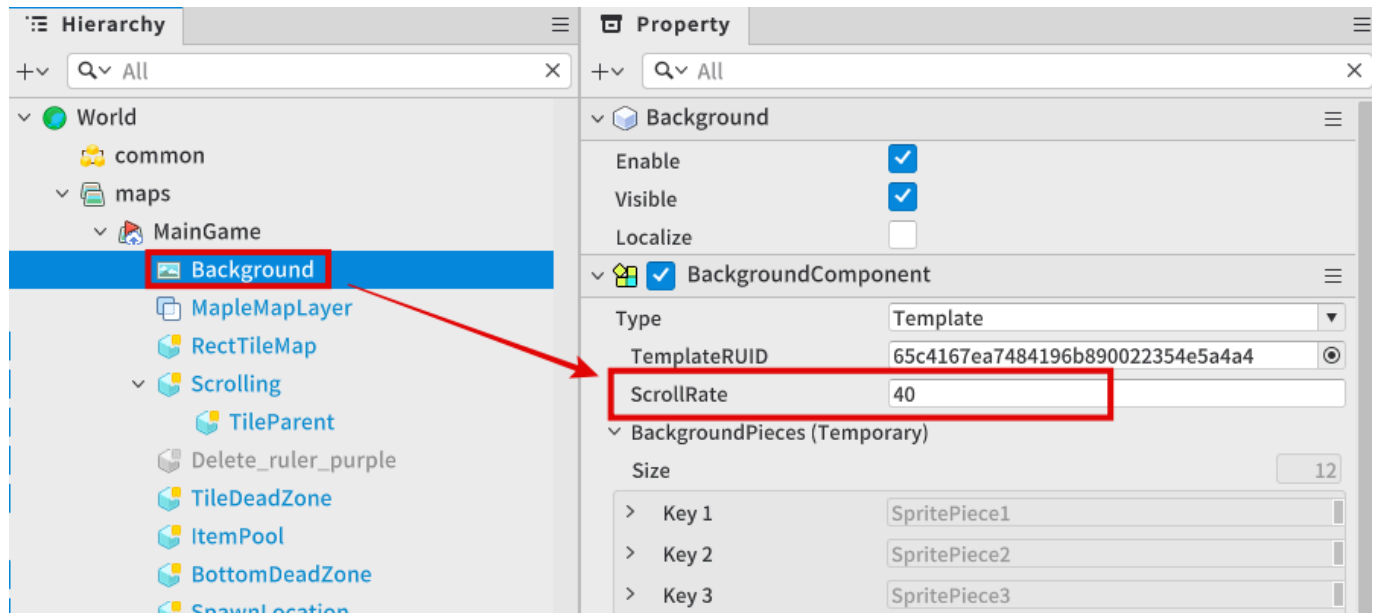
@ExecSpace("ClientOnly")
method void ChangeBackground()
-- 배경화면을 변경하는 메서드입니다.

-- 게임로직에 등록된 현재 스테이지 번호를 참고하여 데이터셋에 저장된 배경 RUID 값으로 배경화면을
설정합니다.
self.background:ChangeBackgroundByTemplateRUID(_DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].BackgroundRUID)
end

end

```

BackgroundComponent의 ScrollRate



- 배경 이미지의 스크롤 속도를 제어하기 위해선 Background 엔티티로 접근합니다.
- BackgroundComponent의 ScrollRate값을 설정합니다.
- 에디터가 플레이 상태가 아니더라도 Scene탭의 화면에서 바로 확인할 수 있습니다.

코드로 ScrollRate 변경하기

- 게임 진행 도중에 실시간으로 배경 스크롤속도를 제어하는 방법입니다.
- ScrollingController 스크립트 내 등록된 SetBackgroundScrollSpeed 메서드를 참고합니다.

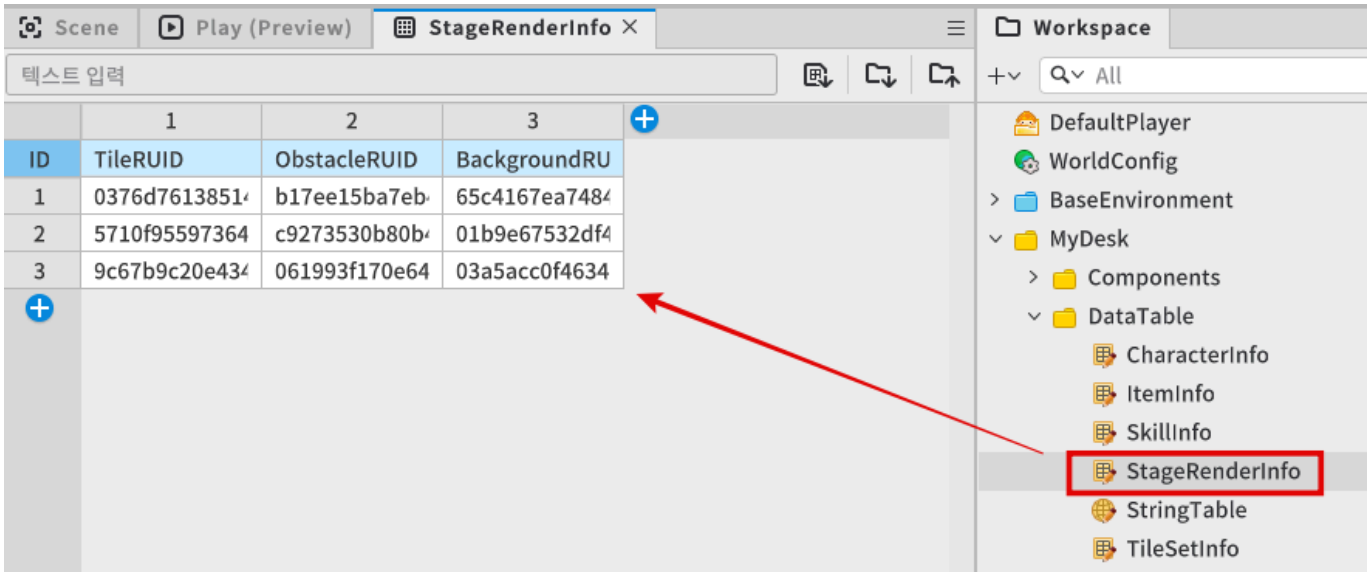
```
void SetBackgroundScrollSpeed (number speed)
{
    self.background.ScrollRate = speed
}
```

실시간 스테이지 변경 방법



- 게임이 진행되며 일정 시간이 경과하면 배경, 바닥 타일, 장애물의 외형이 변경되도록 설정합니다.

스테이지별 적용 될 외형 DataSet 등록하기



이름	설명
TileRUID	캐릭터와 상호작용하는 바닥 타일의 리소스 ID 값입니다.
ObstacleRUID	바닥 타일과 함께 나타나는 장애물 요소의 리소스 ID 값입니다.
BackgroundRUID	backgroundComponent의 templateRUID 값으로 배경으로 적용될 리소스 ID 값입니다.

- 스테이지 별로 적용되어야할 RUID값을 저장하기 위한 DataSet이 필요합니다.
- 샘플 월드에서는 그림과 같이 1,2,3 스테이지로 나뉘며 스테이지 별 바닥 타일의 RUID, 장애물의 RUID, 배경 이미지의 RUID 값을 저장하고 있습니다.

스테이지 변경 방식

- 스테이지 번호를 관리하는 데이터는 GameLogic의 currentStage의 number형 변수로 관리됩니다.
- GameLogic의 OnUpdate메서드에서 일정 시간이 지나면 스테이지가 변경되는 기능을 호출합니다.
- 샘플 월드의 경우 총 3개의 스테이지만 등록되어 있지만, %연산자를 활용하여 1,2,3 -> 1,2로 스테이지를 순환하는 구조로 제작되어 있습니다.
- 바닥 타일, 장애물의 경우 엔티티가 활성화될 때, GameLogic의 currentStage 번호를 확인하고 현재 번호에 맞는 RUID값을 적용합니다.
- 다음 코드는 GameLogic의 코드 중 일부입니다.

```
GameLogic
number currentStage = 1 -- 현재 스테이지 정보를 기록하는 변수입니다.
number stageTimer = 0 -- 스테이지 전환까지 몇초가 남았는지 기록합니다.

void OnUpdate(number delta) -- 매 프레임마다 호출되는 메서드입니다.
{
    if(stageTimer > 0) then -- 타이머에 남은 시간이 남아있다면
        self.stageTimer -= delta -- 타이머에 delta값 감소

    else -- 타이머에 남은 시간이 없다면
        self.StageChange() -- StageChange 메서드를 호출합니다.
        self.stageTimer = _Config.stageChagneTime -- 타이머를 초기 시간값으로 초기
        화 합니다.
    }

void StageChange() -- 스테이지를 전환하는 메서드입니다.
{
    -- 스테이지 번호를 1,2,3,1 로 계속 순환할 수 있는 구조로 제작합니다.
    self.currentStage = (self.currentStage % (totalStageCount+1))+1
    scrollingController:ChangeBackground()
}
```

바닥 타일의 외형 적용 예시

- 샘플 월드에서 제공되는 바닥 타일의 기능을 수행하는 컴포넌트 내에서 엔티티가 활성화될 때, Init메서드내에서 외형을 적용하고 있습니다.
- TileSet 컴포넌트 스크립트는 다음 단원인 [절차적 생성 방식을 통한 맵 관리하기](#) 구체적인 학습을 진행합니다.

```
TileSet

void Init()
{
    local renderers =

    self.Entity:GetChildComponentsByTypeName("SpriteRendererComponent", false)

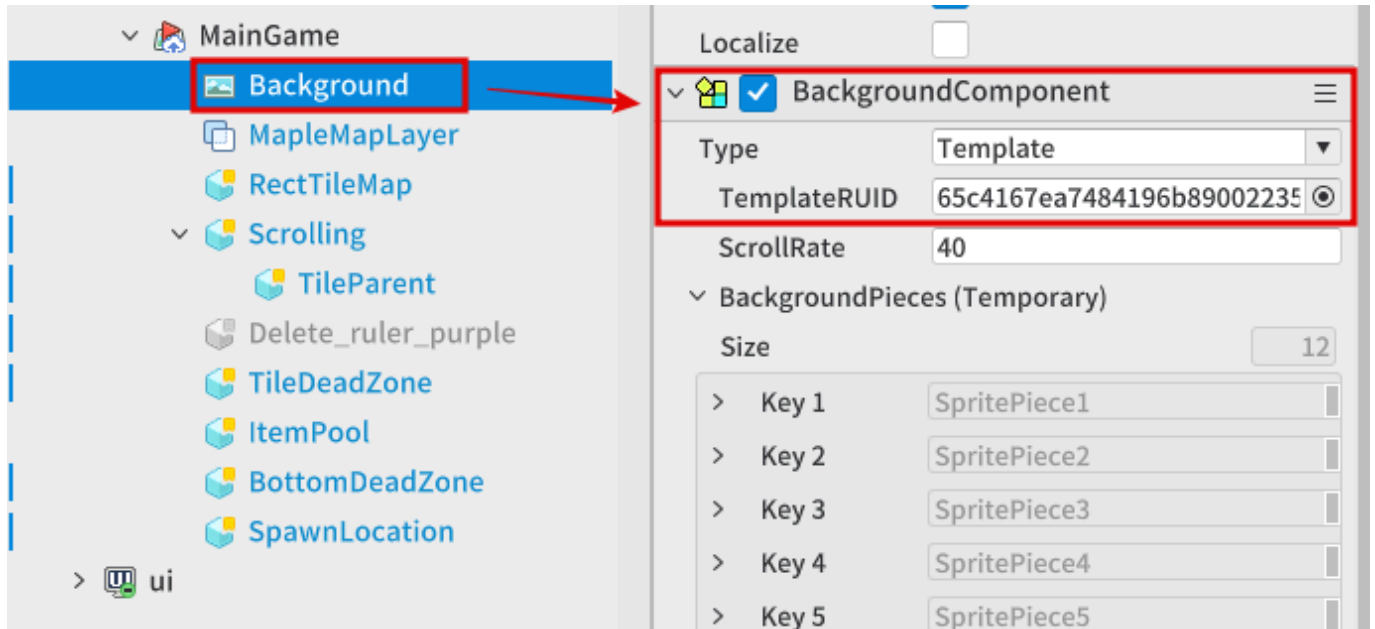
    renderer.SpriteRUID =
```

```

    _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].TileID
}

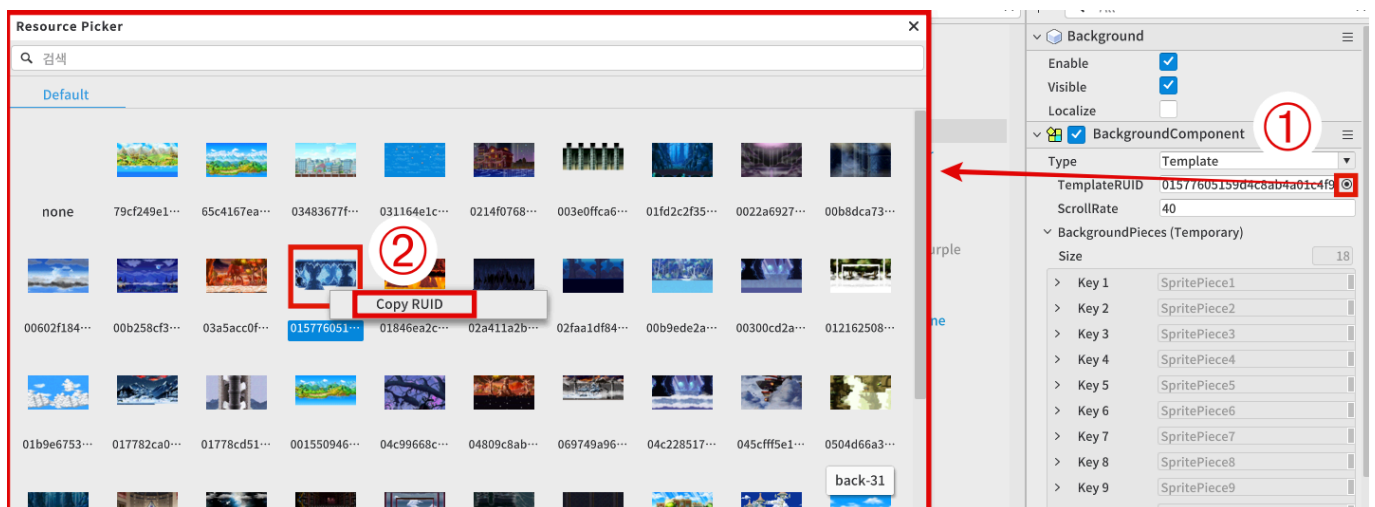
```

배경 외형 변경하기



- 배경 이미지를 관리하는 BackgroundComponent의 RUID를 변경하기 위해선 BackgroundComponent에서 제공하는 메서드를 사용해야 합니다.
- 현재 Background Type을 확인합니다.
- 활용되고 있는 타입은 Template이므로 BackgroundComponent의 ChangeBackgroundByTemplateRUID 메서드를 활용합니다.
 - 메이플스토리 월드 공식 문서, BackgroundComponent [링크](#)

backgroundTemplateRUID 값 확인하기



1. background의 TemplateRUID 값을 확인하기 위해, BackgroundComponent의 RUID의 리소스 피커창을 띄웁니다.
2. 원하는 리소스 형상을 우클릭 후 Copy RUID를 통해 값을 추출합니다.

backgroundTemplateRUID 변경하기

- GameLogic에서 스테이지 변경 기능을 호출할때 ScrollingController 컴포넌트 내에서 코드를 통해 배경 이미지를 변경합니다.
- 등록된 배경의 타입을 확인하고, 적합한 BackgroundComponent에서 제공하는 RUID 변경 메서드를 활용합니다.

```
ScrollingController

BackgroundComponent background

void ChangeBackground()
{
    self.background:ChangeBackgroundByTemplateRUID(
        _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].BackgroundRUID)
}
```

최소 구현 기준

- 기초적인 바닥 타일을 생성과 바닥 타일의 이동 기능을 구현합니다.

응용 및 확장 방향

- 바닥 타일, 장애물, 배경의 RUID를 관리하는 DataSet내에 RUID를 추가하여 더욱 풍부한 스테이지를 구성할 수 있습니다.
- 스테이지 전환 타이머를 조절하여, 원하는 타이밍에 스테이지를 전환할 수 있습니다.

[💡 기초 학습]절차적 생성 방식을 통한 맵 관리하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 절차적 생성 방식을 통한 맵 관리하기: 00:33:49

학습 목표

- 무한히 맵을 늘려나가는 절차적 맵 생성의 개념과 실무적인 필요성을 이해합니다.
- 맵을 일정한 조각(청크) 단위로 나누고, 각 청크의 시작 총수와 끝 총수 데이터를 매칭하여 자연스럽게 맵을 이어 붙이는 논리를 학습합니다.

- 스크립트를 통해 무한히 맵을 생성하며, 바닥 타일 스폰 지점에 새로운 바닥 타일 엔티티를 생성하는 방법을 구현합니다.

해당 영상 시청 후 수행해 볼 내용

- DataSetLogic내에 관리되는 층수 관리 테이블에 데이터를 추가하여 새로운 타일 정보를 추가합니다.
- 조건에 맞는 다음 타일셋이 무작위로 선택되어 마지막 끝 X좌표에 정확히 스폰되는지 플레이 버튼을 눌러 확인합니다.
- 생성된 맵의 발판 높낮이가 어색하지 않은지, 타일 사이에 빈틈이 발생하지 않는지 인게임에서 직접 테스트해 봅니다.

절차적 맵 생성의 개념 이해

- 절차적 맵 생성이란, 지형을 처음부터 끝까지 수작업으로 배치하는 대신 정해진 규칙과 알고리즘에 따라 게임 플레이 도중 실시간으로 맵을 만들어 내는 시스템을 의미합니다.
- 끝없이 달려야 하는 무한 러닝 장르에서 맵의 길이를 제한 없이 연장하고, 매번 다채로운 지형 패턴을 생성하기 위해 필요한 핵심 기반 기술입니다.

무한한 맵의 한계 극복

- 러닝 플랫폼 게임에서 플레이어가 달릴 맵을 에디터 상에 처음부터 끝까지 길게 만들어 두는 것은 작업량뿐만 아니라 시스템 메모리 측면에서도 큰 한계를 가집니다.
- 이를 해결하기 위해 게임 플레이가 진행되는 상황에 맞추어 실시간으로 새로운 맵 조각을 생성해 내는 '절차적 생성' 기법을 도입합니다.
- 이 기법을 적용하면 달리는 기차의 바로 앞에 선로를 계속 깔아주는 것처럼, 맵을 무한히 연장하면서도 시스템 자원을 효율적으로 관리할 수 있습니다.

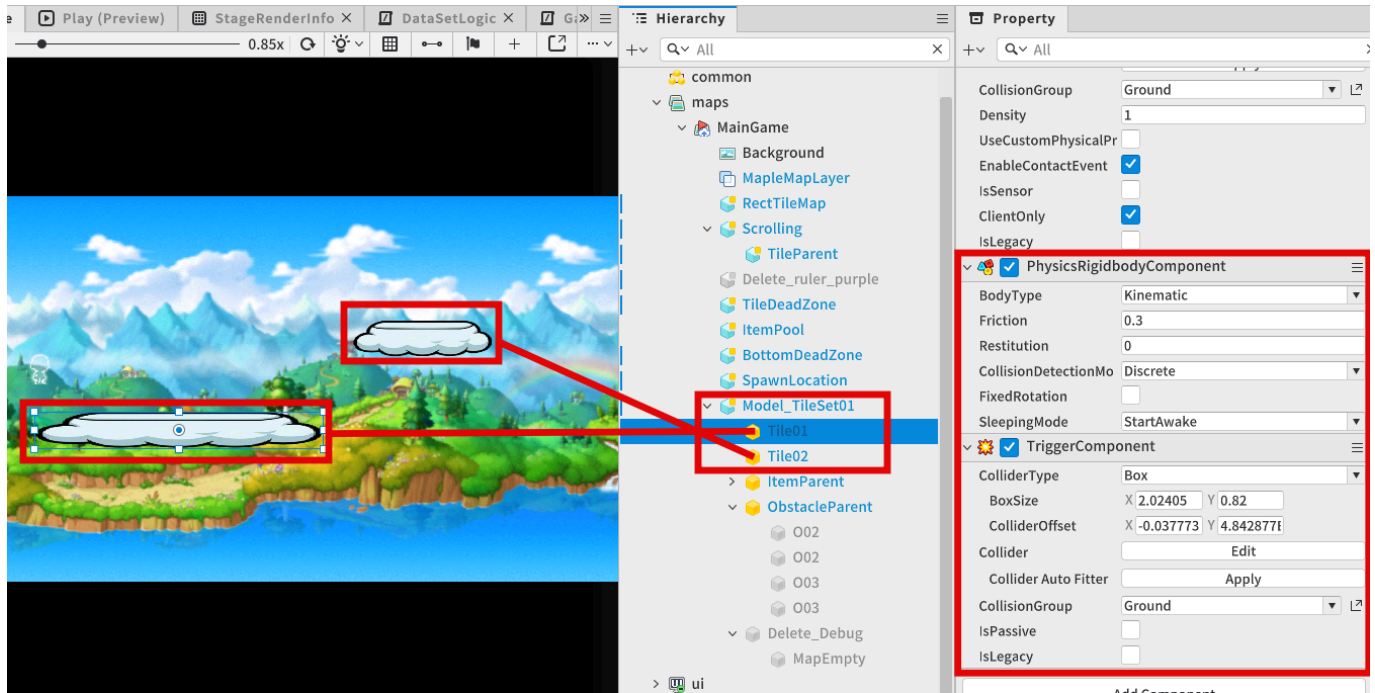
타일셋(청크) 단위의 분할 관리와 무작위성

- 맵을 실시간으로 생성할 때 픽셀이나 타일 하나하나를 무작위로 찍어내는 것이 아니라, 일정한 길이의 맵 조각(청크)들을 여러 패턴으로 미리 제작해 두고 관리합니다.
- 스크립트가 이 미리 만들어진 조각들 중 하나를 규칙에 맞게 무작위로 선택하여 이전 조각의 끝부분에 레고 블록처럼 조립하는 방식으로 동작합니다.
- 이를 통해 맵의 구조적인 안정성을 유지하면서도, 유저가 게임을 플레이할 때마다 매번 다른 형태의 지형을 마주하게 만들어 게임의 리플레이성을 크게 높일 수 있습니다.

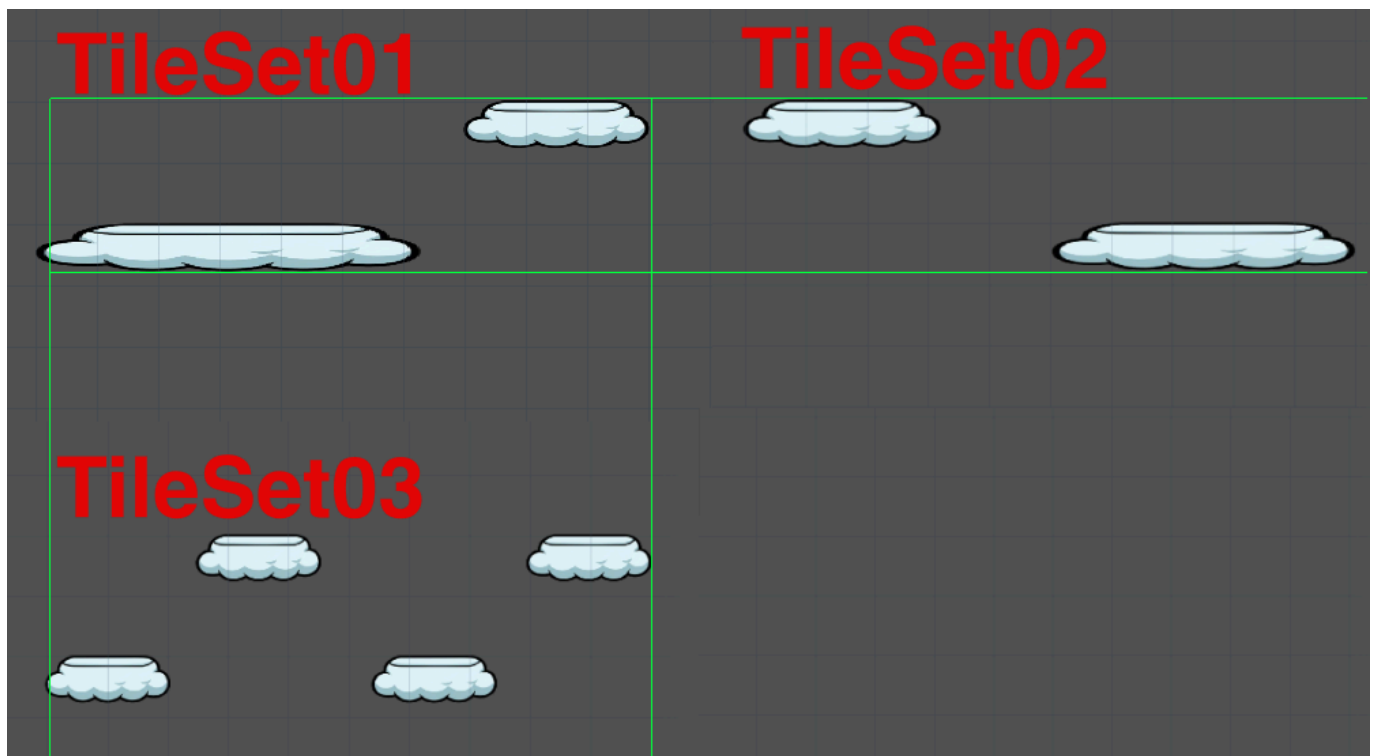
청크 맵 제작하기

- 청크란 절차적 맵 생성 시스템에서 지형을 이어 붙일 때 사용하는 미리 만들어진 일정한 크기의 맵 조각을 뜻합니다.

바닥 타일 엔티티 생성

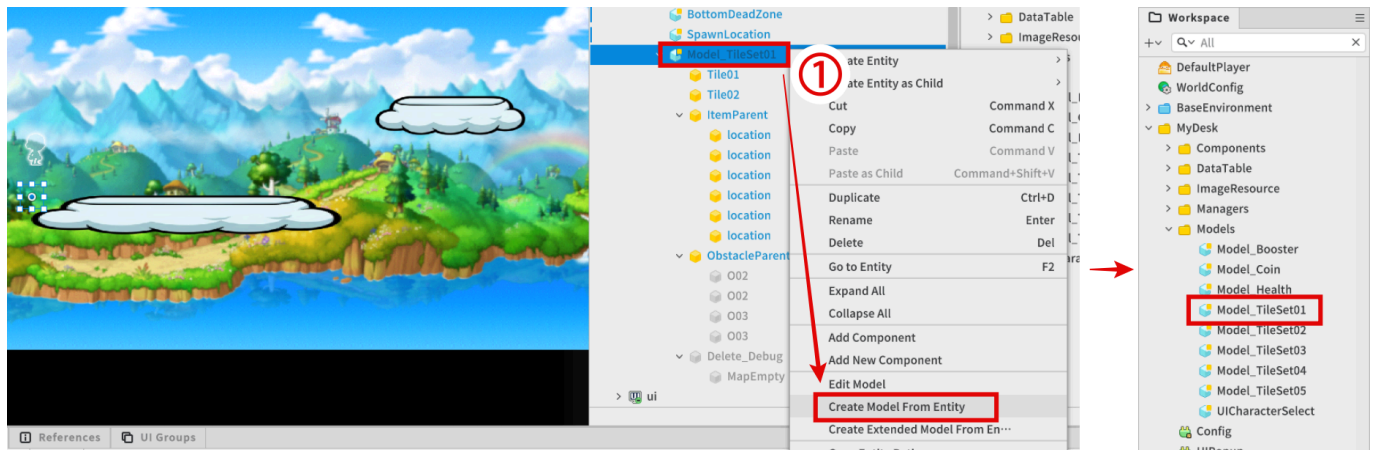


- 한 단위의 청크를 생성하기 위해 타일의 부모가 될 빈 엔티티를 생성합니다.
- 이때 에디터 상에서 타일을 배치할 때 원활한 작업을 위해 부모 엔티티의 좌표를 원점(0,0,0)으로 설정합니다.
- 여러 개의 자식 엔티티를 등록하고 SpriteRendererComponent를 부착합니다.
- SpriteRendererComponent의 RUID값을 설정해 외형을 등록합니다.
- 부모 엔티티를 제외한, 각 바닥 엔티티에 PhysicsRigidbody, PhysicsCollider 컴포넌트를 등록하고 캐릭터와 상호 작용할 수 있도록 설정합니다.
- PhysicsCollider 컴포넌트 내에 collider edit버튼을 통해 콜라이더의 위치, 사이즈를 수정합니다.



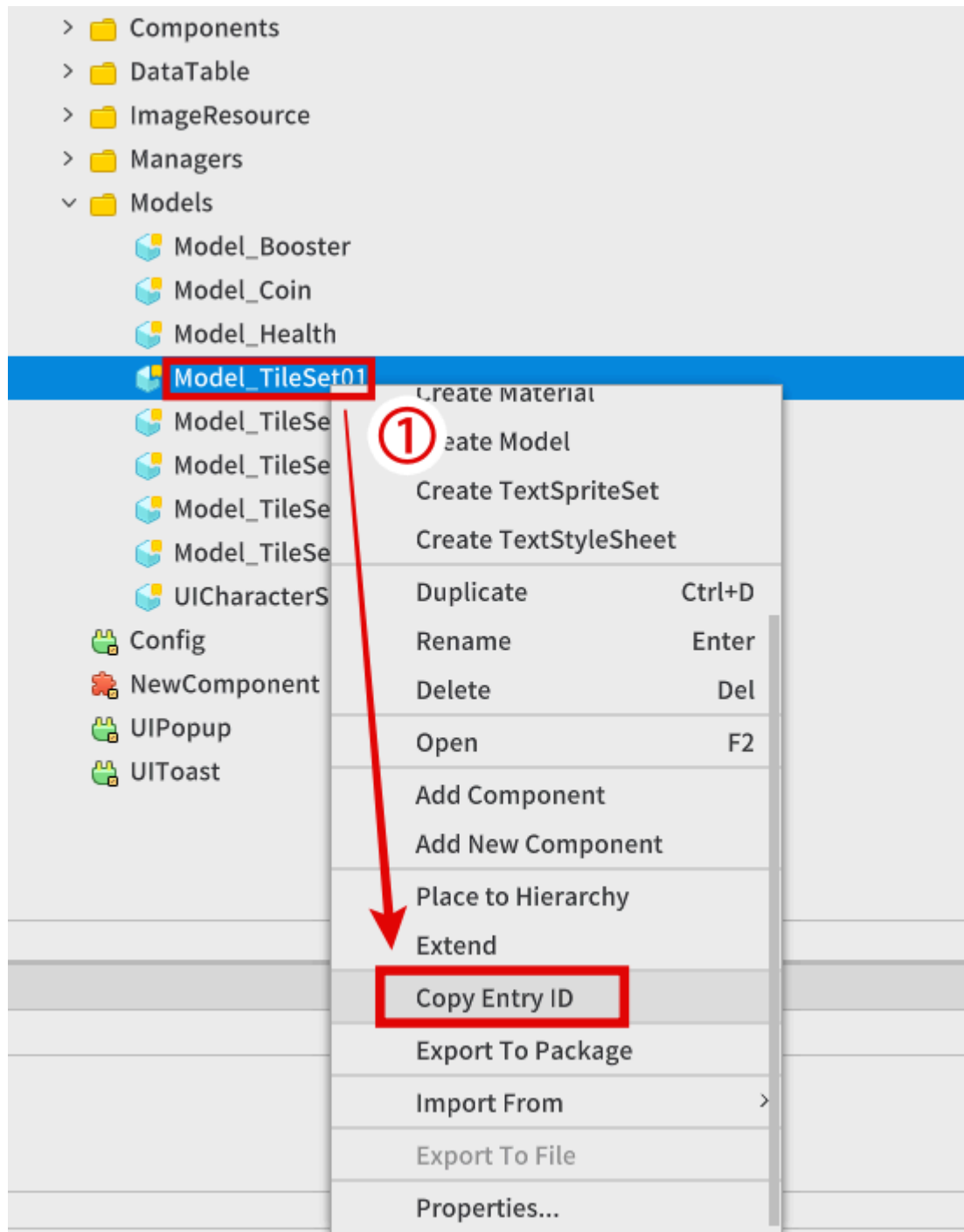
⚠ 모든 청크는 주기적으로 동일한 위치에 생성되는 방식이므로 청크맵마다 길이, 높이가 다를 경우 맵이 끊겨있는 모습처럼 연출될 수 있습니다. 샘플 월드 제작의 경우 하나의 타일셋의 x축 간격은 10을 기준으로 제작했습니다.

제작한 바닥 타일셋 모델 등록하기



- 생성한 청크맵을 미리 제작하고 생성 호출로 필요한 경우 바로 엔티티로 배치할 수 있도록 엔티티의 모델 등록이 필요합니다.
- 제작이 완료된 타일셋 엔티티를 우클릭 후 Create Model From Entity를 선택합니다.
- Workspace에서 해당 엔티티의 모델이 생성된 것을 확인하실 수 있습니다.

제작한 모델 생성하기



- 하나의 원본 바닥 타일을 제작하였으므로 해당 타일을 필요한 만큼 하이어라키에서 손쉽게 생성할 수 있습니다.
- Workspace에서 생성된 Model을 우클릭 후 CopyEntry ID를 선택해 Entry ID값을 복사합니다.
- 코드에서 EntryID와 SpawnService를 활용해 모델과 동일한 엔티티를 월드에 생성합니다.
 - 메이플스토리 월드 공식 문서, SpawnService [링크](#)

스폰 활용예시

```
void Spawn()
{
    local entryID = "model://02e63fc1-cca0-4b98-ada3-d5a36e26f7de"
    _SpawnService:SpawnByModelId(entryID,"TempEntity",Vector3.zero,nil)
}
```

타일 별 시작 지점과 끝 지점 층수 정보 등록하기

	1	2	3
ID	FirstFloor	LastFloor	EntryID
1	1	2	model://02e631
2	2	1	model://d1ca0
3	1	2	model://ec391
4	2	1	model://d00b8
5	1	1	model://30079

- 각 타일의 시작과 끝 정보를 직관적으로 관리하기 위한 데이터 테이블을 제작합니다.
- 테이블에는 각 타일의 첫 시작부분의 층수, 종료부분의 층수를 기록하는 FirstFloor, LastFloor 정보와 각 Model의 EntryID 값을 기록합니다.

DataSet에 각 바닥 타일 데이터 등록하기

①

ID	FirstFloor
1	1
2	2
3	1
4	2
5	1

②

TableName	FirstFloor	TilesNumber
tilesFloorInfoTable	[1]	1,3,5
	[2]	2,4

Red arrows indicate the mapping from the first floor values in table ① to the FirstFloor values in table ②.

1. 실제로 저장된 DataSet(TileSetInfo)구조의 일부입니다.
 2. DataSetLogic에서 어떠한 방식으로 테이블에 저장할지 나타낸 표입니다.
- 샘플 월드의 바닥 타일은 1층과 2층 두 가지로 제작되었습니다. 따라서 tileFloorInfoTable 역시 [1]과 [2] 두 개의 배열로 나누어 별도로 관리 및 접근합니다.

DataSetLogic의 tileFloorInfoTable 저장 코드

```
table tilesFloorInfoTable = {[1]={}, [2]={}}
```

```
void InitTileSetInfoTable()
```

```
{
```

```
    -- 타일 오브젝트와 관련된 데이터셋을 저장하기 위한 메서드입니다.
```

```

-- 월드스페이스에 저장된 데이터셋의 이름을 로컬로 접근하기 위한 변수입니다.
local dataSetName = "TileSetInfo"

-- 대상 데이터셋의 Row수입니다.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 타일셋 종류 수를 저장합니다.
self.totalTileSetCount = rowCount

-- 각 줄의 데이터를 담기 위한 변수입니다.
local rowData

-- 1 부터 데이터셋의 Row수만큼 순회를 시작합니다.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 타일셋 오브젝트의 첫번째 타일의 층수입니다.
        FirstFloor =
tonumber(_DataService:GetCell(dataSetName,i,"FirstFloor")),
        -- 타일셋 오브젝트의 마지막 타일의 층수입니다.
        LastFloor =
tonumber(_DataService:GetCell(dataSetName,i,"LastFloor")),
        -- 타일셋 오브젝트를 생성하기 위한 원본 엔티티 모델(프리팹)Id입니다.
        EntryID = _DataService:GetCell(dataSetName,i,"EntryID")
    }

    -- rowData로 저장한 데이터들을 테이블에 저장합니다.
    self.tileSetInfoTable[i] = rowData

    -- 지금 등록한 타일셋의 층수를 변수로 저장합니다.
    local firstTileFloor = self.tileSetInfoTable[i].FirstFloor

    -- /분기1/ 만약 첫번째 타일 층수가 1층이라면
    if firstTileFloor == 1 then
        -- 1층 테이블 목록에 저장합니다.
        table.insert(self.tilesFloorInfoTable[1],i)
    -- /분기2/ 첫번째 타일 층수가 1층이 아니라면
    else
        -- 2층 테이블 목록에 저장합니다.
        table.insert(self.tilesFloorInfoTable[2],i)
    end
end
}

number GetDataSetRowCount(string Name)
{
    -- 대상 데이터셋의 Row수를 반환합니다.

    -- 매개변수1 string Name : 대상 데이터셋의 이름을 전달받습니다.

    -- 확인한 데이터셋의 Row수를 반환합니다.

```

```

        return _DataService:GetRowCount(Name)
    }

```

DataSet에 등록된 각 바닥 타일 데이터 활용하기

- 저장된 tilesFloorInfoTable의 각 층별 테이블은 다음과 같이 분류할 수 있습니다.
 - tilesFloorInfoTable[1] : 1층부터 시작하는 바닥 타일셋 번호들이 담긴 테이블
 - tilesFloorInfoTable[2] : 2층부터 시작하는 바닥 타일셋 번호들이 담긴 테이블
- 마지막으로 생성된 바닥 타일의 끝 지점의 층수를 확인하고 각 층별 테이블 내에서 무작위 타일번호를 추출합니다.
- 마지막 타일의 층수를 등록하는 방법은 마지막으로 스폰하는 타일셋의 lastFloor값을 갱신합니다.

ScrollingController의 타일을 생성하는 메서드의 일부

```

number lastTileFloor

void SpawnTile(number tileSetNumber)
{
    -- 오브젝트 풀로부터 '매개변수'번의 타일 오브젝트를 전달받습니다.
    local spawnedTile = _ObjectPoolLogic:GetTile(tileSetNumber)

    -- 오브젝트 풀로부터 스폰받은 타일셋의 끝나는 지점의 층수를 기록합니다.
    self.lastTileFloor =
    _DataSetLogic.tileSetInfoTable[tileSetNumber].LastFloor
}

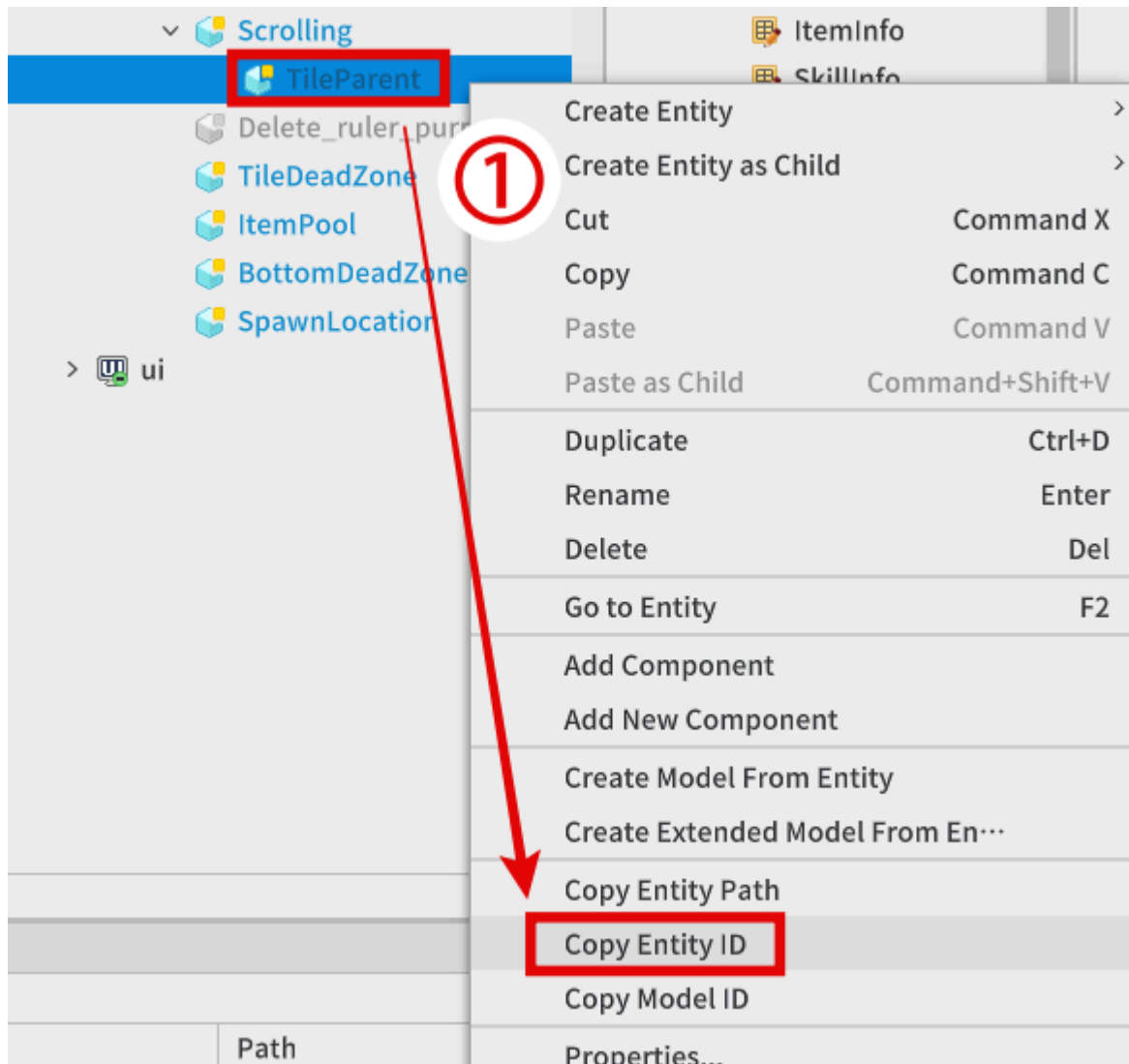
```

여러 청크맵 이동원리



- 월드 내에서 실제로 이동만 담당하는 Scrolling의 자식 엔티티 TileParent입니다.
- 각 청크맵 타일셋 엔티티별로 각자 이동을 수행하는 것이 아닌, 실제로 이동을 수행하는 TileParent의 자식 엔티티로 등록하여 자동으로 이동을 수행하도록 제작되어 있습니다.
- 즉, TileParent는 본격적인 게임 시작 시 -x축(오른쪽에서 왼쪽으로) 지속적으로 이동하게되며 TileParent엔티티의 자식으로 등록할 경우 자동으로 자식 엔티티 또한 동일하게 이동하게 됩니다.

TileParent엔티티의 자식으로 등록하기



- EntityService를 통해서 하이ера키상에 있는 엔티티를 가져올 수 있습니다.
- 대상 엔티티의 Entity ID를 가져오기 위해선 하이ера키상에 배치된 엔티티를 우클릭 후 Copy Entity ID를 통해 ID값을 복사할 수 있습니다.
- SpawnService의 SpawnByModelId 메서드는 4번째 매개변수(Parameter)로 생성될 객체의 부모(Parent) 엔티티를 지정할 수 있습니다.
- 생성했던 바닥 타일셋 모델로부터 새로운 엔티티를 생성할 때 Parent를 설정합니다.

SpawnService의 SpawnByModelId 메서드 활용예시

```
void SpawnTile()
{
    // DataSetLogic으로부터 2번 청크맵 타일의 EntryID가져오기
    local entryID = _DataSetLogic.tileSetInfoTable[2].EntryID

    // 생성될 새로운 엔티티의 이름
    local entityName = "TempEntity"

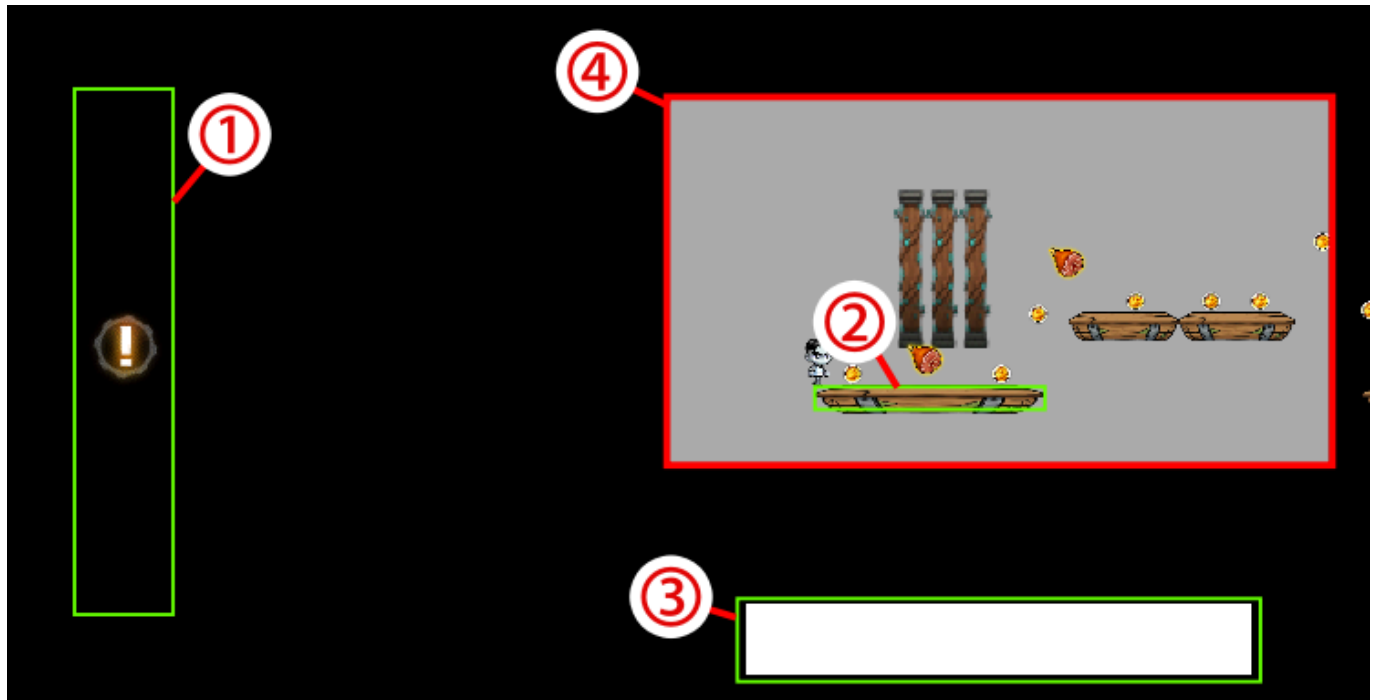
    // 생성될 좌표값
    local spawnPosition = Vector3(50,0,0)

    // 새로 생성되는 엔티티의 부모 엔티티
    local tileParentEntity = _EntityService:GetEntity("d24b5917-1821-4743-
```

```
ae07-77cc9bfc2d0a")
```

```
_SpawnService:SpawnByModelId(entryID,entityName,spawnPosition,tileParentEntity)
}
```

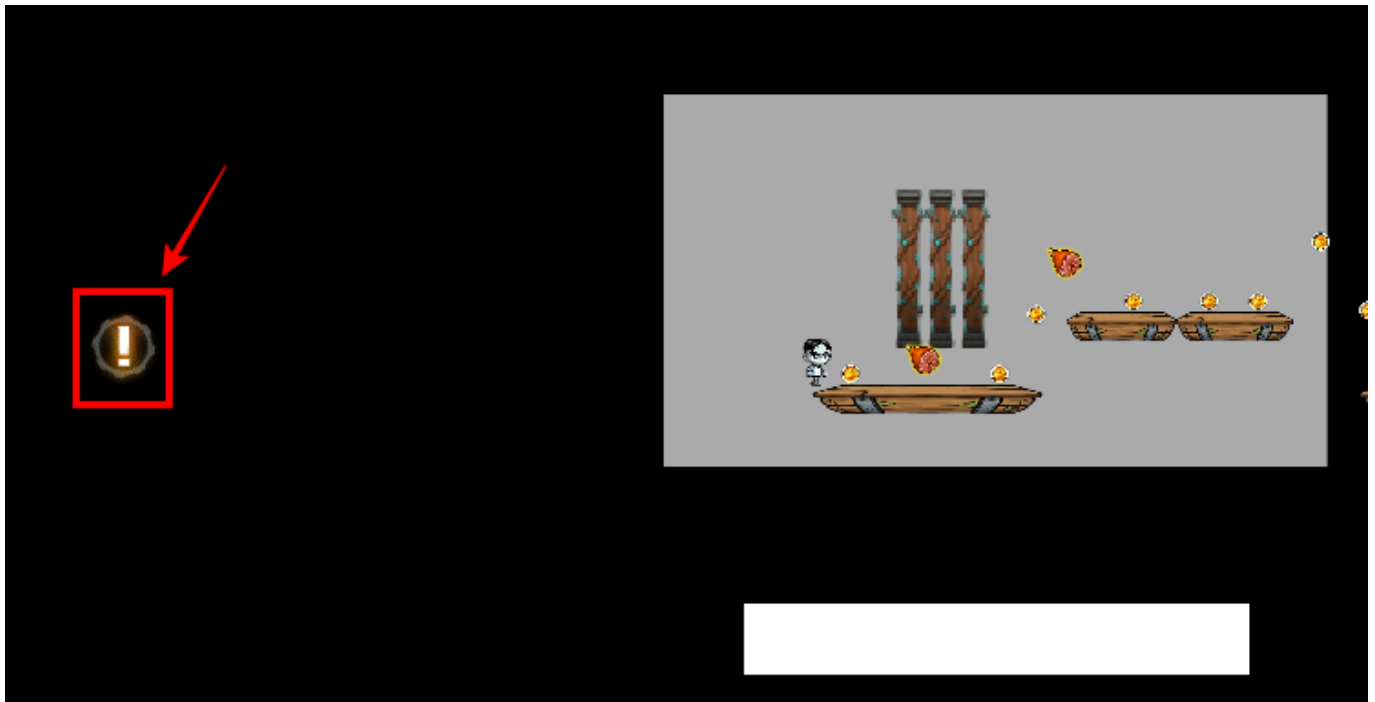
화면 영역에서 벗어난 타일셋 제거하기



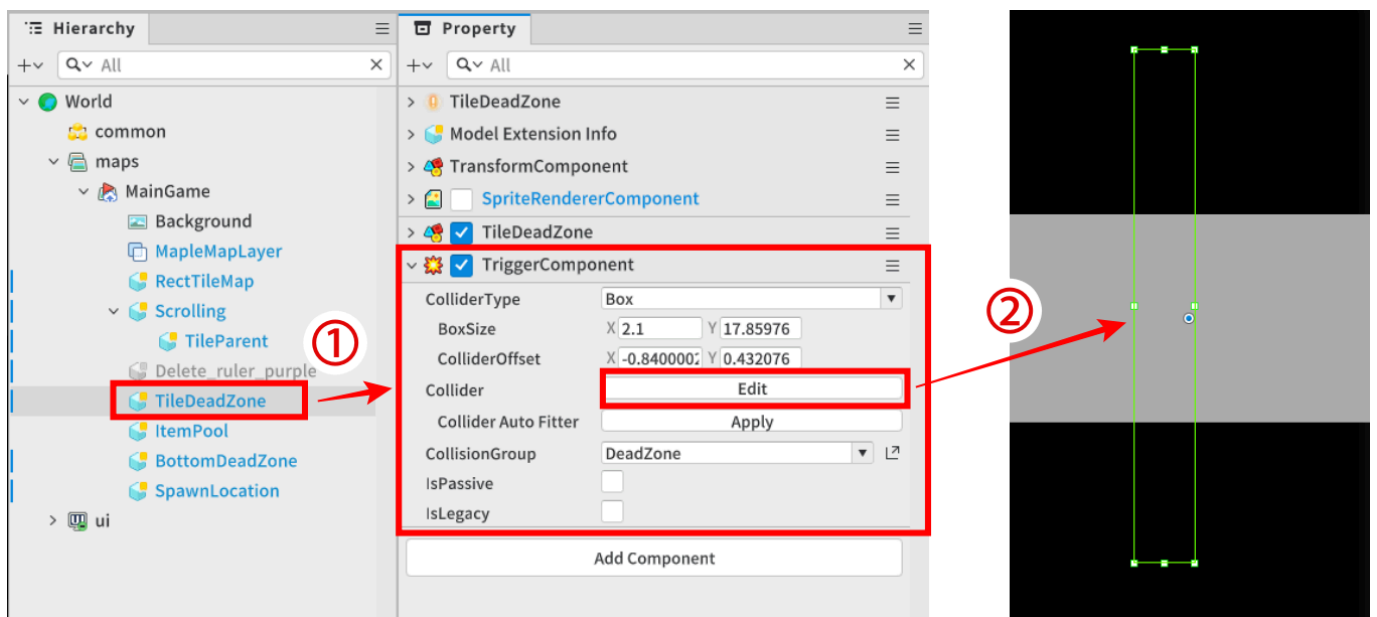
항목	대상	설명
1	DeadZone	왼쪽으로 이동하는 바닥 타일의 충돌을 감지하여 대상 엔티티를 제거하기 위해 배치합니다.
2	Tile Trigger	DeadZone과 상호작용하기 위한 Collider 입니다.
3	DeadZone	캐릭터가 아랫방향으로 떨어졌을 때 게임을 바로 종료시키기 위한 엔티티입니다.
4	View Area	실제 Play 환경에서 보이는 카메라 영역입니다.

- 화면 좌측 경계선을 완전히 벗어난 타일 엔티티는 시스템상 더 이상 렌더링할 필요가 없습니다.
- 바닥 타일과의 충돌을 감지하고 대상 타일셋을 제거할 수 있는 엔티티 배치가 필요합니다.

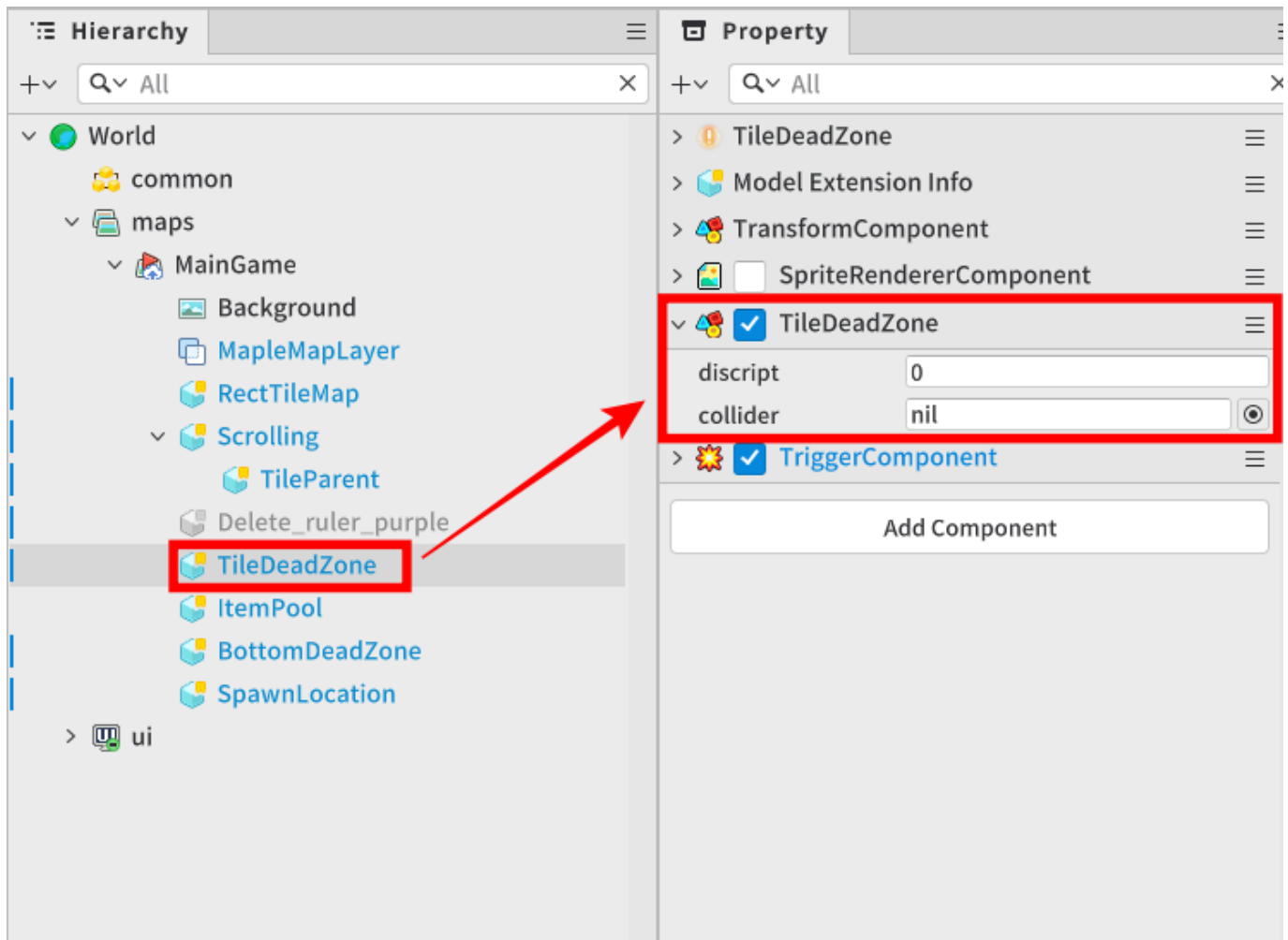
Tile과 상호작용하는 DeadZone 제작하기



- 바닥 타일이 제거되는 상황을 플레이어에게 보이지 않도록 하기 위해 카메라 영역으로부터 먼 거리에 새로운 엔티티를 생성합니다.



1. TileDeadZone 역할을 수행할 엔티티에 TriggerComponent를 등록합니다.
2. 부착된 TriggerComponent의 Collider의 Edit버튼을 통해 충돌 감지범위를 설정합니다.



1. TileDeadZone 컴포넌트 스크립트를 생성하고 TileDeadZone에 등록합니다.
2. TileDeadZone의 Plain Text로 코드 내용을 등록합니다.

```
@Component
script TileDeadZone extends Component

property number discript = 0 -- 타일 오브젝트의 파괴(비활성)를 담당하는 컴포넌트입니다.

property PhysicsColliderComponent collider = nil

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 내부에 필요한 멤버들의 참조를 연결합니다.

-- 충돌체 감지를 수행하기 위한 PhysicsCollider 컴포넌트를 연결합니다.
self.collider = self.Entity.PhysicsColliderComponent
end

@ExecSpace("ClientOnly")
method void Init()
-- TileDeadZone 컴포넌트 초기화를 진행하는 메서드입니다.

-- 변수로 선언된 참조 연결을 수행합니다.
self:ConnectRefs()
end
```

```

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 월드 실행시 호출되는 메서드입니다.

-- 해당 컴포넌트 초기화를 진행합니다.
self:Init()
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 트리거 충돌을 감지했을 때 발생하는 이벤트입니다.

-- 트리거 충돌의 엔티티를 저장합니다.
local TriggerBodyEntity = event.TriggerBodyEntity
-- 충돌된 트리거의 Collision그룹의 id 값을 확인합니다.
local TriggerGroup =
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)

-- Config에 등록한 테이블에 key 값으로 위의 그룹 id값을 대입했을때 반환받는 문자열이
'Ground' 라면
-- 즉, 트리거로 감지한 충돌체가 Ground 라면
if _Config.triggerGroup[TriggerGroup] == "Ground" then

    -- 타일셋 오브젝트 반환을 수행합니다.
    _ObjectPoolLogic:ReturnTile(TriggerBodyEntity.Parent)

    -- 스크롤링 컨트롤러에게 마지막 타일에 이어질 무작위 타일 생성을 호출합니다.
    _GameLogic.scrollingController:SpawnRandomTile()
end
end

end

```

최소 구현 기준

- 시작 층수와 끝 층수 매칭 데이터가 정확하게 동작하며, 타일 간 끊어짐이 없도록 구현합니다.
- 스크롤 이동 속도에 맞춰 새로운 맵 조각이 제때 생성되어, 캐릭터가 발판이 없어 맵 밖으로 추락하는 현상이 없도록 구현합니다.

응용 및 확장 방향

- 다양한 바닥 타일 모델(Model)을 등록하며 종류를 확장하거나 변경하여 난이도가 높은 맵 조각이 자주 등장하도록 무작위 생성 가중치를 조절해 볼 수 있습니다.

[💡 기초 학습]오브젝트 풀 생성하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 오브젝트 풀 생성하기: 00:44:52

학습 목표

- 엔티티(오브젝트) 생성과 파괴가 반복될 때 발생하는 성능 저하의 원인을 파악하고, 이를 해결하기 위한 오브젝트 풀링 기법의 필요성을 이해합니다.
- 게임 시작 시 필요한 맵 청크나 장애물 엔티티를 미리 생성하여 대기 상태로 보관하는 풀 자료구조를 직접 구현할 수 있습니다.
- 화면 밖으로 벗어난 엔티티를 삭제하지 않고 비활성화하여 풀에 반환하고, 필요할 때 위치를 재설정하여 다시 꺼내 쓰는 재사용 메서드를 작성합니다.

해당 영상 시청 후 수행해 볼 내용

- 로직 스크립트를 활용하여 게임 초기화 단계에서 지정된 수량만큼의 타일셋 엔티티를 미리 스폰한 뒤, 비활성화 상태로 보관하는 코드를 작성해 봅니다.
- 스크롤되어 화면 왼쪽 끝의 DeadZone에 닿은 타일이 삭제되지 않고, 속성이 초기화된 채 다시 오브젝트 풀로 돌아가 대기 상태가 되는지 테스트해 봅니다.
- 절차적 맵 생성 시 매번 새로운 엔티티를 만드는 대신, 오브젝트 풀에서 대기 중인 엔티티를 꺼내어 이전 타일의 끝 좌표에 정상적으로 이어 붙이는지 확인해 봅니다.

오브젝트 풀이란?

- 게임을 진행하는 데 있어 수많은 엔티티가 사용됩니다.
- 이러한 엔티티 중, 빈번하게 스폰되는 총알이나 발판 같은 요소는 일정 시간이 지나면 화면에서 사라져 소멸해야 합니다.
- 하지만 생성(Spawn)과 소멸(Destroy) 작업은 메모리 할당과 해제를 동반하므로, 이 과정이 짧은 시간 동안 반복될 경우 컴퓨터 연산에 큰 부하(프레임 드랍)를 유발합니다.
- 하지만 미리 엔티티를 생성하고, 소멸시키는 대신 활성화만 해제한다면 컴퓨터의 부하를 줄일 수 있습니다.
- 스위치를 켜고 끄듯 상태만 전환하여 메모리를 재사용하기 때문입니다.
- 즉, ****오브젝트 풀(Object Pool)****은 미리 만들어 둔 엔티티들을 '수영장(Pool)'에 담아두고 필요할 때마다 꺼내 쓰며 반환하는 방식으로 불필요한 생성과 소멸을 최소화하는 핵심 최적화 기법입니다.

오브젝트 풀 제작하기

- 샘플 월드의 오브젝트 풀은 GameLogic이나 DataSetLogic처럼 어디서든 전역적으로 접근하여 쉽게 엔티티를 대어하고 반환받을 수 있는 중앙 관리(Manager) 구조로 제작됩니다.
- Workspace에서 새로운 로직을 생성하고 ObjectPoolLogic으로 작성합니다.

ObjectPoolLogic 스크립트 작성하기

- 다음과 같이 코드를 작성해야 합니다.
- 해당 코드는 Plain Text 환경으로 작성되었습니다.

```

@Logic
script ObjectPoolLogic extends Logic

property number discript = 0 -- 생성과 파괴가 빈번하게 일어나는 오브젝트들을 효율적으로
관리하기 위한 로직입니다.

property table tilePool = {} -- 타일셋 오브젝트를 관리하기 위한 풀 역할을 수행하는 테이블
입니다.

property table itemPool = {} -- 아이템 오브젝트를 관리하기 위한 풀 역할을 수행하는 테이블
입니다.

property Entity ItemPoolParent = "9e9e98bd-4f7b-46ed-9a93-f0e7d74fac1c"

property table itemDataSetIDTable = [{"Coin"} = 1 , ["Health"} = 2 ,
["Booster"} = 3} -- key : 아이템명 , value : 데이터셋의 Index 번호를 저장하기 위한 테
이블입니다.

@ExecSpace("ClientOnly")
method void ReturnTile(Entity entity)
-- 타일 오브젝트를 오브젝트 풀로 반환하는 메서드입니다.

-- 매개변수 1 entity : 반환(비활성)하고자 하는 오브젝트를 전달받습니다.
-- 매개변수 2 tileNumber : 타일 번호를 전달받습니다.

-- 타일셋 엔티티의 번호를 추출합니다. (테이블 내 번호별로 타일셋 오브젝트를 관리하기 위함입니다.)
local tileNumber = self:GetTileNumber(entity)

-- 타일 번호에 해당하는 풀(테이블)이 유효한지 검사합니다.
if invalid(self.tilePool[tileNumber]) == false or self.tilePool[tileNumber]
== nil then
    -- 테이블이 없었다면 새로운 테이블 공간을 새로 생성합니다.
    self.tilePool[tileNumber] = {}
end

-- 대상 테이블에 대상 엔티티(오브젝트)를 저장합니다.
table.insert(self.tilePool[tileNumber],entity)
-- 엔티티(오브젝트)를 비활성화합니다.
entity.Enable = false
end

@ExecSpace("ClientOnly")
method Entity GetTile(number tileNumber)
-- 타일 풀로부터 지정된 번호 타일 엔티티 반환받는 메서드입니다.

-- 매개변수 1 number tileNumber : 특정 타일셋 번호를 전달받습니다.

-- math.floor메서드를 활용해 tileNumber의 소수점 부분을 버리기 위함입니다. ex) 4.0 => 4
tileNumber = math.floor(tileNumber)

-- 대상 풀(테이블) 유효성 검사를 진행합니다.

```

```

-- 전달받은 tileNumber 타일 오브젝트를 관리하는 풀(테이블)이 없다면
if invalid(self.tilePool[tileNumber]) == false or self.tilePool[tileNumber]
== nil then
    -- 새로운 테이블 공간을 새로 생성합니다.
    self.tilePool[tileNumber] = {}
end

-- /분기1/ 없으면 새로 생성해서 반환합니다.

-- 만약 대상 풀에 담긴 오브젝트가 더 이상 없다면 (모델(프리팍)으로 부터 새로 오브젝트를 생성을 진
행합니다.)
if #self.tilePool[tileNumber] <= 0 then

    -- 테이블 셋에 저장된 EntryID를 확인합니다.
    local entryID = _DataSetLogic.tileSetInfoTable[tileNumber].EntryID
    -- 사전에 미리 저장된(프리팍) 이름의 형태로 이름을 저장합니다.
    local name = "Model_TileSet0"..tileNumber

    -- 임시 위치에 대상 모델(프리팍) 오브젝트를 생성합니다.
    local tempTile =
    _SpawnService:SpawnByModelId(entryID,name,Vector3.up*100,_EntityService:Get
    Entity("d24b5917-1821-4743-ae07-77cc9bfc2d0a"))

    -- 해당 오브젝트를 반환합니다.
    return tempTile
end

-- /분기2/ 있었다면 마지막 요소를 반환합니다.

-- 현재 대상 테이블 수를 확인합니다.
local tableCount = #self.tilePool[tileNumber]

-- 대상 테이블의 마지막 요소를 반환받으며, 해당 엔티티를 테이블에서 제거합니다.
local tempTile = table.remove(self.tilePool[tileNumber],tableCount)

-- 선택한 오브젝트를 활성화합니다.
tempTile.Enable = true

-- 오브젝트를 반환합니다.
return tempTile

end

@ExecSpace("ClientOnly")
method Entity GetItem(string itemName)
-- 오브젝트 풀로부터 아이템 오브젝트를 반환받는 메서드입니다.

-- 매개변수 1 string itemName : 아이템의 이름을 전달받습니다.

-- 아이템 풀에서 대상 아이템들이 담긴 테이블을 가져옵니다.
local targetTable = self.itemPool[itemName]

-- 테이블 내부 요소가 비어있을때

```

```

if #targetTable == 0 then
    -- 대상 아이템을 config에서 설정한 생성 개수만큼 추가 생성을 진행합니다.
    self.CreateItem(itemName, _Config.itemPoolInitCount)
end

-- 가져오고자 하는 테이블로부터 엔티티를 불러옵니다. (테이블의 마지막요소)
---@type Entity
local targetEntity = targetTable[#targetTable]

-- 엔티티를 저장하였으므로 테이블의 마지막 요소를 제거합니다.
table.remove(targetTable, #targetTable)

-- 엔티티(오브젝트)를 활성화합니다.
targetEntity.Enable = true

-- 저장했던 엔티티를 반환합니다.
return targetEntity
end

@ExecSpace("ClientOnly")
method void ReturnItem(Entity entity)
    -- 아이템 오브젝트를 다시 오브젝트 풀로 반환하는 메서드입니다.

    -- 매개변수 1 Entity entity : 반환하고자 하는 아이템 오브젝트 엔티티를 전달합니다.

    -- 반환하는 아이템의 이름을 확인합니다.
    local itemName = entity.Item.name
    -- 대상 테이블로 엔티티를 집어넣습니다.
    table.insert(self.itemPool[itemName], entity)

    -- 타일셋의 기존 Parent(타일셋의 아이템을 배치하기 위해 사전에 등록한 오브젝트)으로부터
    -- 다시 ItemPool 로 Parent를 이전합니다.
    entity.AttachTo(self.ItemPoolParent)
    -- 엔티티(오브젝트)를 비활성화합니다.
    entity.Enable = false
end

@ExecSpace("ClientOnly")
method void CreateItem(string itemName, number createCount)
    -- 모델(프리팹)으로부터 아이템 오브젝트를 새로 생성하여 대상 풀로 집어넣는 메서드입니다.

    -- 매개변수 1 string itemName : 생성하고자 하는 아이템 이름을 전달받습니다.
    -- 매개변수 2 number createCount : 생성할 개수, 오브젝트 풀로 몇개의 오브젝트를 생성해 넣
    어둘지를 결정합니다.

    -- 임시 테이블을 선언합니다.
    local tempTable

    -- 데이터셋에서 참조할 Id 번호를 설정합니다.
    local dataSetID = self.itemDataSetIDTable[itemName]

    -- 데이터셋의 대상 아이템 모델의 EntryID 값을 불러옵니다.
    local entryID = _DataSetLogic.itemInfoTable[dataSetID].EntryID

```

```

-- 데이터셋의 대상 아이템 모델의 Name 값을 불러옵니다.
local name = _DataSetLogic.itemInfoTable[dataSetID].Name
-- 데이터셋의 대상 아이템 모델의 Value 값을 불러옵니다.
local value = _DataSetLogic.itemInfoTable[dataSetID].Value

-- 대상 테이블의 유효성 검사를 진행합니다.

-- /분기 1/ 대상 아이템 풀 내에 대상 풀(테이블) 이 없을 경우
if isValid(self.itemPool[name]) == false or self.itemPool[name] == nil then
    -- 새로운 임시 테이블을 생성합니다.
    tempTable = {}
    -- 아이템 풀에서 내부 풀(테이블)을 임시테이블로 설정합니다.
    self.itemPool[name] = tempTable

-- /분기 2/ 대상 아이템 풀 내에 대상 풀이 이미 있을 경우
else
    -- 임시 테이블(풀) 을 기존에 활용하고 있던 테이블로 선언합니다.
    tempTable = self.itemPool[name]
end

-- 매개변수2 createCount 만큼의 반복문을 수행합니다.
for i = 1, createCount, 1 do
    -- SpawnService를 활용해 오브젝트를 생성합니다.
    local entity = _SpawnService:SpawnByModelId(entryID,name,Vector3.up *
100, self.ItemPoolParent)

    -- 생성한 오브젝트의 설정작업을 진행합니다.

    -- 생성한 오브젝트의 아이템의 고윳값을 설정합니다.
    entity.Item.value = value
    -- 생성한 오브젝트의 아이템의 name 변수를 설정합니다.
    entity.Item.name = name

    -- 담겨야하는 테이블(풀)로 생성된 엔티티를 삽입합니다.
    table.insert(tempTable,entity)

    -- 생성된 엔티티 오브젝트를 비활성화합니다.
    entity.Enable = false
end
end

@ExecSpace("ClientOnly")
method void InitItemPool()
-- 아이템 풀을 초기화하는 메서드입니다.

-- DataSetLogic 으로부터 아이템 데이터셋이 담긴 테이블을 불러옵니다.
local itemDataTable = _DataSetLogic.itemInfoTable

-- Coin 아이템을 Config에서 설정한 생성개수만큼 생성합니다.
self:CreateItem("Coin",_Config.itemPoolInitCount)
-- Health 아이템을 Config에서 설정한 생성개수만큼 생성합니다.
self:CreateItem("Health",_Config.itemPoolInitCount)
-- Booster 아이템을 Config에서 설정한 생성개수만큼 생성합니다.
self:CreateItem("Booster",_Config.itemPoolInitCount)

```

```

end

@ExecSpace("ClientOnly")
method void Init()
-- ObjectPool Logic의 초기화를 수행하는 메서드입니다.

-- 아이템 풀 초기화를 진행합니다.
self:InitItemPool()
end

@ExecSpace("ClientOnly")
method number GetTileNumber(Entity entity)
-- 타일 번호를 추출하는 메서드입니다.

-- 매개변수 1 Entity entity : 대상 엔티티를 전달받습니다.

-- 엔티티의 이름을 추출합니다.
local tileSetName = entity.Name
-- 이름의 마지막 문자열을 추출합니다. ('Model_TileSet02' 문자열에서 14,15번째 문자 추출 > 02)
local numberStr = string.sub(tileSetName,14,15)
-- 문자열을 숫자값으로 변환해 반환합니다.
return tonumber(numberStr)
end

end

```

- ObjectPoolLogic은 주기적으로 생성되며 파괴되는 바닥 타일셋, 각 아이템 요소를 관리합니다.
- 작성된 GetTile 메서드는 타일 풀에서 대기 중인 바닥 타일을 꺼내오는 기능을 수행하며, ScrollingController 컴포넌트에서 새로운 맵 조각을 스폰해야 할 때 호출됩니다.
- ReturnTile 메서드는 화면 밖으로 지나간 바닥 타일이 DeadZone과 충돌했을 때, 해당 타일을 파괴하는 대신 풀로 반환(비활성화)하기 위해 TileDeadZone 스크립트에서 사용됩니다.

최소 구현 기준

- 맵 스크롤 진행 시 타일셋 엔티티가 반복적으로 생성 및 소멸(Destroy)되지 않고, 오브젝트 풀 시스템을 통해 정상적으로 비활성화 및 재활성화되며 순환하도록 구현합니다.
- 모든 타일셋은 오브젝트 풀에서 생성 또는 사용 가능한 엔티티를 반환하는 형태로 구현합니다.

응용 및 확장 방향

- 오브젝트 풀의 원칙인 **미리 생성**하여 사용할 수 있도록 코드를 수정합니다.
- 수정된 코드를 바탕으로 바닥 타일뿐만 아니라 게임 내에 빈번하게 등장하는 장애물이나 아이템 요소들도 오브젝트 풀을 통해 정상적으로 재사용되는지 확인합니다.
- 무조건 많은 양을 초기에 생성해 두는 것이 아니라, 오브젝트 풀 내의 대기 엔티티가 부족할 때만 유동적으로 풀의 크기를 늘려(추가 생성) 메모리를 더욱 최적화하는 확장 로직을 구현해 봅니다.

[💡 기초 학습]플레이어 캐릭터 구현하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 플레이어 캐릭터 구현하기: 00:57:50

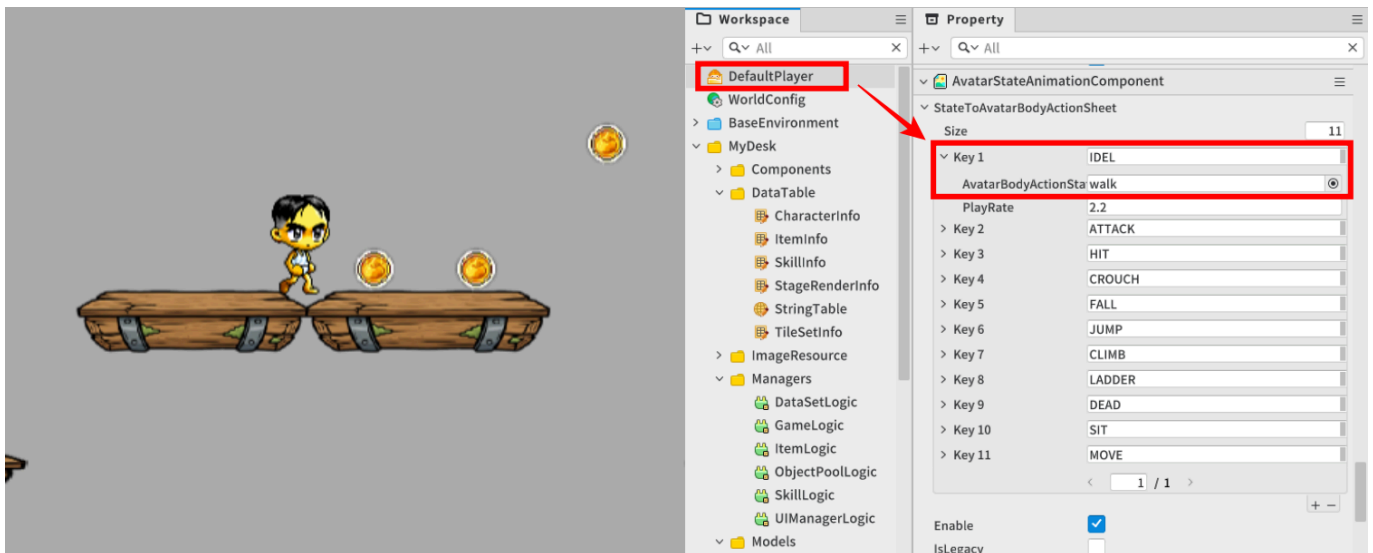
학습 목표

- 키보드 입력을 감지하여 캐릭터에 물리적인 힘을 가하는 점프 기능을 구현할 수 있습니다.
- 다양한 캐릭터를 생성하는 방법에 대해 학습합니다.
- 캐릭터별 사용 가능한 스킬을 정의하고, 스킬 사용시 해당 효과를 구현하는 방법을 학습합니다.
- 플레이어에게 다양한 캐릭터를 선택할 수 있는 환경을 제공할 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

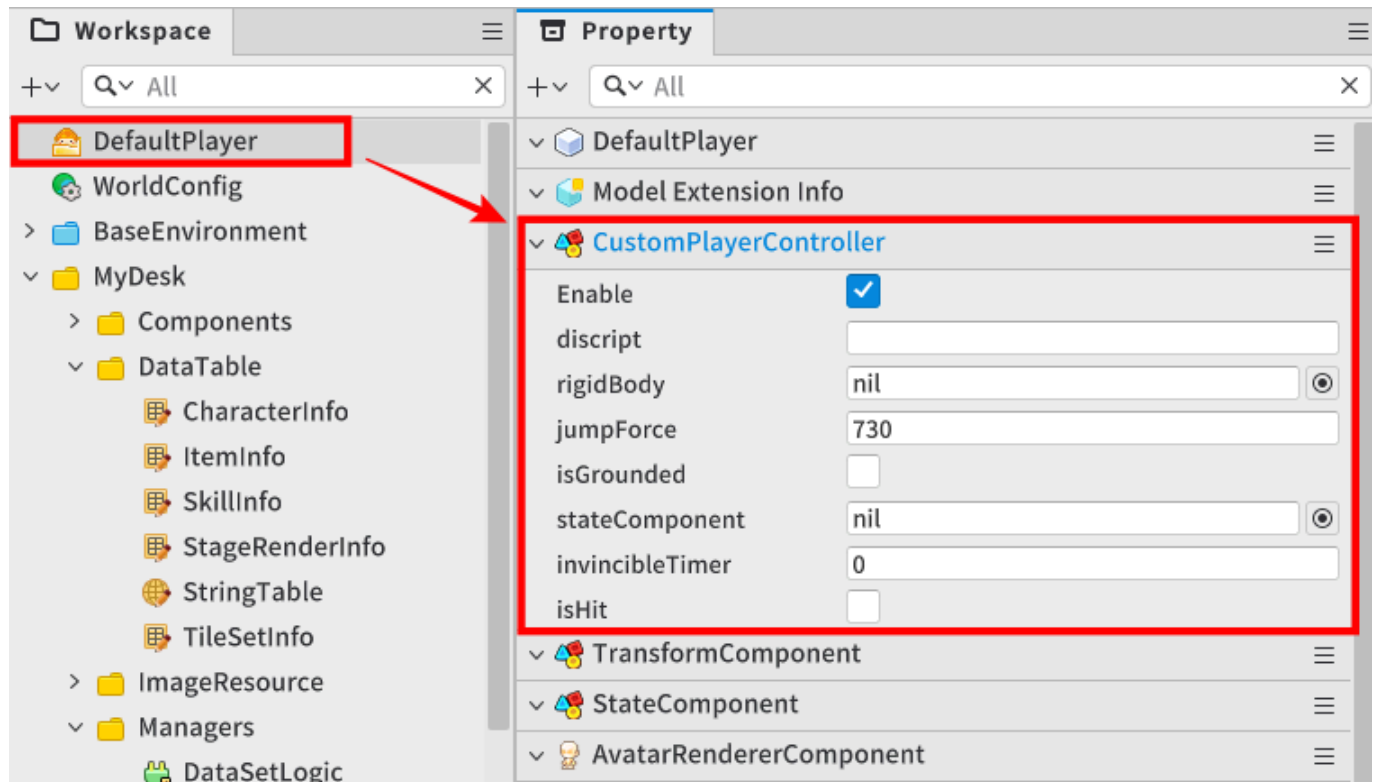
- 샘플 월드에서 제공하는 스킬 키 외 다른 입력 방식으로 스킬을 발동할 수 있도록 세팅해 봅니다.
- 캐릭터의 점프 힘을 변경하여 원하는 물리적 연출을 변경해 봅니다.
- CharacterInfo DataSet에 새로운 캐릭터를 추가하고 캐릭터 선택 창에 새로운 캐릭터로 플레이가 가능한지 확인해 봅니다.

캐릭터 기본 상태 설정하기



- 샘플 월드의 캐릭터는 실제로 이동하지 않는 상태로 게임이 진행되고 있으므로 기본적으로 서 있는 상태가 아닌 달리고 있는 애니메이션이 재생되어야 합니다.
- 캐릭터의 모델이 되는 DefaultPlayer의 컴포넌트 중 AvatarStateAnimationComponent에서 기본적으로 서 있는 상태(IDLE)에 재생되는 애니메이션을 걷는(walk) 애니메이션으로 변경합니다.

캐릭터 기능 제어하기



- 캐릭터의 모델이 되는 DefaultPlayer의 다양한 기능을 직접 코드로 제어하기 위해 CustomPlayerController 컴포넌트 스크립트를 제작하여 부착합니다.

CustomPlayer 스크립트 작성하기

- 다음과 같이 코드를 작성해야 합니다.
- 해당 코드는 Plain Text 환경으로 작성되었습니다.

```
@Component
script CustomPlayerController extends Component

property string discript = "" -- 캐릭터의 기능 및 상태를 정의하기 위한 컴포넌트입니다.

property PhysicsRigidbodyComponent rigidBody = nil

property number jumpForce = 730 -- 캐릭터의 점프 수행 시 필요한 힘 값입니다.

property boolean isGrounded = false -- 캐릭터가 현재 바닥에 있는 상태인지를 확인하는 변수입니다.

property StateComponent stateComponent = nil

property number invincibleTimer = 0 -- 장애물 충돌 시 무적시간을 제어하기 위한 타이머 변수입니다.

property boolean isHit = false -- 캐릭터가 현재 장애물 피격상태인지 확인하는 변수입니다.

@ExecSpace("ClientOnly")
method void Init()
-- CustomPlayerController 컴포넌트 초기화를 진행하는 메서드입니다.
```

```

-- 내부 변수 참조 연결을 수행합니다.
self:ConnectRefs()

-- 기본 상태를 MOVE 상태로 진행합니다.
self:ChangeState("MOVE")

end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 내부에 필요한 멤버들의 참조를 연결하는 메서드입니다.

-- 캐릭터에 부착된 Rigidbody 컴포넌트를 연결합니다.
self.rigidBody = self.Entity.PhysicsRigidbodyComponent
-- 캐릭터에 부착된 State 컴포넌트를 연결합니다.
self.stateComponent = self.Entity.StateComponent
end

@ExecSpace("ClientOnly")
method void Jump()
-- 캐릭터의 점프동작을 수행하는 메서드입니다.

-- 만약 바닥 감지를 못했다면
if self.isGrounded == false then
    -- 점프 수행을 중단합니다.
    return
end

-- 캐릭터의 y 축으로 적용되고 있던 속도값을 0으로 설정함으로 y 축 운동을 제거합니다.
self.rigidBody:SetLinearVelocityY(0)

-- y 축 방향으로 rigidbody를 통해 Force값 적용합니다.
self.rigidBody:ApplyForce(Vector2.up * self.jumpForce)

-- 점프 로직을 수행할때 누워 있는 상태가 아니라면
if self.stateComponent.CurrentStateName ~= "CROUCH" then
    -- 점프 상태로 전환합니다.
    self:ChangeState("JUMP")
end

_SoundService:PlaySound("24ee4f2425924a1abd4f655e8faf2791",1)

-- 점프를 수행하므로 캐릭터가 현재 바닥에 붙어있는 상태가 아님을 설정합니다.
self.isGrounded = false
end

@ExecSpace("ClientOnly")
method void ChangeState(string state)
-- 전달받은 상태로 캐릭터의 상태를 변경시키는 메서드입니다. (캐릭터의 애니메이션 제어 용도입니다.)

-- 매개변수 1 string state : 문자열 형태로 변경하려는 상태를 전달받습니다.

-- 멤버로 선언된 state 컴포넌트로 상태를 설정합니다.
self.stateComponent:ChangeState(state)

```

```

-- 트리거 콜라이더 박스 크기 조정 : 캐릭터가 었드려있을때와 서 있는 상태의 콜라이더 상태를 구분짓
기 위함입니다.

-- 매개변수로 전달받은 상태가 CROUCH 값 이라면
if state == "CROUCH" then
    -- 자신(캐릭터) 엔티티에 포함된 Trigger 컴포넌트의 트리거 Box 사이즈를 Config 에서 설
정한 사이즈로 설정합니다.
    self.Entity.TriggerComponent.BoxSize = _Config.crouchTriggerSize
    -- Trigger 컴포넌트의 트리거 Box 가 Pivot(기준)으로 부터 얼마나 떨어져 있어야하는지 오
프셋값을 설정합니다.
    self.Entity.TriggerComponent.BoxOffset = _Config.crouchTriggerOffset

-- 매개변수로 전달받은 상태가 MOVE 값 이라면
elseif state == "MOVE" then
    -- 자신(캐릭터) 엔티티에 포함된 Trigger 컴포넌트의 트리거 Box 사이즈를 Config 에서 설
정한 사이즈로 설정합니다.
    self.Entity.TriggerComponent.BoxSize = _Config.standTriggerSize
    -- Trigger 컴포넌트의 트리거 Box 가 Pivot(기준)으로 부터 얼마나 떨어져 있어야하는지 오
프셋값을 설정합니다.
    self.Entity.TriggerComponent.BoxOffset = _Config.standTriggerOffset
end
end

@ExecSpace("ClientOnly")
method void ContactGround()
-- 바닥을 감지했을 때 수행하는 메서드입니다.

-- /분기1/ 현재 이미 바닥에 접촉해있는 중이라면
if self.isGrounded == true then
    -- 아무것도 수행하지 않고 메서드를 종료합니다.
    return
end

-- /분기2/ 기존에 바닥과 접촉이 아닌 상태에서 바닥을 접촉했다면

-- 현재 바닥상태임으로 상태를 변경합니다.
self.isGrounded = true

-- 바닥 감지 시 이동 애니메이션을 재생합니다.
self:ChangeState("MOVE")
end

@ExecSpace("ClientOnly")
method void ContactItem(Item item)
-- 아이템별로 수행되어야 할 동작을 진행합니다.

-- 매개변수 1 Item item : 아이템 오브젝트에 포함된 Item 컴포넌트를 전달받습니다.

-- 아이템 습득 시 효과음 재생
_SoundService:PlaySound("ef4235b158414f5283c068bad71f403a",1)

-- 전달받은 아이템 컴포넌트의 name 변수가 Coin 이라면
if item.name == "Coin" then

```

```

    -- 게임로직의 점수 추가 메서드를 수행하며 전달받은 아이템의 Value 값을 전달합니다.
    _GameLogic:AddScore(item.value)
end

-- 전달받은 아이템 컴포넌트의 name 변수가 Health 라면
if item.name == "Health" then
    -- 게임로직의 체력 추가 메서드를 수행하며 전달받은 아이템의 Value 값을 전달합니다.
    _GameLogic:AddHP(item.value)
end

-- 전달받은 아이템 컴포넌트의 name 변수가 Booster 라면
if item.name == "Booster" then
    -- 아이템로직에서 부스터 아이템 사용 메서드를 호출합니다.
    _ItemLogic:UseBooster()
end

-- 아이템 습득 시 멀리 떨어진 곳으로 위치 이동합니다.
-- (오브젝트 풀로 부터 재생성시 문제가 있으므로 아이템 오브젝트의 위치를 일시적으로 보이지 않는 곳
으로 이동합니다.)
item.Entity.TransformComponent.Position = Vector3.one * 100

-- 오브젝트 풀로 아이템 오브젝트를 반환하는 메서드를 수행합니다.
_ObjectPoolLogic:ReturnItem(item.Entity)
end

@ExecSpace("ClientOnly")
method void ContactObstacle()
    -- 장애물을 접촉했을때 수행되는 메서드입니다.

    -- 무적 지속시간이 남아있다면
    if self.invincibleTimer > 0 then
        -- 아무것도 하지 않고 종료합니다.
        return
    end

    -- Config에 정의한 장애물 접촉 데미지를 게임로직의 체력 추가 메서드로 전달합니다.
    _GameLogic:AddHP(_Config.obstacleDamage)

    -- 피격 중이므로 상태를 설정합니다.
    self.isHit = true

    -- 무적 지속시간 타이머를 Config에서 설정한 초기값으로 설정합니다.
    self.invincibleTimer = _Config.invincibleTime

    -- 피격 이펙트를 재생합니다.
    self:PlayOnDamagedEffect()

    -- 아바타 렌더러의 투명도를 설정합니다. (피격상태를 전달하기 위함)
    self.Entity.AvatarRendererComponent:SetAlpha(0.5)
end

@ExecSpace("ClientOnly")
method void PlayOnDamagedEffect()
    -- 피격 시 발생하는 이펙트를 재생하는 메서드입니다.

```

```

-- 피격 이펙트 리소스 ID 값을 저장합니다.
local effectRUID = "41e13add2ba746d9bbe624a31205a848"
-- 맵 엔티티를 가져옵니다.
local mapEntity = _EntityService:GetEntityByPath("/maps/MainGame")
-- 이펙트 발생지점을 설정합니다.
local position = self.Entity.TransformComponent.Position

-- 이펙트 서비스를 활용해 이펙트를 재생합니다.
_EffectService:PlayEffect(effectRUID,mapEntity,position,0,Vector3.one * 3)
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 매 프레임마다 호출되는 메서드입니다.

-- 피격중인 상태라면
if self.isHit == true then
    -- 피격 타이머가 남은 지속시간이 있다면
    if self.invincibleTimer > 0 then
        -- 피격 타이머에 지속시간을 감소시킵니다.
        self.invincibleTimer -= delta
    -- 피격 타이머가 남은 지속시간이 없다면
    else
        -- 피격상태 해제상태로 변경합니다.
        self.isHit = false
        -- 아바타 렌더러의 투명도값을 1로 초기화합니다.
        self.Entity.AvatarRendererComponent:SetAlpha(1)
    end
end
end

@ExecSpace("ClientOnly")
method void ResetCharacter()
-- 게임 재시작 시 필요한 초기값 설정을 진행하는 메서드입니다.

-- 해당 엔티티의 위치값을 절대좌표 x = 0, y = 1 로 설정합니다.
self.Entity.TransformComponent.Position = Vector3(0,1,0)
-- Rigidbody의 BodyType 을 유동적으로 설정하기 위해 Dynamic으로 설정합니다.
self.rigidBody.BodyType = BodyType.Dynamic

-- 캐릭터를 보이도록 설정합니다.
self.Entity.Visible = true
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 월드 실행시 호출되는 메서드입니다.

-- 해당 컴포넌트 초기화 진행합니다.
self:Init()
end

```

```

@EventSender("Service", "InputService")
handler HandleKeyDownEvent(KeyDownEvent event)
-- 유저가 키 입력을 눌렀을때 호출되는 메서드입니다.

-- 게임이 종료된 상태라면
if _GameLogic.isEnd == true then
-- 해당 메서드 수행을 중단합니다.
    return
end

-- 입력된 키 정보를 저장합니다.
local key = event.key

-- 스페이스바 키 입력 시
if key == KeyboardKey.Space then
    -- 점프 동작을 수행합니다.
    self:Jump()
end

-- 방향키 아래 키 입력 시
if key == KeyboardKey.DownArrow then
    -- 캐릭터의 상태를 'CROUCH' 로 설정합니다.
    self:ChangeState("CROUCH")
end

-- 'K' 키 입력 시
if key == KeyboardKey.K then
    -- 캐릭터의 스킬을 사용합니다.
    _SkillLogic:UseSkill()
end

end

@EventSender("Service", "InputService")
handler HandleKeyUpEvent(KeyUpEvent event)
-- 유저의 키 입력을 종료할때 호출되는 메서드입니다.

-- 게임이 종료된 상태라면
if _GameLogic.isEnd == true then
-- 해당 메서드 수행을 중단합니다.
    return
end

-- 입력된 키 정보를 저장합니다.
local key = event.key
-----

-- 방향키 아래 키 입력 해제 시
if key == KeyboardKey.DownArrow then
    -- 다시 Crouch 상태를 해제합니다.
    self:ChangeState("MOVE")
end
end

```

```

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 캐릭터의 트리거 박스 내 다른 트리거와 충돌 시 호출되는 메서드입니다.

-- 게임이 종료된 상태라면
if _GameLogic.isEnd == true then
-- 해당 메서드 수행을 중단합니다.
    return
end

-- 트리거 충돌의 대상 엔티티를 저장합니다.
local TriggerBodyEntity = event.TriggerBodyEntity
-- 충돌된 트리거의 그룹명을 저장합니다.
local TriggerGroup =
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)
-----

-- 대상 트리거가 아이템 오브젝트의 트리거라면
if _Config.triggerGroup[TriggerGroup] == "Item" then
    -- 아이템을 감지했을 때의 로직을 수행합니다.
    self:ContactItem(TriggerBodyEntity.Item)
end

-- 대상 트리거가 장애물 요소 오브젝트의 트리거라면
if _Config.triggerGroup[TriggerGroup] == "Obstacle" then
    -- 장애물을 감지했을 때의 로직을 수행합니다.
    self:ContactObstacle()
end

-- 대상 트리거가 DeadZone 요소 오브젝트의 트리거라면
if _Config.triggerGroup[TriggerGroup] == "DeadZone" then

    -- 현재 캐릭터와 데드존이 만나는 조건은 화면 하단으로 캐릭터가 떨어지는 상황이므로
    -- 게임 종료 명령 수행이 필요합니다.

    -- Rigidbody의 BodyType 을 Static으로 물리운동이 진행되지 않도록 설정합니다.
    self.rigidbody.BodyType = BodyType.Static
    -- 게임 로직의 게임 종료 메서드를 수행합니다.
    _GameLogic:EndMainGame()
end
end

@EventSender("Self")
handler HandlePhysicsContactBeginEvent(PhysicsContactBeginEvent event)
-- Physics 강체의 물리 충돌이 감지되었을때 발생하는 이벤트입니다.

-- 충돌한 강체의 엔티티를 가져옵니다.
local ContactedBodyEntity = event.ContactedBodyEntity

-- 충돌한 강체의 부모 오브젝트의 TileSet 컴포넌트를 가져옵니다.
local tileSet = ContactedBodyEntity.Parent.TileSet

-- 접촉한 오브젝트에서 타일셋이 검출되었다면

```

```

if tileSet ~= nil then

    -- 접촉한 오브젝트의 Transform 컴포넌트를 가져옵니다.
    local contactTransform = ContactedBodyEntity.TransformComponent
    -- 캐릭터의 Transform 컴포넌트를 가져옵니다.
    local playerTransform = self.Entity.TransformComponent

    -- 만약 충돌체 대상의 오브젝트의 위치 y 값이 캐릭터의 위치 y 값 보다 더 높다면
    if contactTransform.Position.y >= playerTransform.Position.y then
        -- 해당 메서드 수행을 중단합니다.
        return
    end

    -- 현재 바닥에 있는 상태임을 전달합니다.
    self.isGrounded = true
    -- 만약 현재 상태가 'JUMP' 라면
    if self.stateComponent.CurrentStateName == "JUMP" then
        -- 'MOVE' 상태로 전환합니다.
        self:ChangeState("MOVE")
    end
end
end

end

```

점프 기능 수행과정 이해하기

- 설정된 환경에서 활용할 수 있는 점프기능은 캐릭터에 등록된 PhysicsRigidbody의 물리적인 기능을 활용하는 방법이 있습니다.
- CustomPlayerController 컴포넌트 내에서 캐릭터의 물리 기능을 제어하기 위해 PhysicsRigidbodyComponent 타입의 변수를 선언하고 컴포넌트 참조를 연결합니다.
- 엔티티에 위쪽방향으로 힘을 가하기 위해서는 PhysicsRigidbody의 ApplyForce메서드를 활용합니다.
- 캐릭터가 점프상태임을 알 수 있도록 점프 애니메이션을 재생시키도록 합니다.

캐릭터 점프기능 구현 예시

```

PhysicsRigidbodyComponent rigidBody;
StateComponent state;

number jumpForce = 730
boolean isGrounded = false

void OnBeginPlay()
{
    -- 물리적인 힘을 가할 PhysicsRigidbodyComponent 연결
    self.rigidBody = self.Entity.PhysicsRigidbodyComponent
    -- 캐릭터의 상태전환을 위한 StateComponent 연결
    self.state = self.Entity.StateComponent
}

```

```

void Jump()
{
    if self.isGround == false then
        return -- 만약 캐릭터가 지면에 있지 않다면, 점프기능을 수행하지 않고 종료합니다.
    end

    -- 점프 수행이 시작될때, 캐릭터가 받고있는 Y축 속도,힘을 소멸합니다.
    self.rigidBody:SetLinearVelocityY(0)

    -- Vector2.up 캐릭터의 위쪽 방향으로 jumpForce 만큼 힘을 가합니다.
    self.rigidBody:ApplyForce(Vector2.up * self.jumpForce)

    -- 점프 애니메이션을 적용하기 위해 StateComponent에게 Jump상태로 변경을 요청합니다.
    self.state:ChangeState("Jump")

    -- 점프를 수행하므로 캐릭터가 현재 바닥에 있지 않음으로 상태를 전환합니다.
    self.isGround = false;
}

HandleKeyDownEvent(KeyDownEvent event)
{
    -- 키보드의 키가 눌렸을때 호출되는 메서드입니다.

    -- 입력된 키 정보를 저장합니다.
    local key = event.key

    -- 만약 입력된 키가 Space 키라면
    if key == KeyboardKey.Space then
        -- Jump 메서드를 수행합니다.
        self:Jump()
    end
}

```

바닥 감지 시 캐릭터 상태 전환 방식

- 캐릭터의 연속적인 점프 수행을 방지하기 위해 boolean isGround 상태 변수를 활용하고 있습니다.
- 캐릭터의 점프가 시작될 때 해당 변수를 false로 전환합니다.
- 점프가 완료되고 중력에 의해 바닥에 착지했을 때, isGrounded 상태 변수를 다시 true로 전환하여 점프 가능한 상태로 복구해야 합니다.
- PhysicsCollider와 EventHandler를 통해 상태 전환이 가능합니다.

바닥 충돌 상태전환 예시코드

StateComponent state

```

void OnBeginPlay()
{
    -- 캐릭터의 상태전환을 위한 StateComponent 연결

```

```

state = self.Entity.StateComponent
}

HandlePhysicsContactBeginEvent(PhysicsContactBeginEvent event)
{
    -- PhysicsCollider로 부터 다른 충돌체와 충돌이 감지되었을때 호출되는 메서드입니다.

    -- 충돌체 대상의 엔티티를 저장합니다.
    local ContactedBodyEntity = event.ContactedBodyEntity
    -- 충돌체 대상으로부터 TileSet 컴포넌트를 가져오도록 합니다.
    local tileSet = ContactedBodyEntity.TileSet

    -- 만약 TileSet 컴포넌트를 확인했다면 (바닥 타일임을 확인했다면)
    if tileSet ~= nil then
        -- isGround 상태를 true로 전환합니다.
        isGround = true
        -- 점프 상태였던 애니메이션을 다시 걷는 애니메이션으로 전환합니다.
        self.state.ChangeState("Move")
    end
}

```

다양한 캐릭터 제작하기

캐릭터별 정보 테이블 생성하기

	1	2	3	4	+
ID	Name	Speed	SkillId	RUID	
1	TestA	3	1	ac753a90b9814	
2	TestB	4	2	b65a754c25174	
+					

DefaultPlayer
WorldConfig
> BaseEnvironment
MyDesk
> Components
DataTable
CharacterInfo
ItemInfo
SkillInfo
StageRenderInfo
StringTable
TileSetInfo
> ImageResource
Managers
DataSetLogic

대상	설명
Name	유저에게 제공될 캐릭터명 입니다.
Speed	캐릭터마다 스크롤링 속도가 다르도록 설정하기 위해 추가된 값입니다.
Skill id	캐릭터마다 고유한 스킬 번호입니다.
RUID	캐릭터의 등록된 CostumeManagerComponent의 Body파츠에 등록될 리소스 ID 값입니다.

- 제작한 CharacterInfo DataSet은 DataSetLogic에서 InitCharacterInfoTable메서드를 통해 각 캐릭터의 정보가 테이블로 저장됩니다.

스킬 데이터 등록하기

	1	2	3	4	
ID	Name	Value	EffectRUID	UI_RUID	
1	Heal	20	db4e6e7fcb1b4	c99e612dbd54	
2	Bonus	500	db4e6e7fcb1b4	eb0b26a38476	
+					

WorldConfig

- BaseEnvironment
- MyDesk
 - Components
 - Character
 - Item
 - TileSet
 - UI
 - DataTable
 - CharacterInfo
 - ItemInfo
 - SkillInfo**
 - StageRenderInfo
 - StringTable
 - TileSetInfo

대상	설명
Name	유저에게 제공될 스킬명 입니다.
Value	스킬별 사용시 효과별 고윳값입니다.
EffectRUID	스킬별 사용시 이펙트 리소스 ID 값입니다.
UI_RUID	스킬별 UI로 표기될 Sprite 리소스 ID 값입니다.

- 샘플 월드에서 플레이어가 특정 스킬 키 입력 시 게임 진행 중에 발동될 수 있는 기능은 추가 점수 획득, 체력 회복이 있습니다.
- 제작한 CharacterInfo DataSet은 DataSetLogic에서 InitSkillInfoTable메서드를 통해 각 스킬 정보가 테이블로 저장됩니다.

캐릭터별 스킬 연결 방식

CharacterInfo

	1	2	3	4	
ID	Name	Speed	SkillId	RUID	
1	TestA	3	1	ac753a90b9814	
2	TestB	4	2	b65a754c25174	
+					

SkillInfo

	1	2	3	4	
ID	Name	Value	EffectRUID	UI_RUID	
1	Heal	20	db4e6e7fcb1b4	c99e612dbd54	
2	Bonus	500	db4e6e7fcb1b4	eb0b26a38476	
+					

- CharacterInfo DataSet에 등록된 캐릭터별 SkillID는 SkillInfo DataSet의 각 ID 번호를 의미합니다.
 - 예를 들어, TestB 캐릭터의 SkillId가 2라면, SkillInfo 데이터셋에서 인덱스 2번에 해당하는 Bonus 스킬이 해당 캐릭터의 스킬로 할당됩니다.

Skill Logic의 UseSkill 동작 원리

- CustomPlayerController의 HandleKeyDownEvent메서드에서 스킬 발동 키(K key) 입력 시 SkillLogic의 UseSkill메서드를 호출하고 있습니다.
- CustomPlayerController는 키 입력을 받아 SkillLogic에 스킬 발동 명령만 전달할 뿐, 구체적인 스킬 구현 로직은 처리하지 않도록 역할을 명확히 분리합니다.
- Skill Logic은 자신에게 등록된 변수인 skillId 번호에 맞는 스킬을 그대로 발동합니다.
- skillId 번호는 타이틀 화면에서 캐릭터를 선택할 때 값이 변경됩니다.

Skill Logic의 UseSkill 원리

```
number skillId;

void UseSkill()
{
    local skillName = _DataSetLogic.skillInfoTable[self.skillId].Name

    if skillName == "Heal" then
        -- 체력 회복 스킬 사용 시 구체적인 동작과정 정의
    elseif skillName == "Bonus" then
        -- 보너스 스킬 사용 시 구체적인 동작과정 정의
    end
}
```

SkillLogic 스크립트 작성하기

- 다음과 같이 코드를 작성해야 합니다.
- 해당 코드는 Plain Text 환경으로 작성되었습니다.

```
@Logic
script SkillLogic extends Logic

property number discript = 0 -- 캐릭터의 고유 스킬 사용시 구체적인 동작을 수행하기 위해
선언한 로직입니다.

property number skillId = 0 -- 캐릭터의 고유 SkillId 값을 설정하기 위한 변수입니다.

property boolean useable = true -- 스킬사용 가능 상태를 저장하기 위한 상태변수입니다.

@ExecSpace("ClientOnly")
method void UseSkill()
-- 현재 등록된(SkillId)의 구체적인 동작을 수행하는 메서드입니다.

-- 해당 메서드 호출 시 사용할수 있는 상태가 아니라면
if self.useable == false then
    -- 메서드를 종료합니다.
```

```

        return
    end

    -- 현재 캐릭터 스킬명을 저장합니다.
    local skillName = _DataSetLogic.skillInfoTable[self.skillId].Name
    -- 현재 캐릭터 스킬고웁값을 저장합니다.
    local skillValue = _DataSetLogic.skillInfoTable[self.skillId].Value

    -- 스킬명이 'GetLife' 라면
    if skillName == "Heal" then
        -- 스킬 고웁값만큼 체력을 회복합니다.
        _GameLogic:AddHP(skillValue)

    -- 스킬명이 'GetScore' 라면
    elseif skillName == "Bonus" then
        -- 스킬 고웁값만큼 추가 점수를 획득합니다.
        _GameLogic:AddScore(skillValue)
    end

    -- 스킬 이펙트를 재생합니다.
    self:PlaySkillEffect()

    -- 스킬 효과음 재생
    _SoundService:PlaySound("56c1b5e3c7fc4abda561c1cc00813bda",1)

    -- 스킬을 사용했으므로 사용할 수 없음으로 상태를 전환합니다.
    self.useable = false

    -- 스킬 아이콘 UI 의 렌더러의 투명도 값을 낮게 설정합니다.
    _UIManagerLogic.skillRenderer.Color.a = 0.3
end

@ExecSpace("ClientOnly")
method void Reset()
    -- 내부 상태를 초기화하는 메서드입니다.

    -- 스킬을 사용할수 있게 전환합니다.
    self.useable = true
end

@ExecSpace("ClientOnly")
method void PlaySkillEffect()
    -- 스킬 이펙트를 재생하기 위한 메서드입니다.

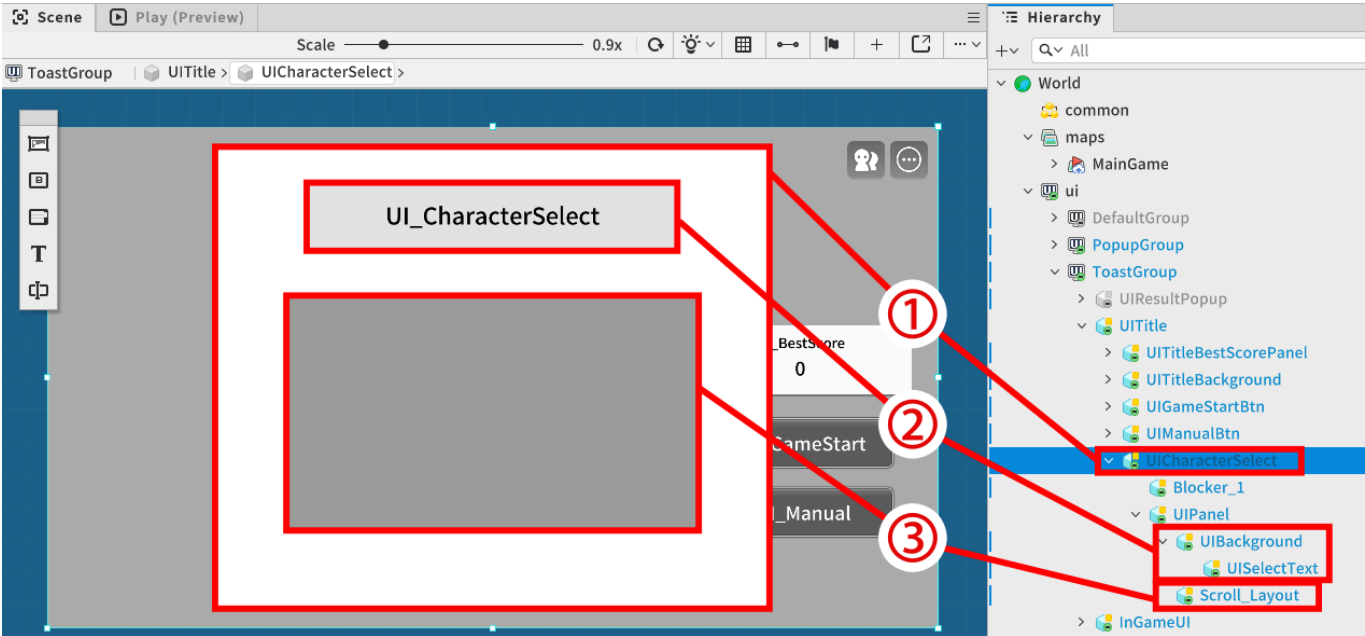
    -- SkillInfoTable에 저장된 스킬 이펙트 RUID 값을 가져옵니다.
    local effect = _DataSetLogic.skillInfoTable[self.skillId].EffectRUID

    -- EffectService를 활용해 캐릭터 오브젝트에 부착하여 스킬 이펙트를 재생합니다.
    _EffectService:PlayEffectAttached(effect,_UserService.LocalPlayer,Vector3.zero,0,Vector3.one)
end

```

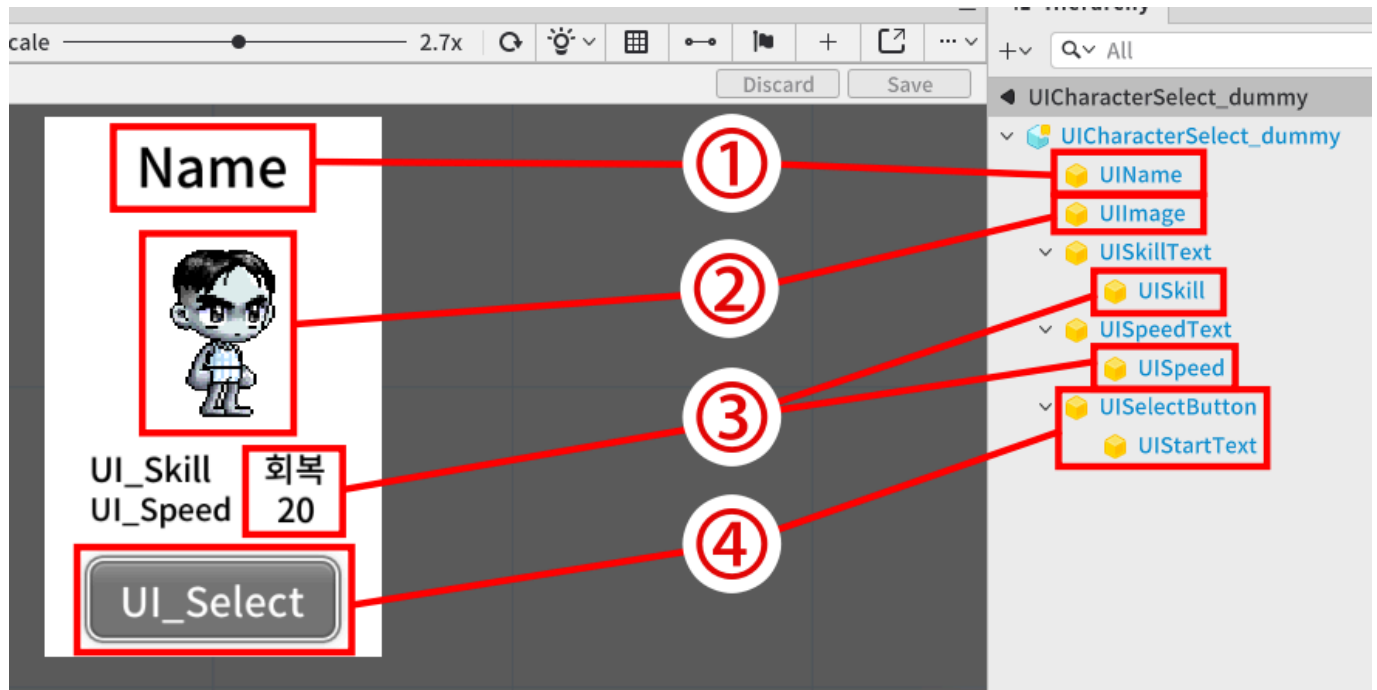
end

캐릭터 선택 구현하기



항목	엔티티	설명
1	UICharacterSelect	캐릭터 선택 창이 최상단 부모가 되는 엔티티로 캐릭터 선택창을 보이게 하거나 보이지 않게 하기 위해 해당 엔티티의 Enable 값을 제어합니다.
2	UIBackground UISelectText	해당 팝업이 캐릭터 선택창임을 나타내는 타이틀 텍스트를 표기하는 구간입니다.
3	Scroll_Layout	다양한 캐릭터창이 표기될 영역으로 'UICharacterSelect' 슬롯들을 나열하는 공간입니다. 아이템요소들이 해당영역의 범위를 초과하는 경우 마우스 드래그를 통해 좌,우로 뷰 영역을 제어할 수 있습니다.

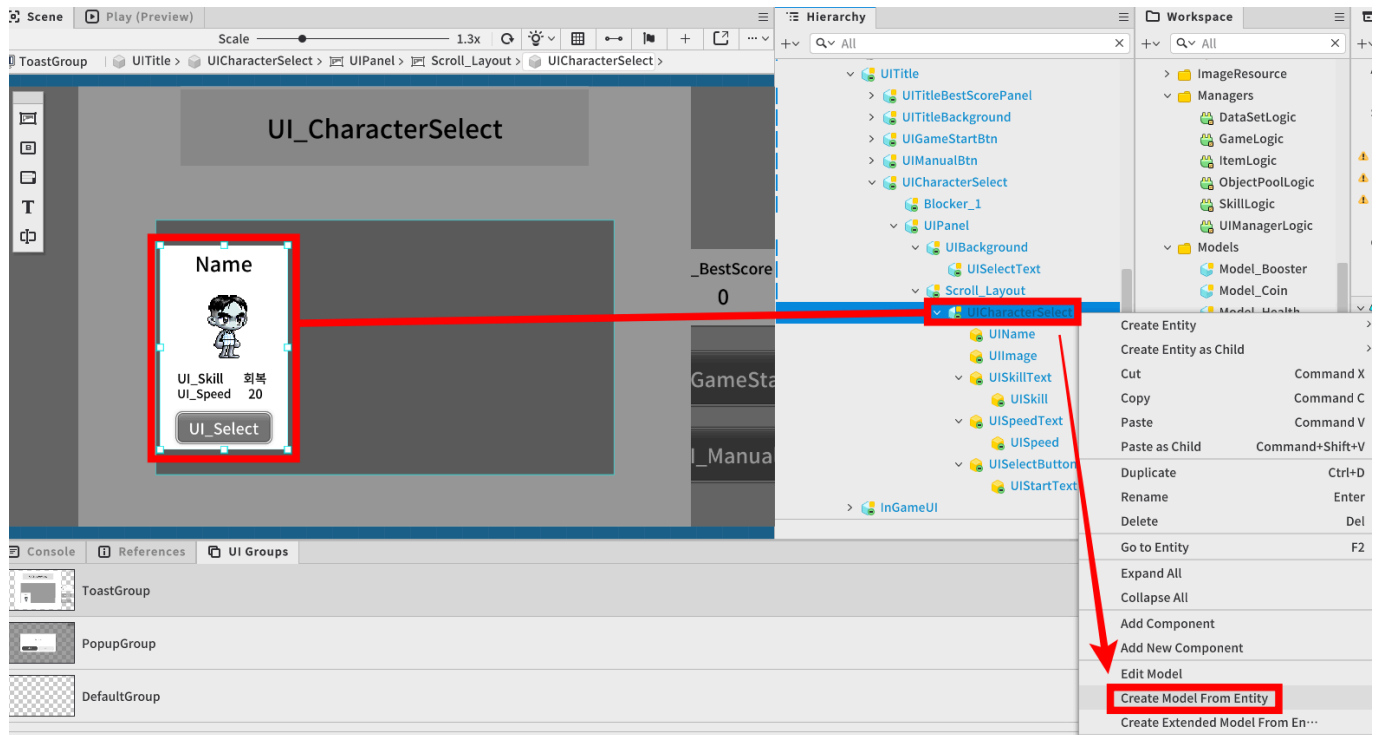
캐릭터 정보를 나타내는 UI Slot 만들기



항목	엔티티	설명
1	UIName	Slot 상단에 캐릭터명을 나타냅니다.
2	UIImage	캐릭터의 외형을 플레이어가 시각적으로 확인할 수 있게 제공합니다.
3	UISkill UISpeed	캐릭터별 사용 가능한 스킬과 캐릭터별 스크를 속도를 나타냅니다.
4	UISelectButton	Slot에 등록된 캐릭터로 플레이를 원할 경우 해당 캐릭터로 본격적인 게임을 진행할 수 있는 버튼을 제공합니다.

- UI Slot이란, 하나의 엔티티 안에 개별 캐릭터의 고유 정보(초상화, 이름, 능력치 등)를 담아 화면에 표시하는 독립적인 UI 단위를 의미합니다.
- 리스트 형태로 여러 캐릭터를 나열할 때, 이 UI Item 하나를 '기본 틀(모듈)'로 삼아 데이터만 교체하며 반복적으로 재사용하게 됩니다.

Slot 아이템 Model 제작하기



- 하나의 캐릭터 슬롯을 제작할 때, 실제 UI 배치 또는 크기를 직관적으로 확인하기 위해 직접 슬롯 내에서 UI를 배치해보며 제작을 진행하는 방식을 추천합니다.
- 제작된 하나의 단위 Slot이 제작되었다면 해당 엔티티를 우클릭 후 Create Model From Entity를 선택해 Workspace에 재사용할 하나의 모델로 등록합니다.

캐릭터 Slot에 캐릭터별 정보 등록하기

- 월드 실행 시 모든 캐릭터의 Slot 아이템을 미리 전부 생성합니다.
- 각 슬롯은 CharaterInfo DataSet의 모든 요소와 캐릭터의 데이터를 전부 출력해야 합니다.
- Slot에 캐릭터 정보를 등록하는 과정은 다음과 같습니다.
 - 월드 실행 -> 자동으로 호출되는 OnBeginPlay내에서 RegisterCharacter메서드를 호출하도록 합니다.
 - RegisterCharacter메서드의 기능은 다음과 같습니다.

UICharacterSelectPopup 의 RegisterCharacter메서드

Entity scrollViewEntity; -- Slot 아이템이 배치되어야할 부모 엔티티

```
void RegisterCharacter()
{
```

-- Workspace에 등록된 Slot 아이템의 ModelID를 저장합니다.

```
local modelId = _EntryService:GetModelIdByName("UICharacterSelect")
```

-- 캐릭터 정보 테이블을 호출하는 코드를 간결하게 작성하기 위해 지역변수로 저장합니다.

```
local characterTable = _DataSetLogic.characterInfoTable
```

-- 캐릭터 정보 테이블에 등록된 개수만큼 순회를 진행합니다. (CharacterInfo DataSet에 등록된 캐릭터 수)

```
for i = 1, #characterTable, 1 do
```

-- SpawnService의 SpawnByModelID메서드를 활용해 하나의 Slot아이템을 생성합니다.

```
local characterInfo = _SpawnService:SpawnByModelID(
```

```
modelId,"characterInfoPopup",Vector3.zero,scrollViewEntity).UICharacterInfo
Component

-- 캐릭터의 이름을 표기하는 UI의 텍스트를 캐릭터 이름으로 설정합니다.
characterInfo.nameText.Text = characterTable[i].Name

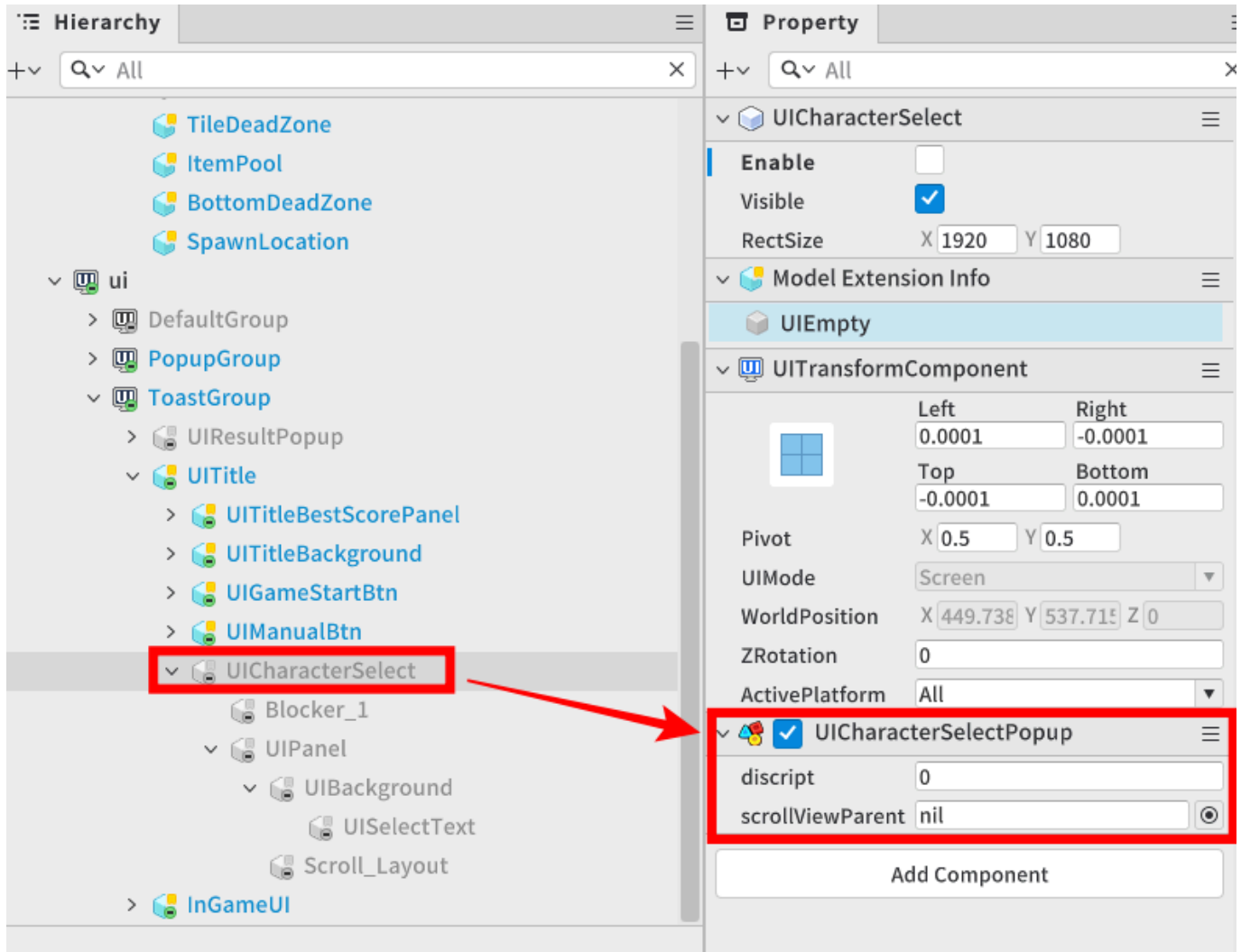
-- Slot에 미리 등록된 캐릭터 외형(CustomBodyEquip)의 리소스 ID 값을 설정합니다.
characterInfo.costumImage.CustomBodyEquip = characterTable[i].RUID

-- CharacterInfo DataSet에 저장된 SkillId 번호를 추출합니다.
local skillId = tonumber(characterTable[i].SkillId)
-- SkillInfo DataSet에 저장된 Skill 정보를 불러와 해당 스킬의 이름을 등록합니다.
characterInfo.skillText.Text =
_DataSetLogic.skillInfoTable[skillId].Name

-- 캐릭터의 스크롤 속도를 표기하는 UI의 텍스트를 캐릭터 고유 스크롤 속도로 설정합니다.
characterInfo.speedText.Text = characterTable[i].Speed

end
}
```

캐릭터 선택창 제작하기



- 캐릭터 선택 창 의 최상위 엔티티에 월드 실행 시 캐릭터 Slot을 생성 기능을 수행하는 컴포넌트를 등록합니다.
- 제작이 완료된 경우, 플레이어가 게임 시작 버튼을 클릭하기 전까지 해당 팝업이 보이지 않아야하므로 엔티티를 비활성화 해둡니다.
- 다음과 같이 코드를 작성해야 합니다.
- 해당 코드는 Plain Text 환경으로 작성되었습니다.

```
@Component
script UICharacterSelectPopup extends Component

property number discript = 0 -- 캐릭터 선택창 팝업을 띄울때 필요한 기능을 정의한 컴포넌트
입입니다.

property Entity scrollViewParent = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 월드 시작 시 호출되는 메서드입니다.

-- 모델(프리팹)으로 부터 생성되는 오브젝트들의 부모로 설정될 엔티티를 연결합니다.
self.scrollViewParent =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect/UI
Panel/Scroll_Layout")
```

```

-- 캐릭터 정보등록을 진행합니다.
self:RegistCharacter()
end

@ExecSpace("ClientOnly")
method void RegistCharacter()
-- 캐릭터 선택 창 내 캐릭터 정보를 등록하는 메서드입니다.
-- 모델(프리팹)으로 부터 캐릭터 정보를 나타내는 팝업이 생성될때 내부에 기입되는 데이터 값들을 설정
하기 위함입니다.

-- 기본 모델(프리팹)요소의 모델Id를 가져옵니다.
local modelId = _EntryService:GetModelIdByName("UICharacterSelect")
-- 데이터셋으로부터 캐릭터 정보 테이블을 가져옵니다.
local characterTable = _DataSetLogic.characterInfoTable

-- 테이블 내에 등록된 캐릭터 수만큼 반복문을 수행합니다.
for i = 1, #characterTable, 1 do

    -- 스폰 서비스를 활용해 UI 팝업을 생성하며 캐릭터인포 컴포넌트를 저장합니다.
    local characterInfo =
    _SpawnService:SpawnByModelId(modelId,"characterInfoPopup",Vector3.zero,self
    .scrollViewParent).UICharacterInfoComponent

    -- 스킬인포 테이블을 식별하기 위한 skillId값을 불러옵니다.
    local skillId = tonumber(characterTable[i].SkillId)
    -- 캐릭터 인포 내에 설정될 캐릭터 스킬 번호를 설정합니다.
    characterInfo.skillId = skillId

    -- 캐릭터 인포 내 설정될 캐릭터명을 설정합니다.
    characterInfo.nameText.Text = characterTable[i].Name
    -- 캐릭터 인포 내 코스튬 바디 RUID값을 설정합니다.
    characterInfo.costumeImage.CustomBodyEquip = characterTable[i].RUID
    -- 캐릭터 인포 내 스킬 명을 설정합니다 (저장된 SkillId 번호를 기반으로 스킬 인포 테이블에
    서 스킬명 가져옵니다.)
    characterInfo.skillText.Text =
    _DataSetLogic.skillInfoTable[skillId].Name
    -- 캐릭터 인포 내 속도 텍스트를 설정합니다.
    characterInfo.speedText.Text = characterTable[i].Speed

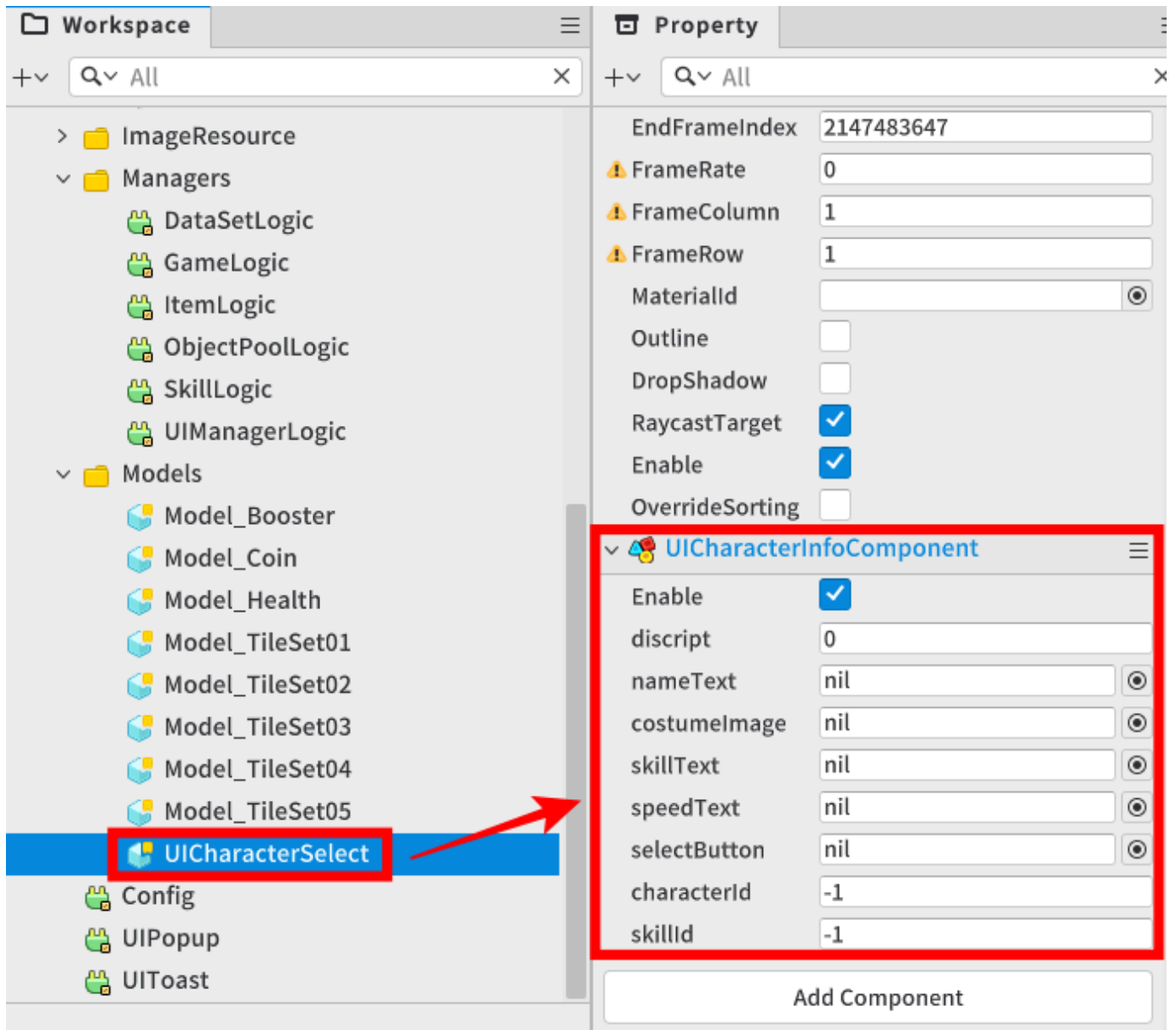
    -- 캐릭터 ID(인덱스)값을 설정합니다.
    characterInfo.characterId = i
end

end

end

```

캐릭터 Slot 제작하기



- Slot의 하단에 배치된 캐릭터 선택 기능을 수행할 버튼의 기능을 동작시키기 위한 스크립트를 제작합니다.
- 버튼의 이벤트로 CloseSelectPopup메서드를 호출할 수 있도록합니다.
- CloseSelectPopup 메서드는 다음과 같은 동작을 수행합니다.
 - 게임 시작 시 보이는 타이틀화면을 구성하는 UI요소 비활성화합니다.
 - 플레이어가 사용하는 캐릭터의 외형을 변경합니다.
 - 해당 캐릭터의 스킬 번호를 SkillLogic에게 전달합니다.
 - GameLogic에게 현재 캐릭터가 몇 번의 캐릭터인지 전달하며 게임시작 기능을 호출합니다.
- 다음과 같이 코드를 작성해야 합니다.
- 해당 코드는 Plain Text 환경으로 작성되었습니다.

```
@Component
script UICharacterInfoComponent extends Component

property number discript = 0 -- 캐릭터 선택 창에서 각 캐릭터의 정보를 표기하기 위한 컴포넌트입니다.

property TextComponent nameText = nil

property CostumeManagerComponent costumeImage = nil
```

```

property TextComponent skillText = nil

property TextComponent speedText = nil

property ButtonComponent selectButton = nil

property number characterId = -1 -- 캐릭터 인덱스를 구분하기 위한 변수입니다.

property number skillId = -1 -- 캐릭터 고유 스킬 번호입니다.

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 월드 시작 시 호출되는 메서드입니다.

-- 해당 컴포넌트 초기화를 진행합니다.
self:Init()
end

@ExecSpace("ClientOnly")
method void Init()
-- CharacterInfo 컴포넌트 초기화를 진행하는 메서드입니다.

-- 모델(프리팸) 자식 오브젝트 컴포넌트 연결을 수행합니다.

-- 캐릭터명을 기입할 텍스트 컴포넌트를 자식오브젝트에서 가져옵니다.
self.nameText = self.Entity:GetChildByName("UIName").TextComponent
-- 캐릭터 이미지를 등록 할 코스튬 컴포넌트를 자식오브젝트에서 가져옵니다.
self.costumeImage =
self.Entity:GetChildByName("UIImage").CostumeManagerComponent
-- 스킬명을 기입할 텍스트 컴포넌트를 자식오브젝트에서 가져옵니다.
self.skillText = self.Entity:GetChildByName("UISkill",true).TextComponent
-- 속도값을 기입할 텍스트 컴포넌트를 자식오브젝트에서 가져옵니다.
self.speedText = self.Entity:GetChildByName("UISpeed",true).TextComponent
-- 마우스 클릭을 감지할 버튼 컴포넌트를 자식오브젝트에서 가져옵니다.
self.selectButton =
self.Entity:GetChildByName("UISelectButton").ButtonComponent

-- 캐릭터 선택버튼 클릭시 수행될 메서드(CloseSelectPopup)를 등록합니다.
self.selectButton.Entity:ConnectEvent(ButtonClickEvent,self.CloseSelectPopu
p)

end

@ExecSpace("ClientOnly")
method void SetInfo(string name, string ruid, string skill, string speed)
-- 캐릭터 정보창에 필요한 정보,내용을 설정하기 위한 메서드입니다.

-- 매개변수 1 string name : 캐릭터 명을 전달 받습니다.
-- 매개변수 2 sting ruid : 캐릭터 몸통 RUID 값을 전달 받습니다.
-- 매개변수 3 string skill : 캐릭터 스킬명을 전달 받습니다.
-- 매개변수 4 string speed : 캐릭터 스크롤링 속도값을 전달받습니다.

-- 캐릭터 명 텍스트를 설정합니다.

```

```

self.nameText.Text = name
-- 코스튬 바디파츠 부분 리소스를 설정합니다.
self.costumeImage.CustomBodyEquip = ruid
-- 스킬 명 텍스트를 설정합니다.
self.skillText.Text = skill
-- 스크롤링 속도값 텍스트를 설정합니다.
self.speedText.Text = speed
end

@ExecSpace("ClientOnly")
method void CloseSelectPopup()
-- 캐릭터 선택창 팝업을 비활성화 시키는 메서드입니다.

-- 게임 시작전 타이틀 패널(시작버튼, 조작법, 캐릭터 선택 창의 부모) 엔티티를 가져옵니다.
local TitlePanel = _EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")

-- 패널 오브젝트를 비활성화합니다.
TitlePanel.Enable = false

-- 캐릭터의 코스튬 바디 리소스값을 설정합니다.
_UserService.LocalPlayer.CostumeManagerComponent.CustomBodyEquip =
self.costumeImage.CustomBodyEquip

-- 게임내에서 사용할 캐릭터가 고유한 스킬Id를 설정합니다.
_SkillLogic.skillId = self.skillId

-- 게임 로직에 선택한 캐릭터Id 번호로 게임 진행을 요청합니다.
_GameLogic:GameStart(self.characterId)
end

end

```

최소 구현 기준

- 플레이어 캐릭터가 바닥 타일과의 충돌을 정상적으로 감지하며, 공중에서 무한히 도약하지 않고 지면 상태에서만 점프를 수행하도록 제한 기능이 정상적으로 동작해야 합니다.
- 캐릭터 선택 창에서 캐릭터 Slot에 등록된 버튼 입력 시 GameLogic에게 GameStart를 호출할 수 있도록 제작되어야 합니다.

응용 및 확장 방향

- CharacterInfo DataSet에 캐릭터별 스크롤링 속도 외에도 점프력, 최대 체력, 무적 시간 등의 세부 능력치 열을 추가해 캐릭터마다 다른 플레이 스타일을 제공해 볼 수 있습니다.
- 장애물을 파괴하며 달리는 '무적 돌진'이나 주변의 코인을 한 번에 흡수하는 등 다채로운 액션 스킬을 기획하고 추가해 봅니다.
- 캐릭터를 선택할 때 바디 코스튬 하나만 교체하는 것을 넘어, 헤어, 얼굴, 망토 등 다양한 파츠 데이터를 묶어 변경하고 스크립트를 고도화할 수 있습니다.

[💡 기초 학습]Life 시스템 구현하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- Life 시스템 구현하기: 01:20:58

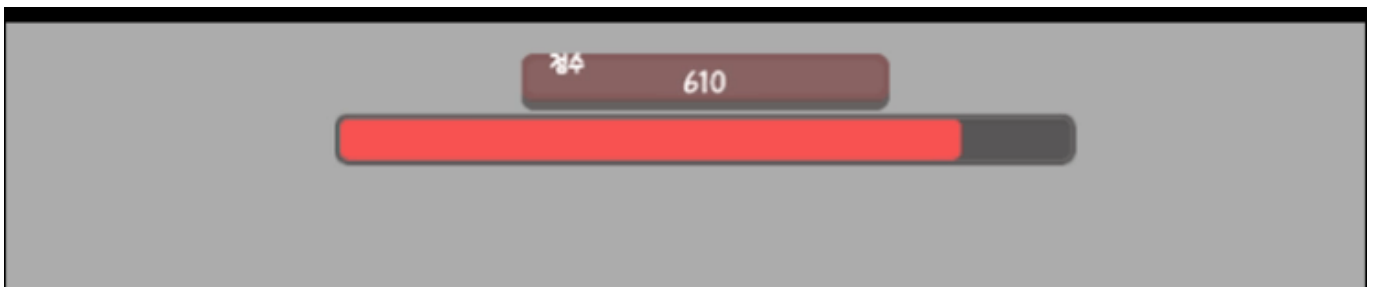
학습 목표

- OnUpdate메서드와 delta시간의 개념을 이해하고, 기기 프레임에 구애받지 않고 시간에 따라 일정하게 체력이 감소하는 로직을 구현할 수 있습니다.
- 캐릭터의 체력이 0 이하로 떨어졌을 때 이를 감지하여 게임 오버 처리를 수행하는 상태 제어 흐름을 학습합니다.
- UI 엔티티에 SliderComponent를 부착하여 시각적인 체력 바를 만들고, 내부 데이터와 화면상의 UI를 실시간으로 동기화하는 방법을 익힙니다.

해당 영상 시청 후 수행해 볼 내용

- Config Logic스크립트에 설정된 decreaseHpValue 변수값을 변경해 테스트를 진행하며 얼마나 체력을 빠르게 감소시킬지 설정해 봅니다.
- 인게임에서 시간이 지남에 따라 UI의 체력 게이지가 부드럽게 감소하는지, 그리고 체력이 0이 되었을 때 결과 창으로 정상 전환되는지 플레이 버튼을 눌러 테스트해 봅니다.

시간에 따른 체력 감소 기능 구현



- 체력 제어 기능은 아이템 획득 및 장애물 충돌 시의 즉각적인 변화뿐만 아니라, 시간이 지남에 따라 주기적으로 체력이 감소하는 기능도 함께 포함하여 제작되었습니다.
- 코드로 체력을 감소시키는 기능은 GameLogic의 AutoDecreaseHp와 AddHP, SetHP메서드의 기능으로 정의되어 있으며, 각 기능은 GameLogic의 OnUpdate메서드 내에서 수행됩니다.
- GameLogic코드의 내용이 필요한 경우 '2.Logic 이해하기' 단원에 첨부된 GameLogic스크립트 내용이 첨부되어 있습니다.

체력 설정 기능

- 체력을 추가, 감소 시키는 기능은 GameLogic의 AddHP메서드를 활용합니다.

GameLogic 내 AddHP 메서드

```
number hp;
```

```
void AddHP(number addValue)
```

```
{
    -- 매개변수로 전달받은 값을 현재 체력에 그대로 더하거나 뺍니다.
    -- AddHP 메서드를 호출할 때 양수 또는 음수를 전달하여 회복과 감소 기능을 모두 유연하게 수
    행할 수 있습니다.

    self.hp += addValue;

    -- 체력값의 변동이 발생되었으므로 UIManagerLogic이 관리하는 슬라이더의 정보를 갱신하도록
    요청합니다.
    _UIManagerLogic.SetHpSliderValue(self.hp)

    -- 체력이 0 이하로 떨어진 경우 (게임 오버 조건 충족 시)
    if self.hp <= 0 then
        -- 게임 종료 기능 수행
    end
}
```

지속적인 체력감소 기능 구현

- 매 프레임 호출되는 OnUpdate 메서드를 활용하면 체력이 서서히 줄어드는 연출을 구현할 수 있습니다.
- 단, 이때 프레임 간 시간 변화량인 delta를 곱해주지 않으면, 기기의 프레임 속도(FPS)에 따라 체력이 비정상적으로 빠르게 증발하는 치명적인 문제가 발생합니다.
- 초당 프레임이 60FPS인 경우 프레임 간 시간 변화량 delta값은 $1\text{Sec}/60\text{FPS} = 0.0167\text{Sec}$ 입니다.
- 만약 1초당 5의 체력을 감소시키고 싶다면, 매 프레임마다 $5 * \text{delta}$ 값을 계산하여 감소시키면 됩니다. 기기가 60프레임으로 동작하든 30프레임으로 동작하든, 1초가 지났을 때 누적 감소량은 정확히 5가 됩니다.

GameLogic의 OnUpdate메서드의 체력감소 기능 부분 예시

```
void OnUpdate(number delta)
```

```
{
    -- Config : 한번 설정된 값이 변경되지 않을 경우 해당 Logic스크립트에 등록되어 관리되고
    있습니다.
    -- 설정한 초당 체력 감소값 * delta 만큼 체력을 반영
    self:AddHP(delta * _Config.decreaseHpValue)
}
```

```
void AddHP(number addValue)
```

```
{
    -- 매개변수로 전달받은 값을 체력값 그대로 추가, 감소시켜주도록 합니다.
    -- AddHP메서드를 활용할때 양수값, 음수값을 전달해 의도대로 기능을 수행할 수 있습니다.
}
```

```
self.hp += addValue;
```

-- 체력값의 변동이 발생되었으므로 UIManagerLogic이 관리하는 슬라이더의 정보를 갱신하도록 요청합니다.

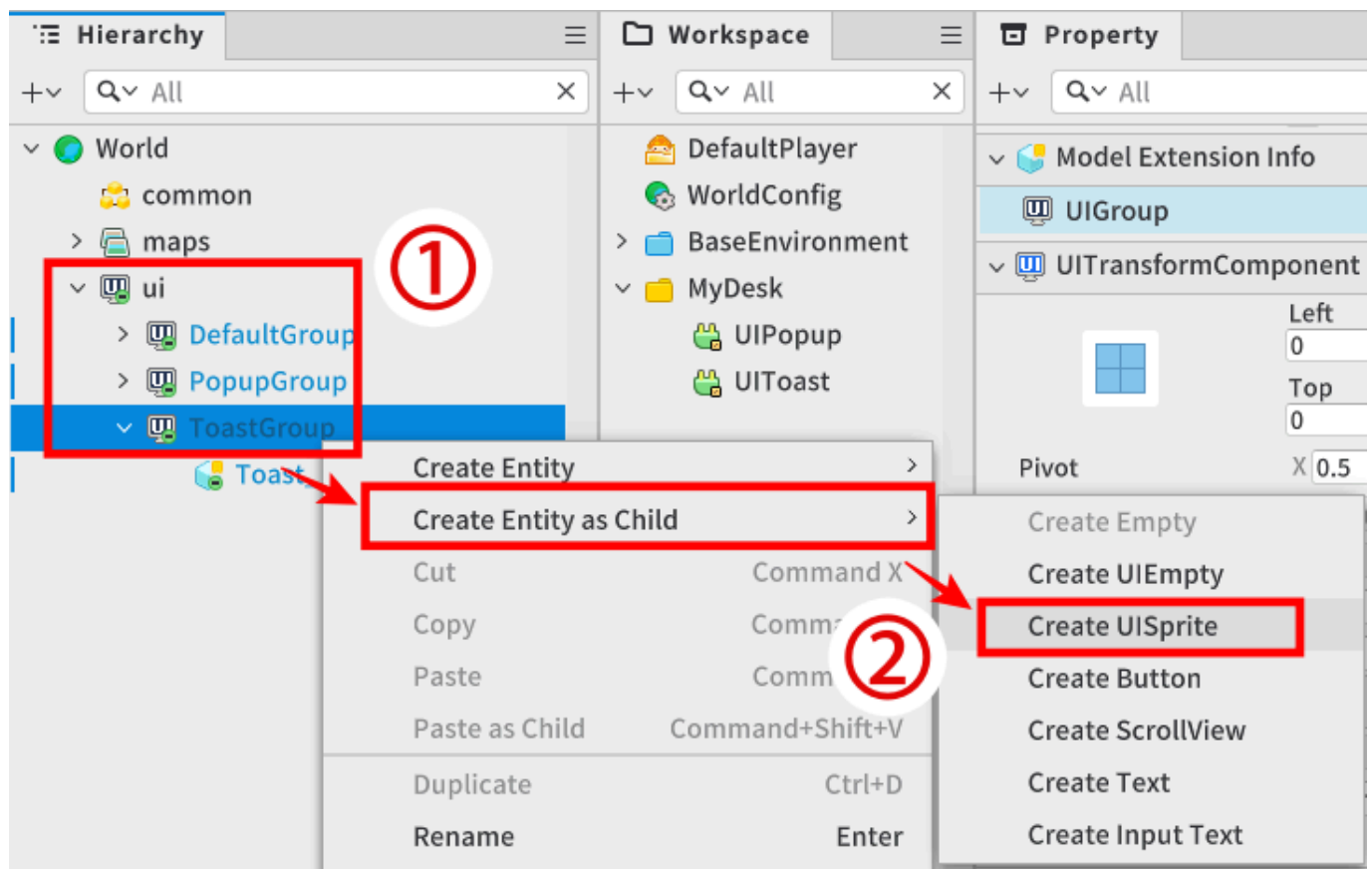
```
_UIManagerLogic.SetHpSliderValue(self.hp)
```

-- 체력값이 0이 되었을 경우(더 이상 게임을 진행할 수 없을 때)

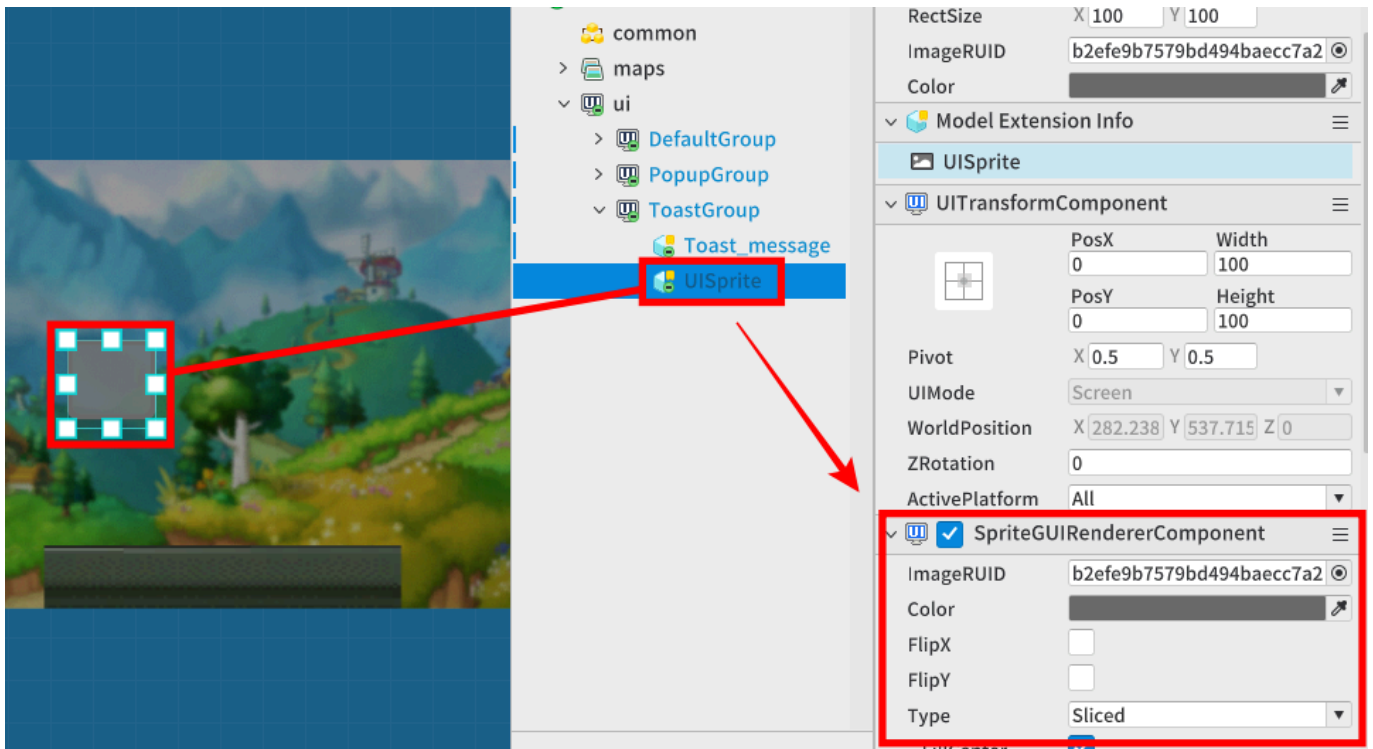
```
if self.hp <= 0 then
    -- 게임 종료 기능 수행
end
}
```

SliderComponent를 활용한 체력 바(UI) 연동

체력 바 UI 제작하기

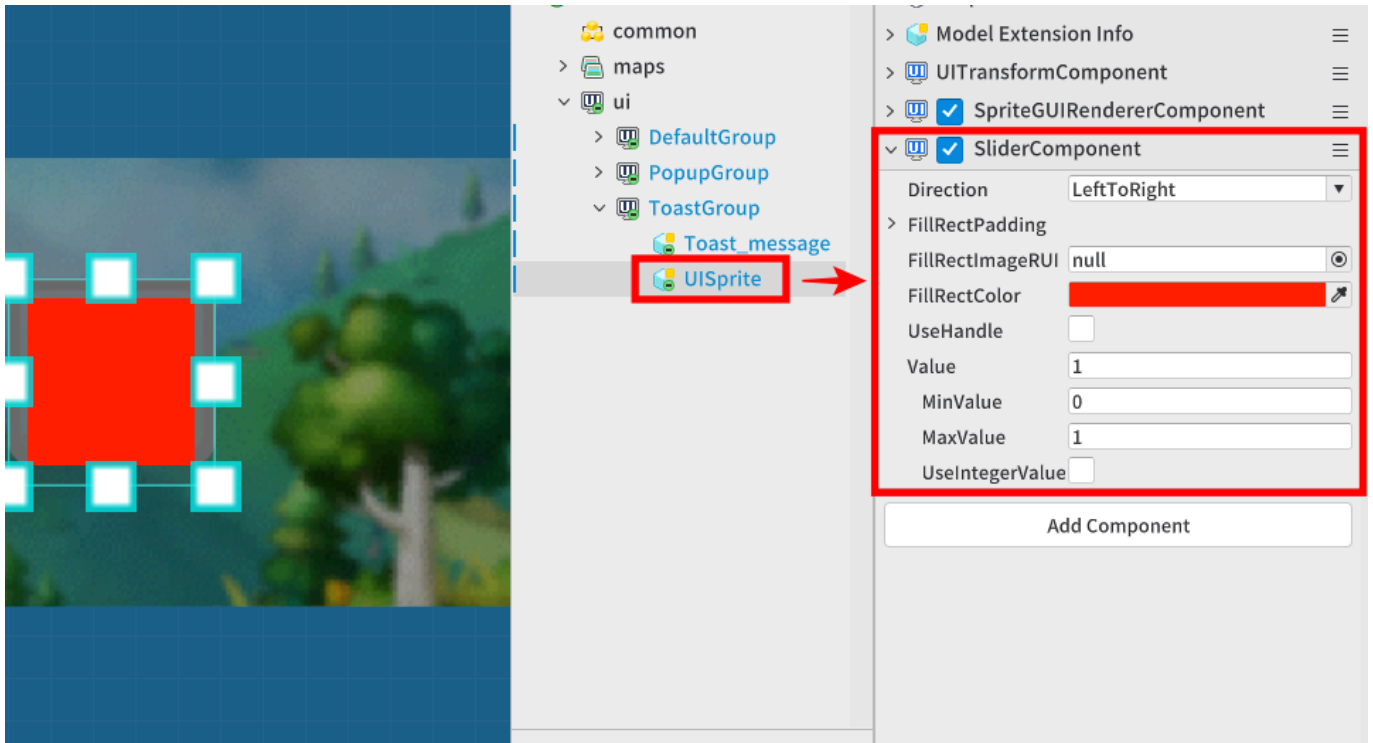


1. Ui 하위 항목 내에서 캔버스 역할을 수행하는 엔티티를 선택합니다.
2. Create Entity as Child에서 Create UISprite를 선택하여 Sprite로 활용될 UI를 생성합니다.
3. 해당 UI는 체력바의 배경이 될 Sprite 이미지입니다.



- 생성된 엔티티에서 SpriteGUIRendererComponent에서 체력바의 배경으로 활용할 이미지 RUID, 색상을 설정합니다.

✅ 생성된 UI 엔티티가 보이지 않는다면, 엔티티에 기본적으로 등록되는 Component인 UITransformComponent의 좌표가 잘못되어 있는지 확인합니다.



- 체력바의 배경 위를 덮을 수 있도록 SliderComponent를 등록합니다.
- FillRectColor 색상을 설정하고 정상적으로 배경 이미지를 덮는지 확인합니다.
- 컴포넌트의 내부 속성인 Value를 변경해 보며 정상적으로 게이지 역할을 수행하는지 체크합니다.
- 샘플 월드의 경우 최대 체력값은 100이라는 설정하에 제작이 진행되어 있습니다.

UIManagerLogic으로 체력 바 UI 제어하기

- 게이지 역할을 수행할 엔티티는 UIManagerLogic에 등록되어 있습니다.
- UIManagerLogic코드의 내용이 필요한 경우 '2.Logic 이해하기' 단원에 첨부된 UIManagerLogic스크립트 내용이 첨부되어 있습니다.
- SliderComponent가 부착된 엔티티의 참조를 연결하고, GameLogic에서 체력이 변할 때마다 이 로직을 호출하여 Slider의 Value 속성에 최신 체력값을 실시간으로 덮어씹웁니다.

UIManagerLogic의 SetHpSliderValue메서드 구현 예시

```
SliderComponent hpSlider;
```

```
void SetHpSliderValue(number value)
```

```
{
```

```
    -- 설정될 슬라이더값을 math.clamp메서드를 활용해 0 미만, 100 초과 값 넘지 않도록 설정합니다.
```

```
    self.hpSlider.Value = math.clamp(value, 0, 100)
```

```
}
```

최소 구현 기준

- 스크립트 내부의 체력 데이터(hp)변화가 UIManagerLogic을 통해 전달되어, 화면 상의 SliderComponent 게이지 UI에 실시간으로 동기화되어야 합니다.
- 체력이 0 이하로 떨어지는 즉시 이를 감지하여 더 이상 체력이 마이너스로 내려가지 않도록 제한하고, 게임 오버 처리가 정상적으로 호출되어야 합니다.

응용 및 확장 방향

- 체력이 일정 수치(예:20%) 이하의 위험 상태로 떨어졌을 때, SliderComponent의 게이지 색상을 붉은색으로 깜빡이게 변경하여 플레이어에게 시각적인 긴장감과 경고 피드백을 제공해 볼 수 있습니다.

[💡 기초 학습]UI 구성하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- UI 구성하기: 01:31:40

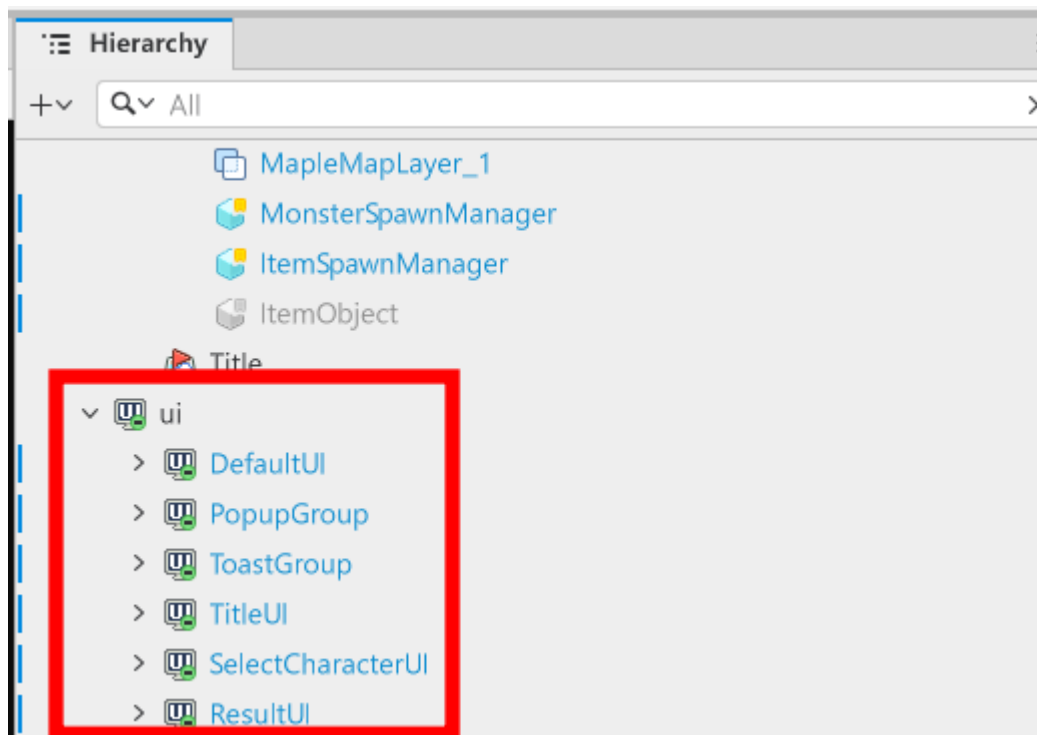
학습 목표

- 개별 UI 요소들을 하나씩 제어하는 대신, 하나의 UI 엔티티 단위로 묶어 게임 상태(타이틀, 인게임, 결과)에 따라 화면을 효율적으로 전환하는 구조를 이해합니다.
- 게임 종료 시 호출되는 결과 화면(Result Popup)을 제작하고, GameLogic과 UIManagerLogic을 연동하여 현재 점수와 최고 점수를 UI 텍스트에 동기화하는 방법을 학습합니다.

해당 영상 시청 후 수행해 볼 내용

- 하이어라키 창에서 타이틀, 인게임, 결과 화면에 필요한 UI 요소들을 각각의 부모 엔티티(UI Group) 하위로 묶고, 최상위 엔티티의 Enable 속성을 켜고 끄며 화면이 정상적으로 전환되는지 테스트해 봅니다.
- 인게임에서 일부러 장애물에 부딪혀 게임을 종료시킨 뒤, 결과 팝업 창에 내가 방금 획득한 점수와 기존 최고 점수가 텍스트 UI에 올바르게 출력되는지 직접 확인해 봅니다.

UI Group의 이해

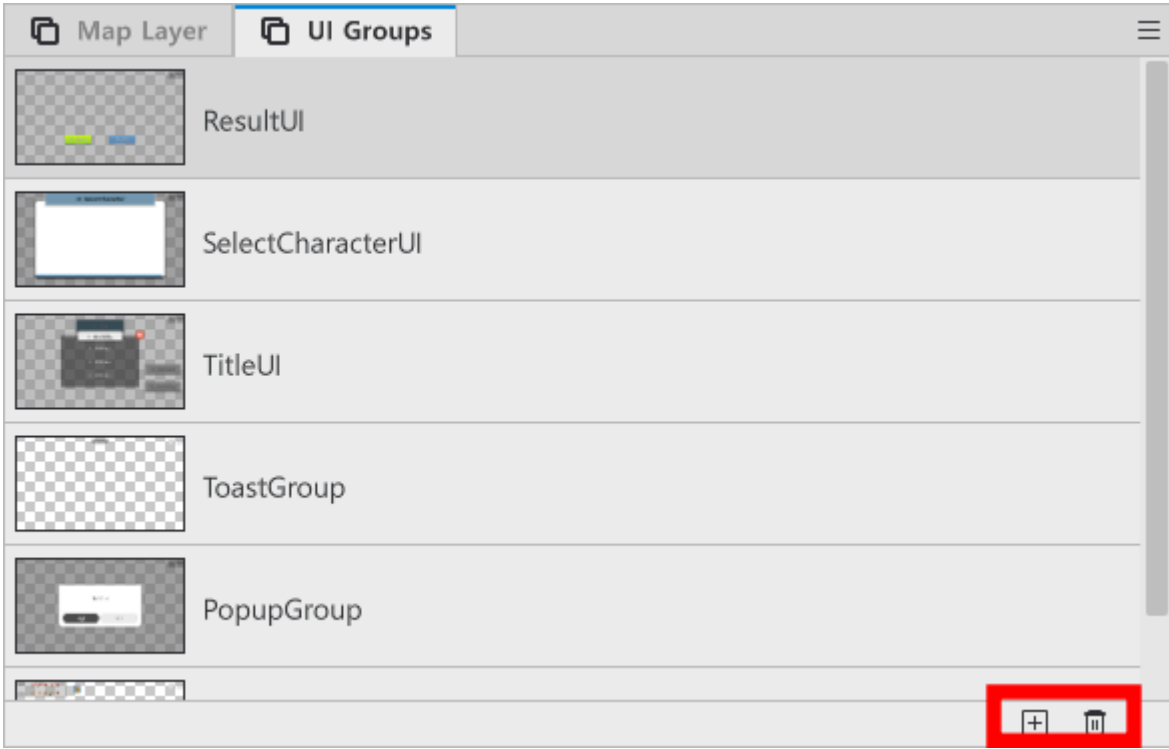


- 하이어라키 창의 아래를 보면, ui라는 이름으로 구성된 엔티티들을 확인할 수 있습니다.
- 해당 엔티티는 ui를 출력하기 위해 구성된 ui group입니다.
- ui group을 수정하여 출력해야 하는 ui를 변경하거나 제작할 수 있습니다.

UI Group 생성하기



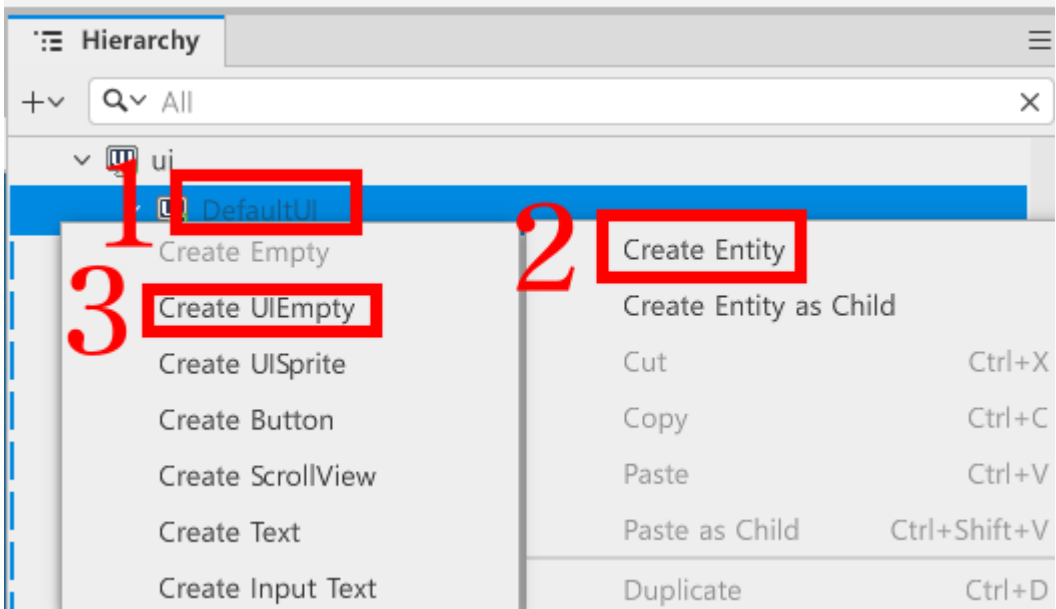
- 에디터 상단의 UI 버튼을 클릭하여 UI 편집상태로 변경합니다.



- 에디터 창 하단의 UI Groups에서 + 버튼을 클릭해 새로운 UI Group를 추가할 수 있습니다.
- 삭제해야 하는 UI Groups가 있을 경우, 삭제하려는 UI Group을 클릭한 뒤, 휴지통 버튼을 클릭해 삭제할 수 있습니다.

빈 UI 엔티티 제작하기

- UI에서도 기존의 엔티티를 제작하듯이 엔티티 제작이 가능합니다.
- 차이점은 빈 엔티티를 생성할 때, Create Empty가 아닌 **Create UICollection**을 사용한다는 차이점이 있습니다.

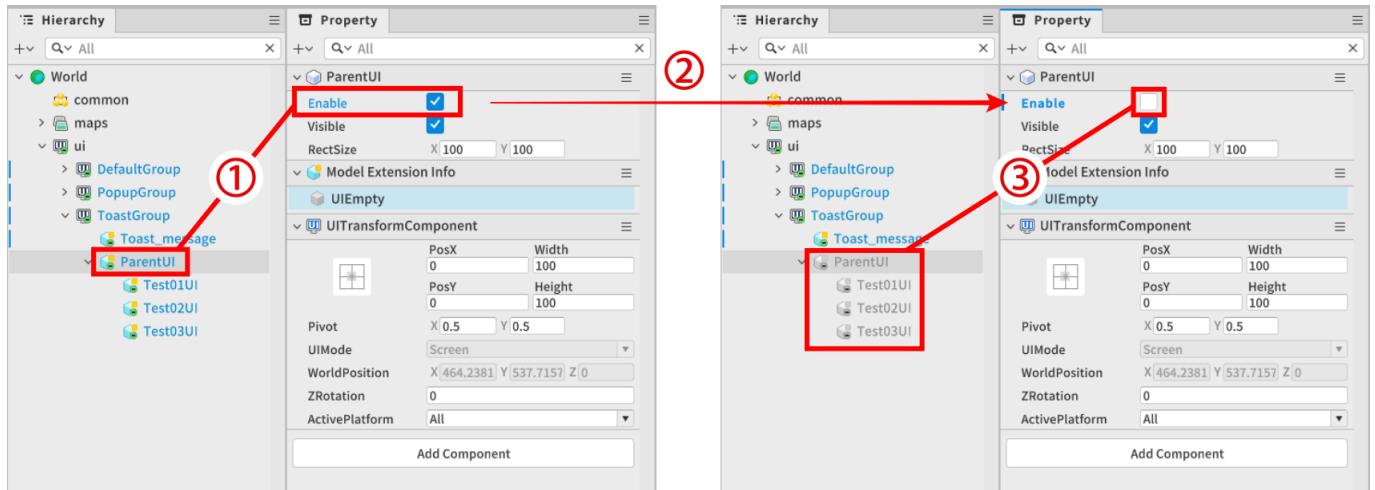


- 하이어라키 창에서 엔티티를 추가하려는 ui group을 우클릭합니다.
- Create Entity를 클릭합니다.
- Create UICollection을 클릭하여 빈 UI 엔티티를 생성할 수 있습니다.

UI 엔티티에 Component 부여하기

- UI 엔티티 역시 일반 엔티티처럼 Component를 부여하여 추가적인 기능들을 수행할 수 있습니다.
- Component를 부여하는 방법은 일반 엔티티에 Add Component를 하는 방식과 동일합니다.
- 해당 단원에서는 자주 사용되는 Component에 대해 설명드리겠습니다.
 - **SpriteGUIRendererComponent**: UI에서 이미지를 출력할 때 사용되는 Component입니다. UI에서 이미지를 출력할 땐 SpriteRendererComponent보다 해당 Component를 더 권장드립니다.
 - **TextGUIRendererComponent**: UI에서 텍스트를 출력할 때 사용되는 Component입니다.
 - **ButtonComponent**: UI가 버튼의 역할을 수행해야 하는 경우 사용되는 Component입니다.

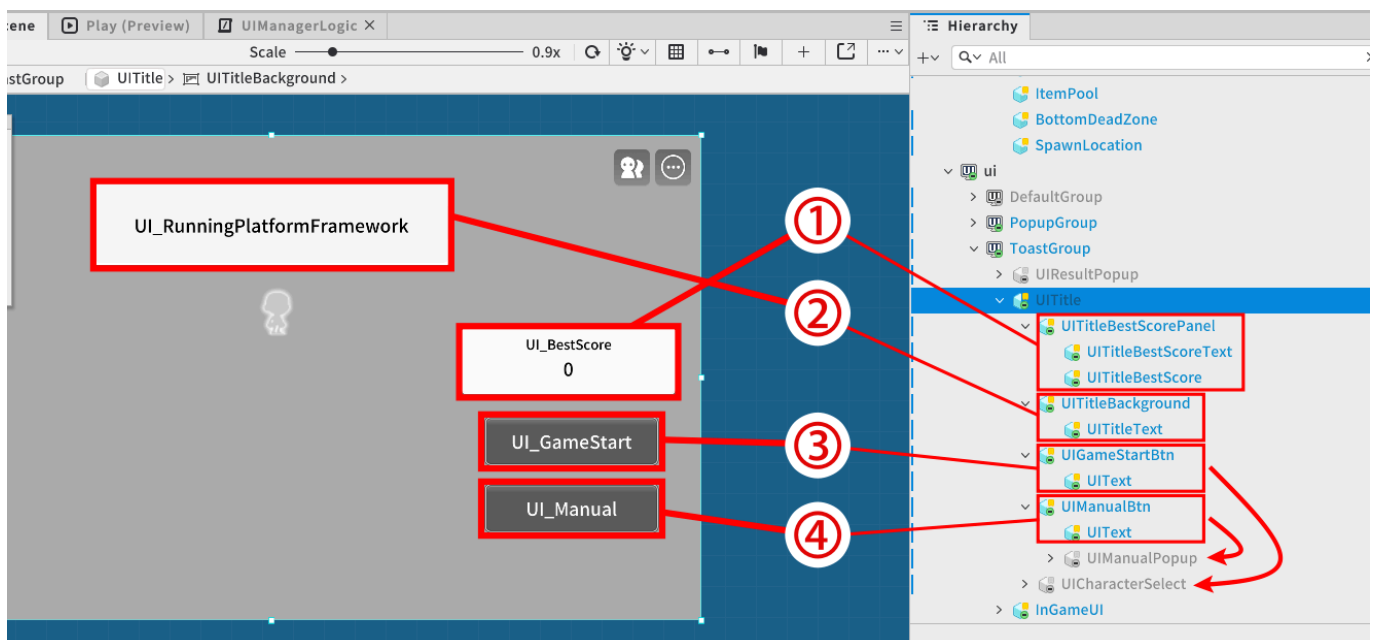
UI Groups를 활용한 그룹단위 제어



1. 하나의 그룹의 가장 상위의 엔티티요소의 Enable 값이 true 상태로 체크가 되어있을땐, 자식 엔티티는 개별적으로 Enable을 true 또는 false 상태로 켜두거나 꺼둘수 있습니다.
2. 가장 상위의 엔티티요소의 Enable 상태를 false로 체크를 해제합니다.
3. 부모 엔티티가 비활성화되면서 자동으로 자식 엔티티들 또한 비활성화되는것을 확인하실 수 있습니다.

- 엔티티 특성상 부모 엔티티를 비활성화 할 경우 자동으로 엔티티에 속한 자식 엔티티 또한 비활성화 됩니다.
- 이러한 특성을 활용하여 게임의 상황별로 화면에 표기되어야 할 UI들을 각 요소 하나씩 제어하지 않고, 그룹 단위의 최상위 엔티티만 활성화, 비활성화하여 손쉽게 제어할 수 있습니다.

Title UI Group 제작하기



항 목	그룹	설명
1	UITitleBestScorePanel	월드 시작 시 플레이어의 최고 점수값을 나타내는 영역입니다. UITitleBestScoreText - 패널 내 상단 '최고 점수'를 나타내는 텍스트입니다. UITitleBestScore - 실제 최고 점수값만 나타내는 용도의 텍스트입니다.
2	UITitle	월드 시작 시 게임의 제목, 타이틀을 나타내는 영역입니다. UITitleText - 게임의 제목을 출력하는 텍스트입니다.
3	UIGameStartBtn	플레이어의 게임시작을 원할 경우 해당 버튼을 통해 게임을 진행할 수 있습니다. 해당 버튼은 캐릭터 선택창 팝업(UICharacterSelect)을 활성화 되도록 연결되어 있습니다. UIText - '게임시작'을 나타내는 텍스트입니다.
4	UIManualBtn	플레이어가 게임 조작방식을 학습하고 싶을때 해당 버튼을 통해 조작방식을 확인할 수 있습니다. 해당 버튼은 조작법 팝업(UIManualPopup)을 활성화 되도록 연결되어 있습니다. UIText - '조작법'을 나타내는 텍스트입니다.

- TitleUI 그룹의 경우 월드 시작 시 플레이어가 바로 확인할 수 있어야하므로 기본적인 Enable을 true 상태로 설정해 둡니다.
- TitleUI 그룹을 직접 활성화 하는 시점은 본격적인 게임이 종료되고, 결과창 화면에서 메인메뉴로 다시 전환되는 상태입니다.
- TitleUI 그룹을 직접 비활성화 하는 시점은 플레이어가 캐릭터를 선택하고 본격적인 게임상태로 진입할 경우입니다.

캐릭터 선택 창에서 캐릭터 Slot(UICharacterSelect)에서 캐릭터 선택 버튼을 통해 게임이 시작되는 상황

UICharacterInfoComponent의 CloseSelectPopup메서드

```
void CloseSelectPopup()
{
    -- 타이틀 UI 그룹의 최상위 엔티티를 저장합니다.
    local TitlePanel =
    _EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")

    -- 최상위 엔티티를 비활성화 합니다.
    TitlePanel.Enable = false

    -- GameLogic에게 게임시작을 호출을 요청합니다.
    _GameLogic:GameStart(self.characterId)
}
```

게임 결과창에서 플레이어가 '메인메뉴'버튼을 통해 타이틀 화면으로 되돌아가는 상황
UIManagerLogic의 ReturnToMenu메서드

```

Entity mainTitlePanel
Entity resultPopup
Entity resultToMenuButton

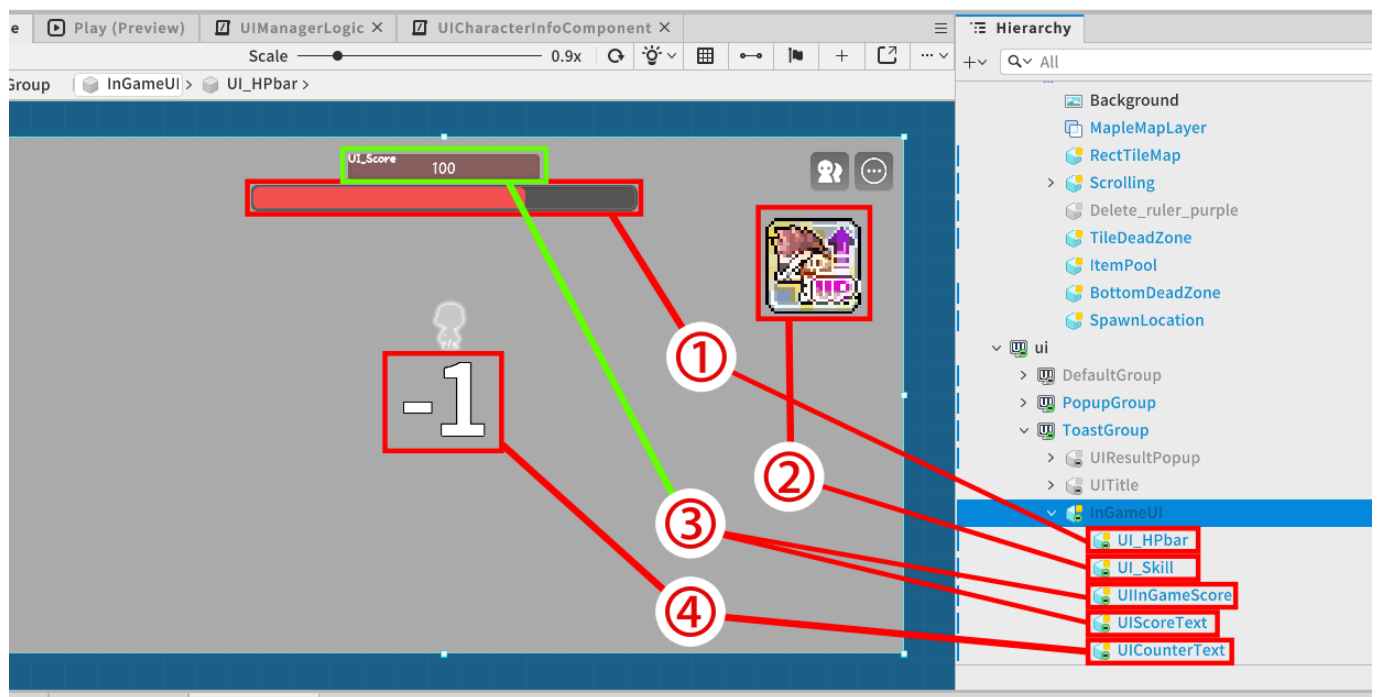
void OnBeginPlay()
{
    -- 월드 실행 시, 결과 창 팝업의 '메인메뉴'버튼에 이벤트 등록합니다.

    self.resultToMenuButton.ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- 결과창 팝업을 비활성화 합니다.
    self.resultPopup.Enable = false
    -- TitleUI 최상위 엔티티를 활성화 합니다.
    self.mainTitlePanel.Enable = true
}

```

InGame UI Group 제작하기



항목	그룹	설명
1	UI_HPbar	플레이어가 캐릭터의 현재 체력 상태를 확인할 수 있는 UI 입니다.
2	UI_Skill	캐릭터가 사용할 수 있는 스킬이 무엇인지 확인하고, 스킬이 사용되었는지 확인할 수 있는 UI 입니다.

항목	그룹	설명
3	UI_Score	UIInGameScore - 게임이 진행됨에 따라 플레이어가 습득한 실제 점수값을 나타내는 UI 입니다. UIScoreText - 해당 패널 좌측 상단의 '점수' 텍스트를 나타내는 UI 입니다.
4	UICounterText	본격적인 게임이 시작되기 직전, 플레이어에게 N초 뒤 스크롤링이 시작됨을 알립니다.

- InGameUI 그룹을 직접 활성화 하는 시점은 플레이어가 캐릭터 선택을 완료하고 본격적인 게임이 시작되는 상황입니다.
- InGameUI 그룹을 직접 비활성화 하는 시점은 캐릭터가 사망한 뒤, 결과창에서 메인메뉴로 넘어가는 상황입니다.

플레이어가 캐릭터 선택을 마쳤을때, 인게임에 필요한 UI들을 호출하는 상황
UIManagerLogic의 ShowPlayerInfo메서드

```
SliderComponent hpSlider
Entity inGameCurrentScoreText
Entity inGameScoreText
SpriteGUIRendererComponent skillRenderer

void ShowPlayerInfo(boolean active)
{
    -- 체력 게이지를 나타내는 엔티티를 전달받는 active 상태로 전환합니다.
    self.hpSlider.Entity.Enable = active
    -- 플레이어의 점수값을 나타내는 엔티티를 전달받는 active 상태로 전환합니다.
    self.inGameCurrentScoreText.Entity.Enable = active
    -- '점수' 텍스트를 표기하는 UI요소를 active 상태로 전환합니다.
    self.inGameScoreText.Entity.Enable = active
    -- 캐릭터의 스킬 정보를 나타내는 엔티티를 전달받는 active 상태로 전환합니다.
    self.skillRenderer.Entity.Enable = active
}
```

GameLogic의 GameStart메서드

```
void GameStart()
{
    _UIManagerLogic:ShowPlayerInfo(true)

    -- 인게임 시작전 필요한 초기화작업을 수행합니다.
}
```

게임 결과창에서 플레이어가 '메인메뉴' 버튼을 통해 타이틀 화면으로 되돌아가는 상황
UIManagerLogic의 ReturnToMenu메서드

```
Entity mainTitlePanel

void OnBeginPlay()
```

```

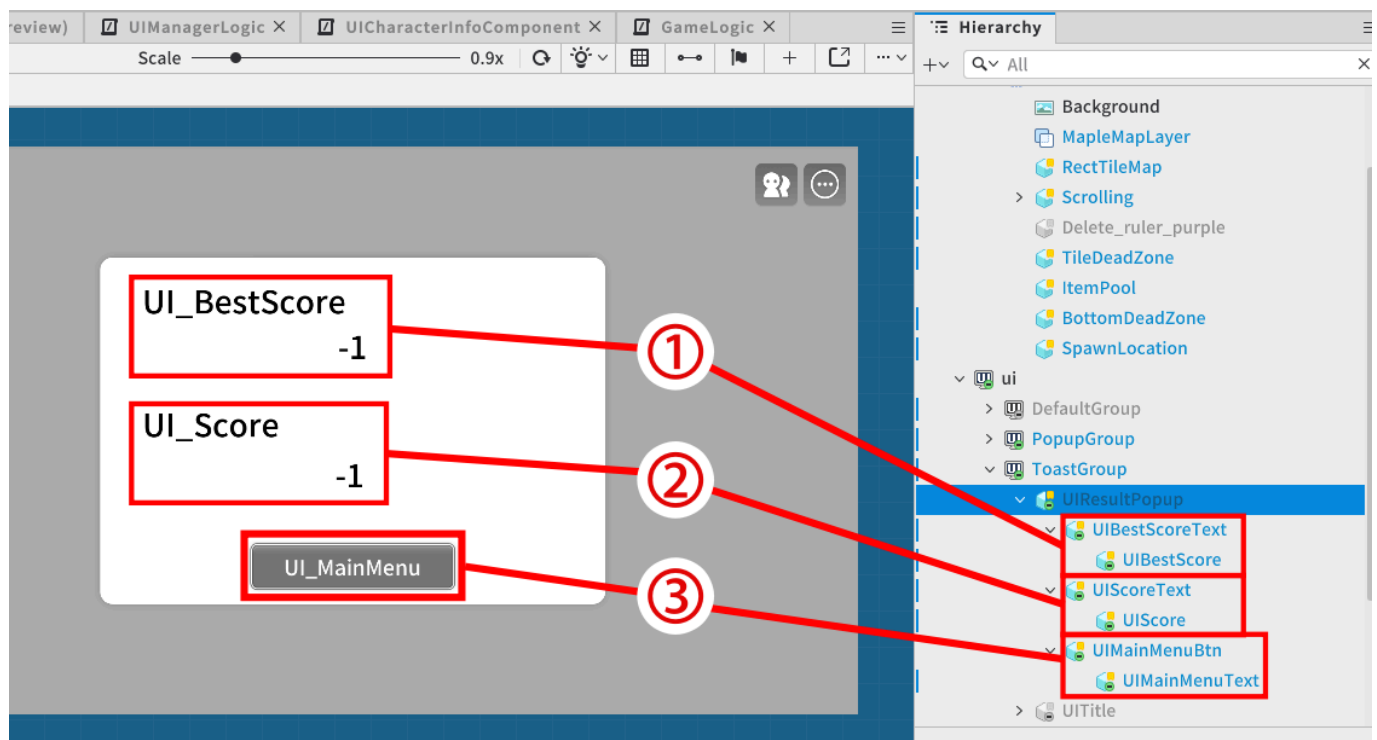
{
    -- 월드 실행 시, 결과 창 팝업의 '메인메뉴'버튼에 이벤트 등록합니다.

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- 인게임에 필요한 UI 요소를 전부 비활성화 합니다.
    self:ShowPlayerInfo(false)
    -- TitleUI 최상위 엔티티를 활성화 합니다.
    self.mainTitlePanel.Enable = true
}

```

Result Popup 제작하기



항목	그룹	설명
1	UIBestScoreText	결과 창 내에 '최고 점수' 텍스트를 나타내는 UI 입니다. UIBestScore : 실제 플레이어의 기존 최고 점숫값을 표기하는 텍스트 UI 입니다.
2	UIScoreText	결과 창 내에 '점수' 텍스트를 나타내는 UI 입니다. UIScore : 실제 플레이어의 현재 종료된 시점의 점수를 표기하는 텍스트 UI 입니다.
3	UIMainMenuBtn	결과 창 팝업을 닫고 Title 상태로 되돌아가는 기능을 수행하는 버튼 UI 입니다. UIMainMenuText : 버튼 내에 '메인 메뉴' 텍스트를 나타내는 UI 입니다.

- ResultPopup 그룹을 직접 활성화 하는 시점은 인게임으로 본격적인 게임이 진행되고 이후 캐릭터가 더 이상 플레이가 불가능한 상황입니다.
- ResultPopup 그룹을 직접 비활성화 하는 시점은 결과창 내에서 '메인메뉴' 버튼을 선택해서 Title 상태로 전환되는 상황입니다.

인게임이 종료되고 결과창 팝업을 띄우는 상황
UIManagerLogic의 ShowResultPopup메서드

```
-- 결과창 내 등록된 실제 이전 최고 점숫값을 등록하는 Text 엔티티
Entity resultBestScoreText
-- 결과창 내 등록된 실제 현재 점숫값을 등록하는 Text 엔티티
Entity currentScoreText

void ShowResultPopup(string bestScore, string currentScore)
{
    -- 최고 점숫값 텍스트를 매개변수로 전달받은 텍스트로 변경합니다.
    self.resultBestScoreText.TextComponent.Text = bestScore

    -- 현재 점숫값 텍스트를 매개변수로 전달받은 텍스트로 변경합니다.
    self.currentScoreText.TextComponent.Text = currentScore

    -- 결과 창을 활성화 합니다.
    self.resultPopup.Enable = true;
}
```

캐릭터 사망 시점에서 호출되는 GameLogic의 EndMainGame메서드

```
number bestScore
number currentScore

void EndMainGame()
{
    -- 게임 종료시 필요한 기능 수행

    _UIManagerLogic:ShowResultPopup(tostring(self.bestScore),tostring(self.currentScore))
}
```

게임 결과창에서 플레이어가 '메인메뉴'버튼을 통해 타이틀 화면으로 되돌아가는 상황
UIManagerLogic의 ReturnToMenu메서드

```
Entity resultPopup
Entity mainTitlePanel

void OnBeginPlay()
{
    -- 월드 실행 시, 결과 창 팝업의 '메인메뉴'버튼에 이벤트 등록합니다.
```

```

self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- 결과 창 팝업을 비활성화 합니다.
    self.resultPopup.Enable = false
    -- TitleUI 최상위 엔티티를 활성화 합니다.
    self.mainTitlePanel.Enable = true
}

```

최소 구현 기준

- TitleUI, InGameUI, ResultPopup 세 가지 상태로 전환할 수 있는 각 버튼 UI와 게임시작, 게임종료, 메인메뉴로 돌아가는 기능이 정상적으로 동작해야 합니다.
- GameLogic에서 계산된 currentScore와 bestScore값이 각 UI 텍스트 컴포넌트에 정확하게 전달되어야 합니다.

응용 및 확장 방향

- ResultPopup이 나타낼 때 단순히 나타나는 것이 아니라, 크기가 커지거나 투명도가 변하며 서서히 나타나는 효과를 추가합니다.
- 버튼을 클릭할 때나 결과 창이 뜰 때 효과음을 재생하여 조작감을 높입니다.

[📖심화 학습]장애물 오브젝트 구현하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 장애물 오브젝트 구현하기: 01:46:25

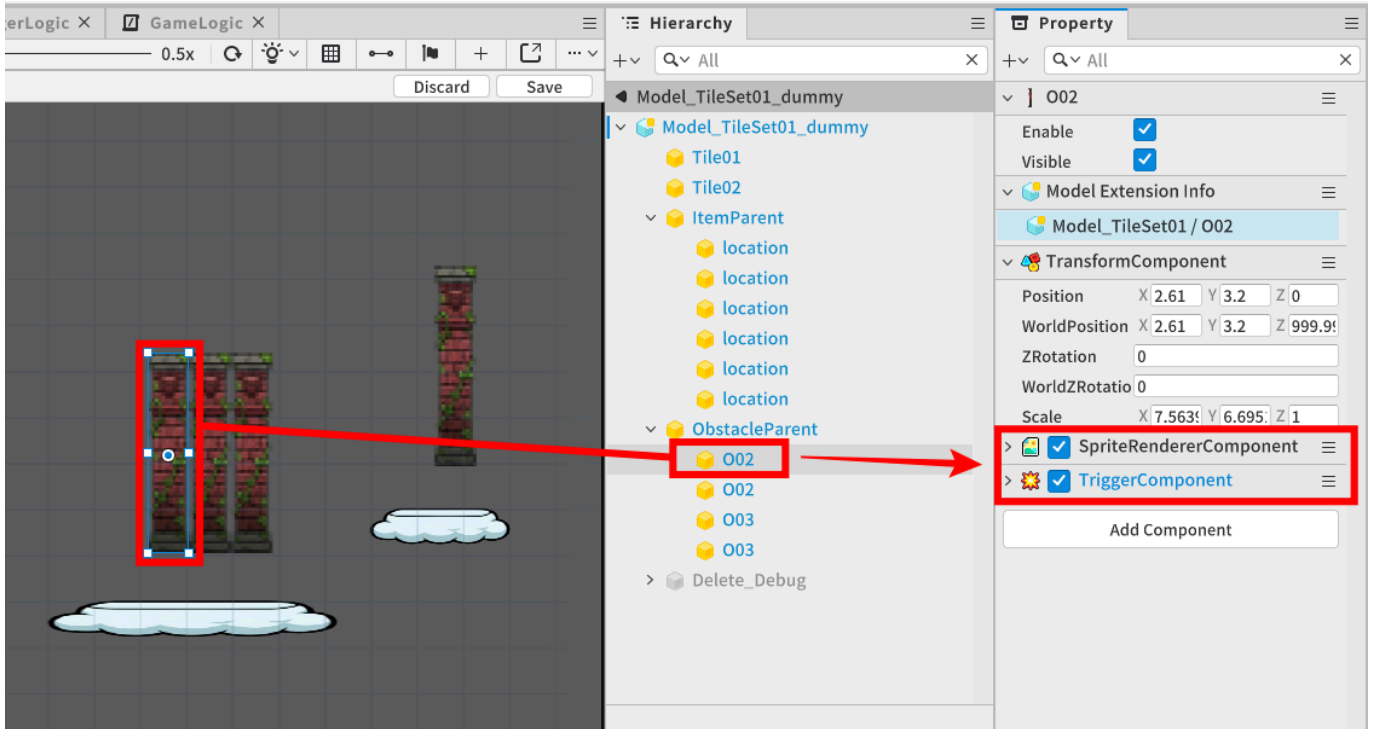
학습 목표

- 타일셋 내에 장애물을 미리 배치하고, 게임 진행 시 확률적으로 활성화하며 리소스를 효율적으로 관리하는 방식을 이해합니다.
- 장애물 충돌 감지 시 캐릭터의 체력을 감소시키고, 일정 시간 동안 캐릭터를 무적 상태로 만드는 기능을 구현할 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

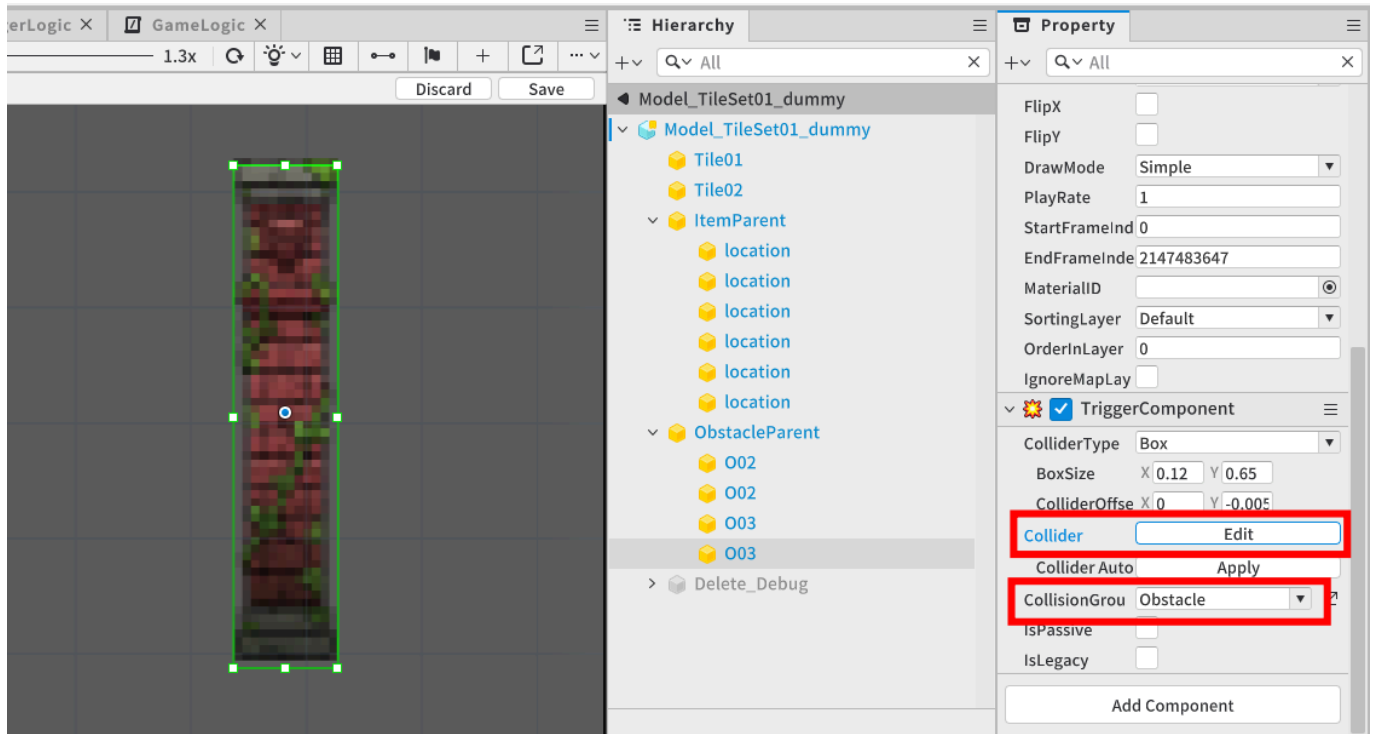
- 장애물 엔티티를 타일셋의 자식 엔티티로 배치하여, 타일 이동 시 별도의 코드 없이 장애물이 정상적으로 함께 이동하는지 확인해 봅니다.
- 장애물 충돌 시 캐릭터의 체력이 줄어드는지 확인하고, 설정한 무적 시간 동안은 추가 데미지를 입지 않는지 테스트합니다.

장애물 제작하기



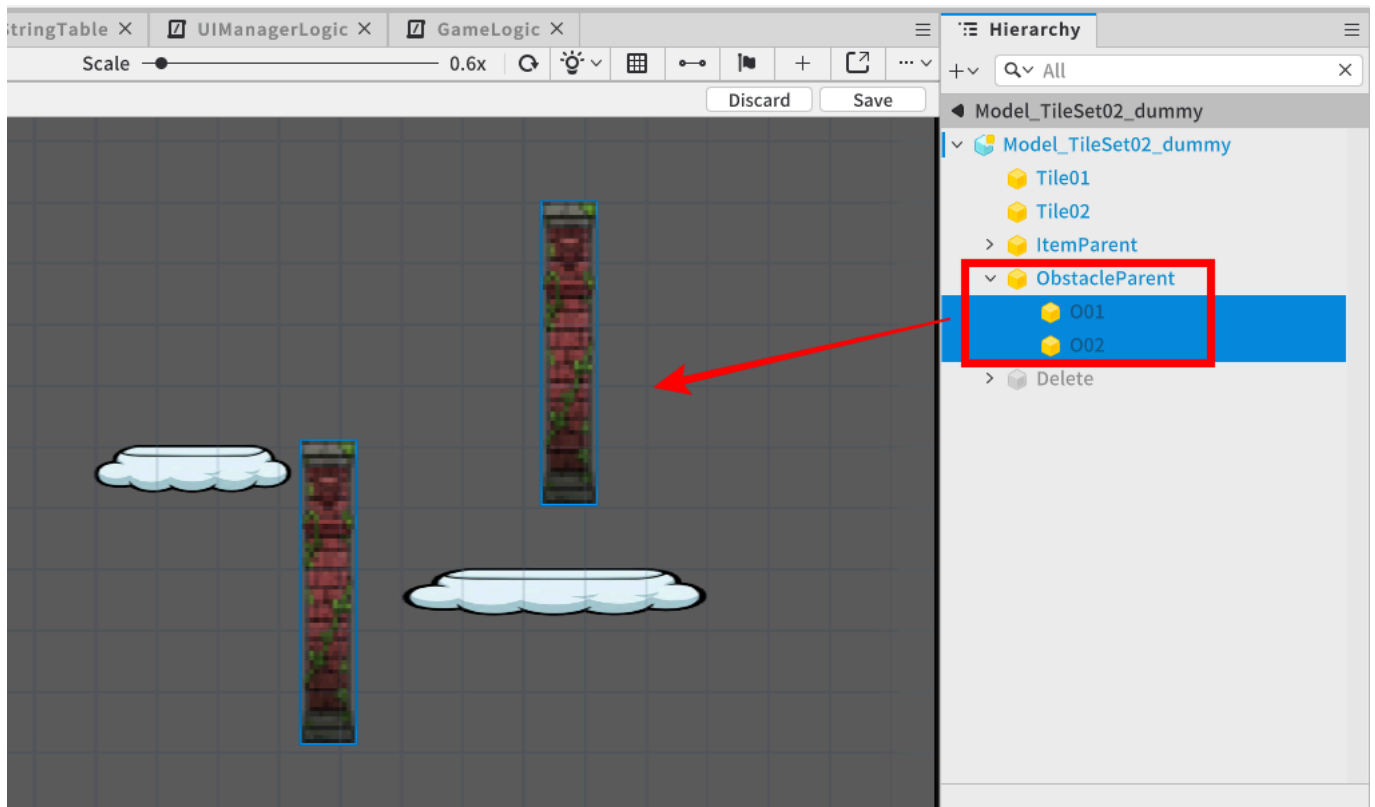
- 장애물 엔티티에 필요한 컴포넌트는 SpriteRendererComponent, TriggerComponent 입니다.
- SpriteComponent는 장애물의 외형을 나타내기 위함입니다.
- TriggerComponent는 캐릭터와의 충돌을 감지하기 위한 용도이며, 캐릭터가 가로막혀 밀려나는 등 실제 물리적인 충돌을 일으키지 않도록 설정하기 위함입니다.

TriggerComponent 설정하기



- 장애물 엔티티에 등록된 TriggerComponent의 Collider범위를 설정합니다.
- CollisionGroup에서 새로운 충돌 그룹 'Obstacle'을 추가하고, 'Player'와의 충돌만 감지되도록 매트릭스를 설정합니다.
- Collider 범위를 넓히거나, 좁혀 게임의 난이도를 조절할 수 있습니다.

장애물 배치 방식



- 장애물을 코드로 위치를 제어하기에는 어려움이 있으므로 장애물 엔티티를 미리 배치해두는 방식으로 제작을 진행합니다.
- 플레이어가 장애물을 회피할 수 있는지 테스트를 진행해 보며 배치 및 수정 작업을 진행합니다.

- 장애물 엔티티 관리 방식이 원활하도록, 장애물들의 부모 엔티티를 준비합니다.

무작위로 장애물 배치하기

- 만약 준비한 장애물을 타일마다 항상 배치한다면 게임이 단조로워질 수 있으므로, 난수(Random) 기능을 활용해 장애물이 불규칙한 패턴으로 등장하도록 구현합니다.
- 미리 배치된 장애물을 무작위로 배치하는 기능은 바닥 타일셋에 등록된 TileSet 컴포넌트 스크립트에서 타일셋이 활성화될 때 진행됩니다.
- 무작위 장애물 배치 기능은 다음 코드와 같이 처리할 수 있습니다.

TileSet 스크립트 내 SpawnObstacle메서드

```
-- 장애물 엔티티들의 부모 엔티티입니다.  
Entity obstacleParent
```

```
void SpawnObstacle()  
{  
    -- 타일셋 내에 사전에 미리 설치한 장애물 오브젝트 수를 확인합니다.  
    -- obstacle들을 보관한 부모 엔티티의 자식(장애물엔티티) 개수를 확인합니다.  
    local spawnLocationCount = self.obstacleParent.Children.Count  
  
    -- obstacleParent 오브젝트의 자식 오브젝트를 순회하며  
    for index , child in pairs(self.obstacleParent.Children) do  
  
        -- 0부터 1까지 무작위 숫자를 생성합니다.(구현 의도 : 0 일 경우 장애물 배치 0 , 1  
        일 경우 장애물 배치 X 로 50% 확률)  
        local randomNumber = math.random(0,1)  
  
        -- /분기1/ 만약 숫자가 0 이라면  
        if randomNumber == 0 then  
            -- 장애물 오브젝트를 활성화합니다.  
            child.Enable = true  
            -- 스테이지 번호에 맞는 리소스 이미지로 덮어 씹웁니다.  
            child.SpriteRendererComponent.SpriteRUID =  
            _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].ObstacleRUID  
  
            -- /분기2/ 만약 숫자가 0이 아니라면  
        else  
            -- 장애물 오브젝트를 비활성화합니다.  
            child.Enable = false  
        end  
    end  
}
```

장애물 피격처리



- 캐릭터의 TriggerComponent와 장애물의 TriggerComponent가 정상적으로 등록되어 있다면, 상호 간 Trigger 이벤트를 활용할 수 있습니다.
- 캐릭터의 기능 및 동작을 직접 제어하기 위해 제작된 CustomPlayer 스크립트 내에서 HandleTriggerEnterEvent 이벤트 메서드를 활용합니다.
- CustomPlayer코드의 내용이 필요한 경우 '6.플레이어 캐릭터 구현하기' 단원에 첨부된 CustomPlayer스크립트 내용이 첨부되어 있습니다.
- 충돌 처리 기능은 다음 코드와 같이 처리할 수 있습니다.

CustomPlayer의 이벤트 핸들러를 활용한 충돌처리

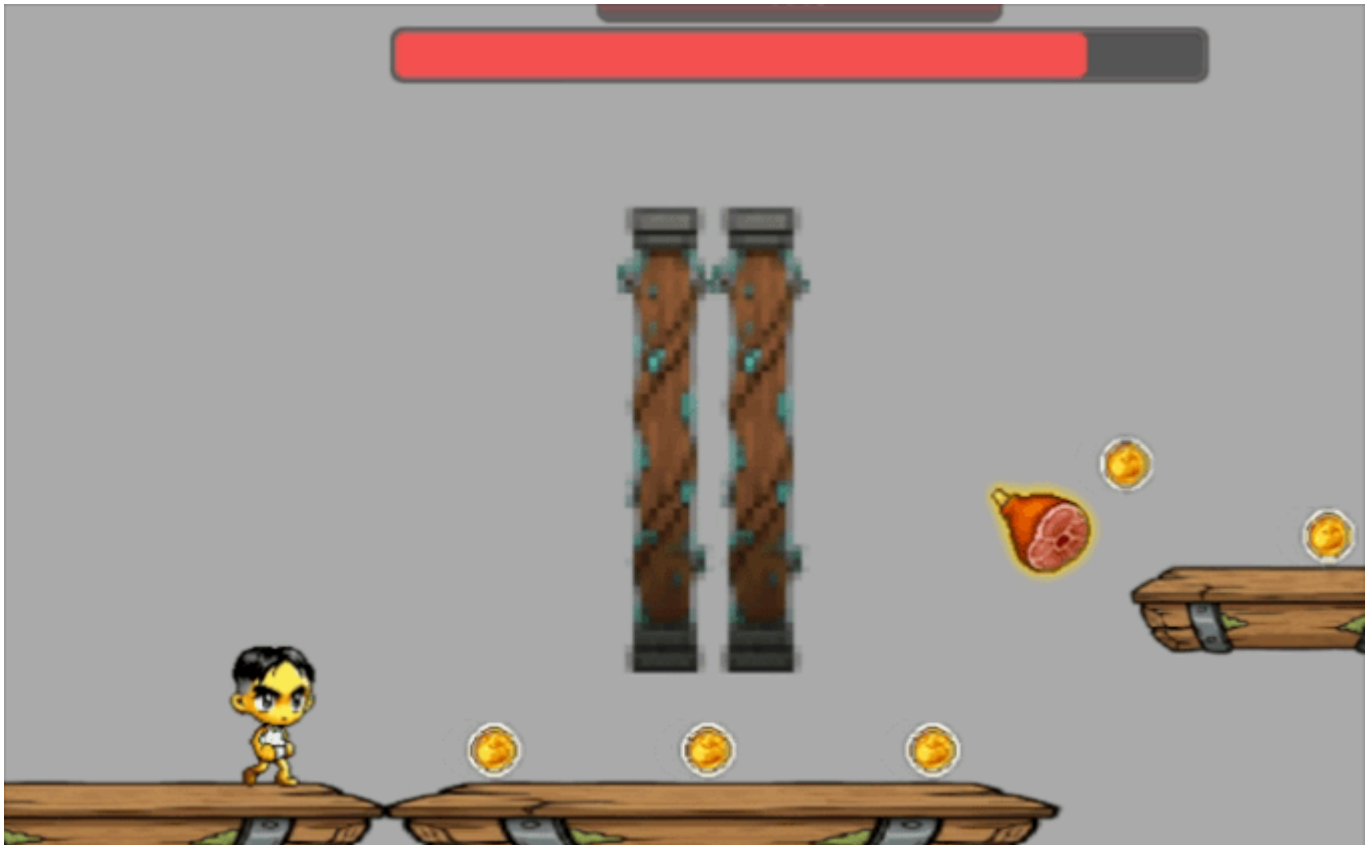
```
HandleTriggerEnterEvent(TriggerEnterEvent event)
{
    -- 충돌된 대상의 엔티티 정보를 저장합니다.
    local TriggerBodyEntity = event.TriggerBodyEntity

    -- 대상 엔티티의 CollisionGroup에 설정된 충돌체 그룹 ID 번호를 추출합니다.
    local TriggerGroup =
    tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)

    -- 해당 충돌체의 그룹명이 "Obstacle"과 동일하다면
    if _Config.triggerGroup[TriggerGroup] == "Obstacle" then
        -- 충돌 판정 처리를 수행합니다.
        self:ContactObstacle()
    end
}

void ContactObstacle()
{
    -- GameLogic에게 Config에 설정된 장애물 피격 데미지값 만큼 체력을 감소시키도록 합니다.
    _GameLogic:AddHP(_Config.obstacleDamage)
}
```

캐릭터 무적 기능 추가하기



- 일정 무적 시간이 주어지지 않으면서 장애물의 피격처리를 HandleTriggerStayEvent 이벤트 메서드로 제작할 경우, 매 프레임마다 데미지를 입게 되어 순식간에 체력이 0이 되는 현상이 발생될 수 있습니다.
- 장애물에 부딪힌 직후 짧은 시간 동안 추가 피해를 막아줌으로써, 플레이어가 상황을 인지하고 자세를 재정비할 수 있는 여유 시간을 제공합니다.
- 일정시간 피격 무시 기능은 다음 코드와 같이 처리할 수 있습니다.

CustomPlayer의 invincibleTimer 활용

-- 일시 무적 타이머 변수를 선언합니다.

```
number invincibleTimer
```

-- 피격 상태임을 나타내는 변수를 선언합니다.

```
boolean isHit
```

```
void ContactObstacle()
```

```
{
```

```
    -- 만약 일시 무적 타이머 변수의 값이 0보다 크다면(일시 무적 타이머가 동작 중인 상황)
```

```
    if self.invincibleTimer > 0 then
```

```
        -- 장애물로 인한 피격처리를 수행하지 않고 메서드를 종료합니다.
```

```
            return
```

```
        end
```

```
    -- 위의 조건문이 수행되지 않았다면 (일시 무적 타이머가 동작 중이지 않은 상황)
```

```
    -- 체력을 감소시킵니다.
```

```
    _GameLogic:AddHP(_Config.obstacleDamage)
```

```

    -- 피격 상태를 true로 전환합니다.
    self.isHit = true;

    -- 플레이어에게 무적 상태임을 인지할 수 있도록 캐릭터 외형의 알파값을 절반으로 감소시켜 반투명 상태로 설정합니다.
    self.Entity.AvatarRendererComponent:SetAlpha(0.5)

    -- 일시 무적 타이머가 동작을 수행할 수 있도록 설정할 무적시간 값을 새로 갱신합니다.
    self.invincibleTimer = _Config.invincibleTime
}

void OnUpdate(number delta)
{
    -- isHit 값을 활용해 isHit 상황이 아닐때는 아래의 코드가 수행되지 않도록 방지합니다.
    if self.isHit == true then

        -- 일시 무적 타이머의 값이 0보다 큰 상황이라면,
        if self.invincibleTimer > 0 then
            -- 타이머의 값을 delta값만큼 감소시킵니다.
            self.invincibleTimer -= delta

            -- 일시 무적 타이머가 남은 지속시간이 없다면,
            else
                -- 플레이어에게 무적상황이 끝났음을 인지할 수 있도록 캐릭터 외형의 알파값을 원래 값(1)으로 증가시켜 불투명 상태로 설정합니다.
                self.Entity.AvatarRendererComponent:SetAlpha(1);
            end
        end
    end
}

```

최소 구현 기준

- 장애물 엔티티가 바닥 타일과 동일하게 이동되어야 합니다.
- 플레이어 캐릭터와 장애물 충돌 시 피격 판정이 발생해야 하며, 장애물로 인해 캐릭터가 뒤로 밀리지 않아야 합니다.
- 장애물로 인한 피격 시 일시 무적 기능이 활성화되어야 하며 타이머가 동작하는 동안에는 데미지를 입지 않아야 합니다.

응용 및 확장 방향

- 고정된 외형의 장애물뿐만 아니라, 제자리에서 위아래로 솟아오르는 동적인 애니메이션을 가진 액티브 장애물을 추가할 수 있습니다.
- 게임의 진행도(시간의 흐름)에 따라 장애물 생성 확률을 높이거나 다르게 설정하여 게임의 난이도를 조절할 수 있습니다.

[📖심화 학습]아이템 구현하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 아이템 구현하기: 01:55:02

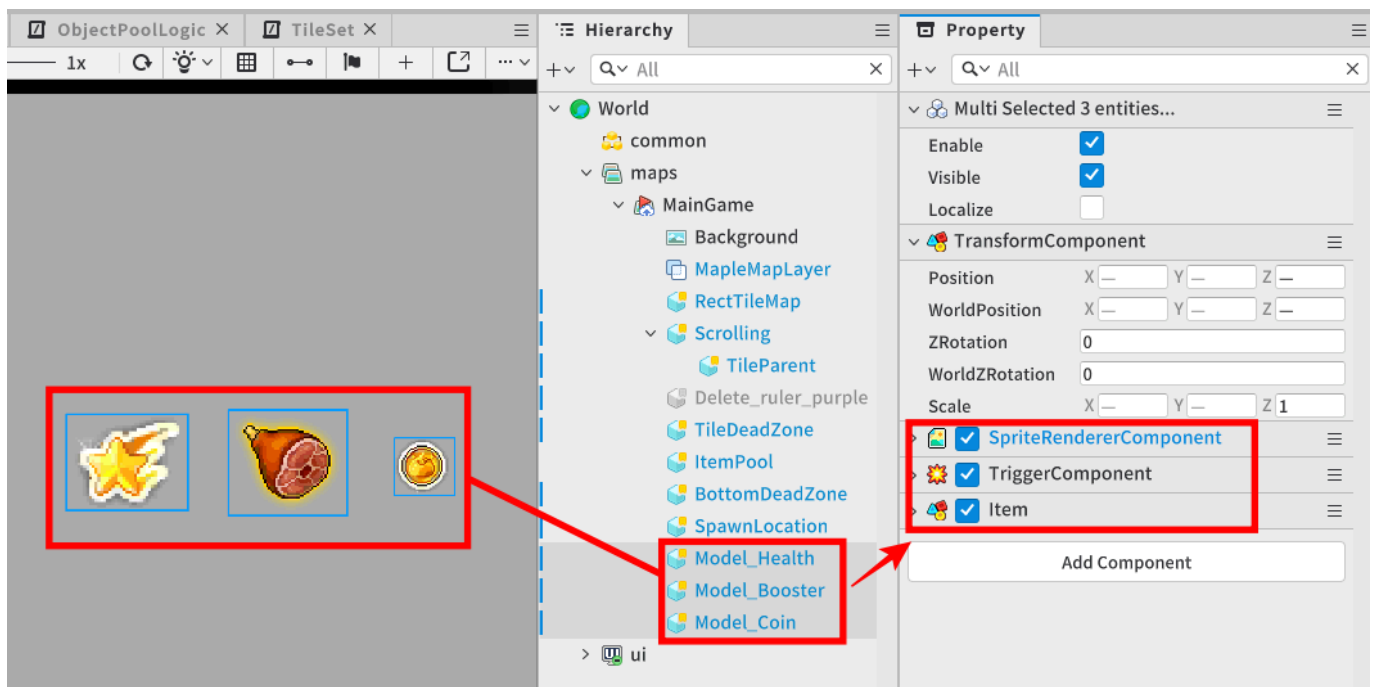
학습 목표

- DataSet에 아이템의 고유 정보(효과 값, 스폰 확률 등)를 정의하고, 이를 활용하기 위해 테이블 형태로 변환하여 관리하는 과정을 이해합니다.
- 오브젝트 풀링 시스템을 응용하여, 난수 연산을 통해 확률적으로 아이템을 스폰하고 맵 타일에 종속시키는 방법을 구현할 수 있습니다.
- 캐릭터와 아이템의 충돌을 감지하고, ItemLogic을 통해 획득한 아이템의 종류에 맞는 개별 효과(점수 증가, 체력 회복, 부스터 등)를 발동시키는 흐름을 학습합니다.

해당 영상 시청 후 수행해 볼 내용

- ItemInfo DataSet에 새로운 기능의 아이템(예:무적 아이템)데이터를 추가하고, 월드 실행 시 로직을 통해 정상적으로 테이블에 등록되는지 확인해 봅니다.
- 아이템의 스폰 확률(SpawnRate)값을 극단적으로 높거나 낮게 변경해 보며, 인게임에서 아이템이 의도한 빈도수로 생성되는지 테스트해 봅니다.
- 획득한 아이템 수치가 UI와 시스템에 정상적으로 반영되는지 플레이를 통해 점검해 봅니다.

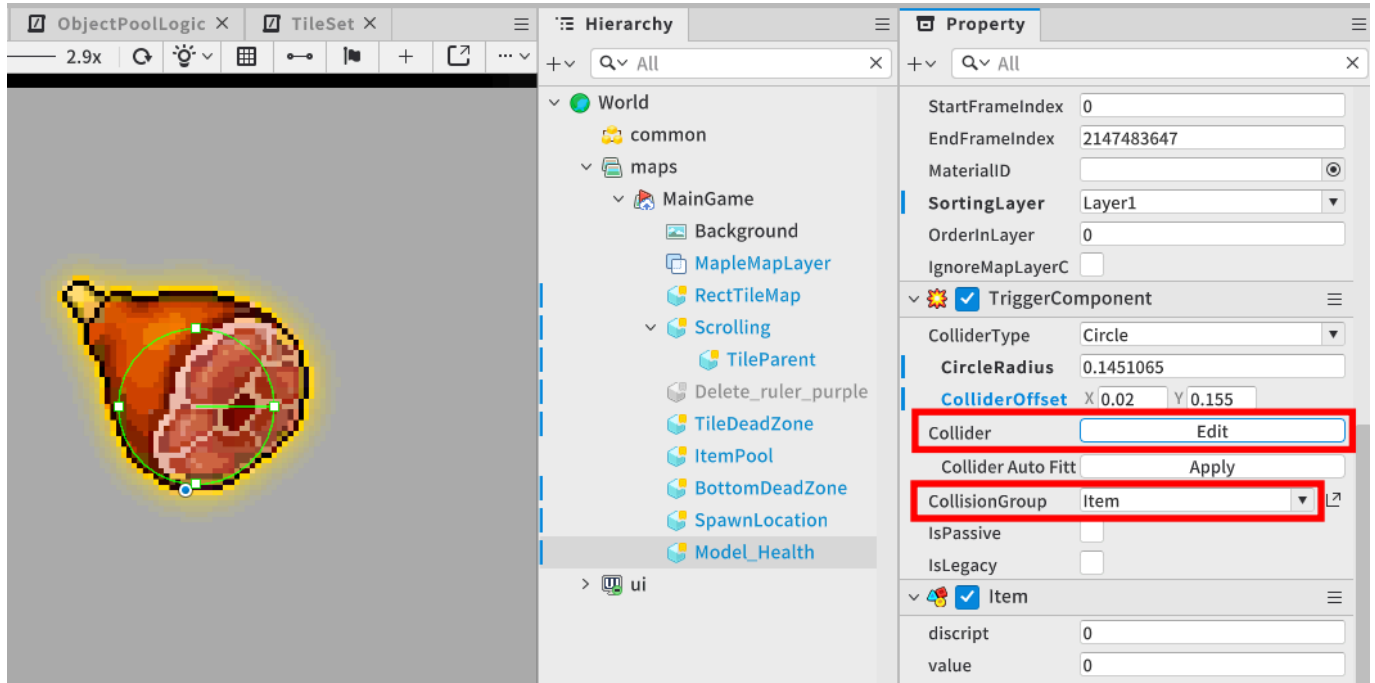
아이템 제작하기



- 샘플 월드에서는 아이템에 등록된 컴포넌트들은 SpriteRendererComponent, TriggerComponent, Item 입니다.
- SpriteRendererComponent는 아이템의 외형을 등록하기 위한 컴포넌트입니다.

- TriggerComponent는 캐릭터와의 충돌을 감지하기 위한 용도이며, 캐릭터가 밀려나는 등 실제 물리적인 충돌을 일으키지 않도록 설정하기 위함입니다.
- Item 컴포넌트는 직접 제작한 스크립트이며, 캐릭터와의 충돌이 감지되었을때 해당 아이템이 어떤 아이템인지 분류하기 위한 컴포넌트입니다.

TriggerComponent 설정하기



- 아이템의 충돌 박스의 형태를 변경하기 위해선 ColliderType의 드롭다운 메뉴를 활용합니다.
- 캐릭터와 상호작용하기 위해 아이템 엔티티에 등록된 TriggerComponent에서 Collider의 Edit기능을 활용해 충돌 범위를 설정합니다.
- CollisionGroup에서 새로운 충돌 그룹 Item을 추가하고, Player와의 충돌만 수행되도록 설정합니다.

Item 스크립트 제작하기

- Item 스크립트는 단순히 아이템 명, 아이템 고유 효과값 변수만 등록됩니다.
- 다음과 같이 코드를 작성합니다.
- 해당 코드는 Plain Text를 기준으로 작성되어 있습니다.

```
@Component
script Item extends Component

property number discript = 0 -- 아이템의 기본적인 기능, 변수를 포함하는 컴포넌트입니다.

property number value = 0 -- 아이템들이 갖는 고유 수치값입니다.

property string name = "" -- 아이템의 이름을 저장하기 위한 변수입니다.

end
```

아이템 데이터셋

ID	Name	RUID	Value	SpawnRate	EntryID
1	Coin	5b9a6180cb99	50	1	model://babb1
2	Health	fe85b89044804	20	0.03	model://938f1e
3	Booster	88b07fd668e34	1.4	0.05	model://8c7e9l

대상	설명
Name	아이템의 이름을 정의합니다.
RUID	아이템의 외형, 리소스 ID 값을 기록합니다.
Value	아이템별 고유 효과값을 정의합니다.
SpawnRate	아이템의 스폰확률을 정의하며 기록 시 0~1 사이의 값을 등록합니다.
EntryID	SpawnService를 통해 새로운 아이템 엔티티를 새로 생성할 때 필요한 원본(Model)의 EntryID 값을 저장합니다.

- 아이템의 이름, 고유 효과 값, 스폰 확률, 외형 데이터를 DataSet에 기입하여 체계적으로 관리하는 방법을 학습합니다.

아이템 데이터셋 테이블 등록하기

- DataSet으로 제작된 아이템별 정보는 DataSetLogic의 itemInfoTable에 등록됩니다.
- 데이터 테이블로 등록하는 기능은 DataSetLogic 스크립트의 InitItemInfoTable메서드 과정으로 진행됩니다.
- DataSetLogic코드의 내용이 필요한 경우 '2.Logic 이해하기' 단원에 첨부된 DataSetLogic스크립트 내용이 첨부되어 있습니다.

DataSetLogic의 InitItemInfoTable메서드

```
table itemInfoTable
```

```
void InitItemInfoTable()
{
```

```
    -- ItemInfo 테이블의 Row수입니다.
```

```
    local rowCount = _DataService:GetRowCount("ItemInfo")
```

```
    -- 각 줄의 데이터를 담기 위한 변수입니다.
```

```
    local rowData
```

```
    -- 1 부터 데이터셋의 Row수만큼 순회 시작합니다.
```

```

for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 아이템 이름을 저장합니다.
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- 아이템의 RUID 를 저장합니다.
        RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID")),
        -- 아이템이 가지는 효과 고윳값을 저장합니다.
        Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
        -- 아이템 생성 확률을 저장합니다.
        SpawnRate =
        tonumber(_DataService:GetCell(dataSetName,i,"SpawnRate")),
        -- 아이템의 모델(프리팸)의 EntryID 값을 저장합니다.
        EntryID =
        tostring(_DataService:GetCell(dataSetName,i,"EntryID"))
    }

    -- rowData로 저장한 데이터들을 테이블에 저장합니다.
    self.itemInfoTable[i] = rowData
end
}

```

아이템 데이터 활용하기

- ItemInfo DataSet으로부터 DataSetLogic으로 데이터를 저장한 후 DataSetLogic으로부터 데이터를 가져올 수 있습니다.
- 샘플 월드에서 아이템을 생성할 때 아이템에 대한 데이터 테이블이 필요합니다.
- 오브젝트 풀에서 월드 시작 시 아이템 엔티티를 미리 생성해둡니다.
- 다음의 코드는 월드 시작 시 아이템을 생성하는 ObjectPoolLogic의 CreateItem메서드입니다.

ObjectPoolLogic의 CreateItem

```

table itemDataSetIDTable = {"Coin" = 1, "Health" = 2, "Booster" = 3}

void CreateItem(string itemName, number createCount)
{
    local tempTable

    local dataSetID = self.itemDataSetIDTable[itemName]

    local entryID = _DataSetLogic.itemInfoTable[dataSetID].EntryID
    -- 데이터셋의 대상 아이템 모델의 Name 값을 불러옵니다.
    local name = _DataSetLogic.itemInfoTable[dataSetID].Name
    -- 데이터셋의 대상 아이템 모델의 Value 값을 불러옵니다.
    local value = _DataSetLogic.itemInfoTable[dataSetID].Value

    -- 대상 테이블의 유효성 검사를 진행합니다.

    -- /분기 1/ 대상 아이템 풀 내에 대상 풀(테이블) 이 없을 경우

```

```

if invalid(self.itemPool[name]) == false or self.itemPool[name] == nil
then
    -- 새로운 임시 테이블을 생성합니다.
    tempTable = {}
    -- 아이템 풀에서 내부 풀(테이블)을 임시테이블로 설정합니다.
    self.itemPool[name] = tempTable

    -- /분기 2/ 대상 아이템 풀 내에 대상 풀이 이미 있을 경우
else
    -- 임시 테이블(풀) 을 기존에 활용하고 있던 테이블로 선언합니다.
    tempTable = self.itemPool[name]
end

-- 매개변수2 createCount 만큼의 반복문을 수행합니다.
for i = 1 , createCount, 1 do
    -- SpawnService를 활용해 오브젝트를 생성합니다.
    local entity = _SpawnService:SpawnByModelId(entryID,name,Vector3.up *
100, self.ItemPoolParent)

    -- 생성한 오브젝트의 설정작업을 진행합니다.

    -- 생성한 오브젝트의 아이템의 고윳값을 설정합니다.
    entity.Item.value = value
    -- 생성한 오브젝트의 아이템의 name 변수를 설정합니다.
    entity.Item.name = name

    -- 담겨야 하는 테이블(풀)로 생성된 엔티티를 삽입합니다.
    table.insert(tempTable,entity)

    -- 생성된 엔티티 오브젝트를 비활성화합니다.
    entity.Enable = false
end
}

```

아이템 스폰 확률에 따른 아이템 생성

ID	Name	SpawnRate
1	Coin	1
2	Health	0.03
3	Booster	0.05

Random = 0.98 ⇒ "Coin"

Random = 0.04 ⇒ "Coin"

Random = 0.02 ⇒ "Health"

- 샘플 월드에 구현된 아이템은 총 3가지로, 점수를 획득하는 코인, 캐릭터의 체력을 회복하는 회복형 아이템, 일시적으로 스크롤 속도를 올려주는 부스터 아이템이 존재합니다.
- 모든 아이템이 동일한 확률로 생성될 경우, 회복형 아이템이 너무 자주 등장하여 게임이 끝나지 않는 밸런스 문제가 발생할 수 있습니다.
- 즉, 생성될 확률을 설정하여 아이템의 출현 빈도를 조절할 수 있습니다.

- 아이템 스폰 시 0부터 1 사이의 난수(Random)를 추출하고, 이를 ItemInfo 데이터셋의 SpawnRate 값과 비교하여 난수가 더 낮거나 같을 경우 해당 아이템을 스폰하도록 확률 로직이 설계되었습니다.

TileSet의 SpawnItem메서드

```
-- 확률형 스폰을 진행합니다. (무작위 숫자를 생성합니다.)
local randomRate = math.random(0,100) * 0.01

-- 엔티티타입으로 설정할 지역변수를 선언합니다.
---@type Entity
local tempEntity

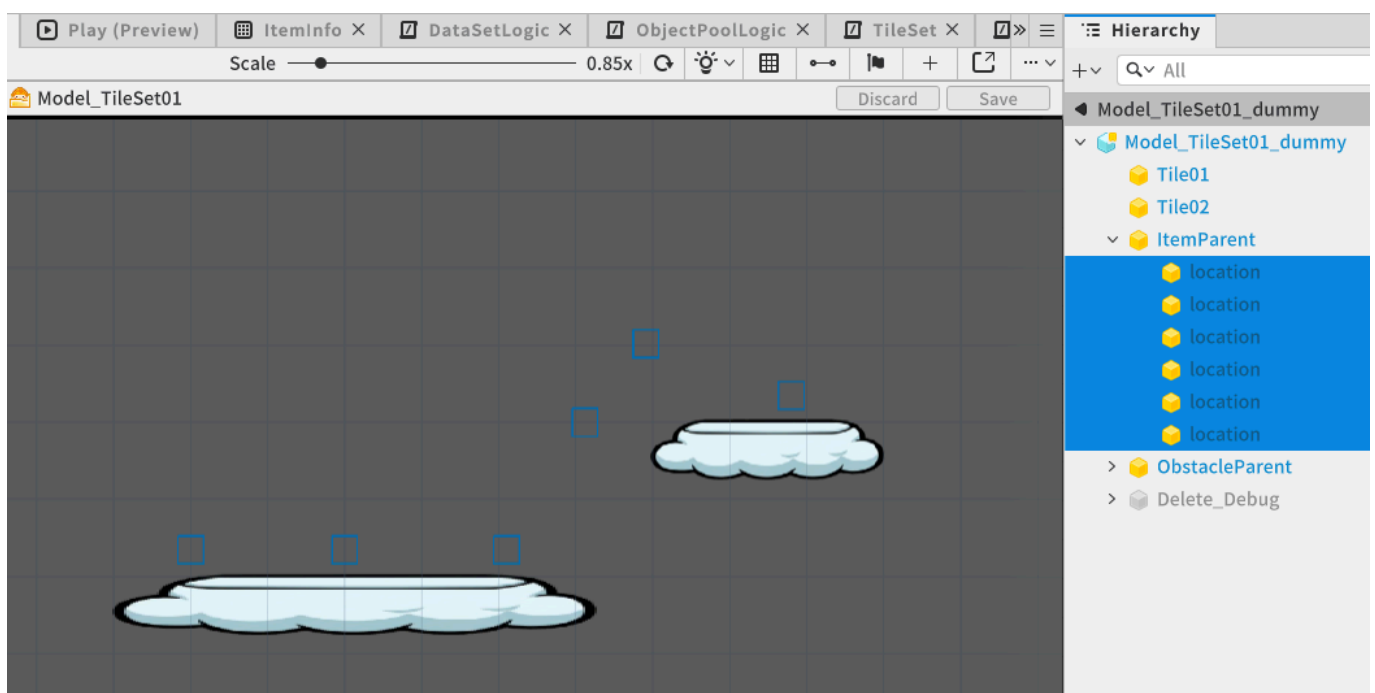
-- /분기1/ 랜덤 확률값이 healthSpawnRate값보다 작거나 같다면
if _DataSetLogic.itemInfoTable[2].SpawnRate >= randomRate then
-- 오브젝트 풀로부터 Health 아이템을 가져옵니다.
tempEntity = _ObjectPoolLogic.GetItem("Health")

-- /분기2/ 랜덤 확률값이 boosterSpawnRate 작거나 같다면
elseif _DataSetLogic.itemInfoTable[3].SpawnRate >= randomRate then
-- 오브젝트 풀로부터 Booster 아이템을 가져옵니다.
tempEntity = _ObjectPoolLogic.GetItem("Booster")

-- /분기3/ 설정한 다른 아이템들의 생성확률에 들지 못했다면 기본 아이템(코인)을 생성합니다.
else
tempEntity = _ObjectPoolLogic.GetItem("Coin")

end
```

지정된 위치에 아이템 배치하기



- 바닥 타일셋 내에는 ItemParent로 아이템의 위치를 설정하기 위한 엔티티들이 등록되어 있습니다.
- 오브젝트 풀로부터 스폰되는 아이템은 바닥 타일과 동일하게 이동되어야 합니다.
- 스폰된 아이템은 기준점 역할을 하는 엔티티의 자식(Child)으로 부착(Attach)한 뒤, 로컬 좌표를 원점(0, 0, 0)으로 초기화하여 부모와 완벽히 동일한 위치에 배치되도록 설정합니다.
- ItemParent의 자식요소의 개수만큼 순회를 진행하며 아이템 스폰을 반복합니다.

TileSet의 SpawnItem메서드

Entity itemParent -- 아이템 위치를 설정하기 위한 엔티티들의 부모 엔티티입니다.

```
void SpawnItem
{
    -- 아이템 위치를 설정하기 위한 엔티티들의 자식요소들의 순회를 진행합니다.
    for index, child in pairs(self.itemParent.Children) do

        local tempEntity

        -- 랜덤 확률과 SpawnRate값을 비교해 tempEntity의 아이템 엔티티 설정 과정

        -- 아이템 엔티티의 부모를 child로 설정합니다.
        tempEntity:AttachTo(child)

        -- 아이템 엔티티의 로컬 좌표값을 원점으로 부모와 동일한 위치로 설정합니다.
        tempEntity.TransformComponent.Position = Vector3(0,0,0)
}
```

최소 구현 기준

- 아이템이 스폰될 경우 바닥 타일셋에 부착되어 맵의 스크롤 이동과 함께 정상적으로 화면 왼쪽으로 이동해야 합니다.
- 캐릭터가 아이템의 Trigger 충돌 발생 시 화면에서 아이템 엔티티가 사라지며 오브젝트 풀로 반환되어야 합니다.

응용 및 확장 방향

- 게임 후반부로 갈수록 긍정적인 효과의 아이템뿐만 아니라, 획득 시 오히려 시야가 가려지거나 점수가 깎이는 함정 아이템을 확률적으로 스폰하여 게임의 난이도를 조절할 수 있습니다.
- 일정 시간 동안 장애물 충돌을 무시하는 실드 아이템이나, 주변의 코인을 한 번에 습득하는 자석 아이템 등 새로운 기믹의 아이템을 기획하고 전용 스크립트로 확장할 수 있습니다.

[📖심화 학습]데이터 저장 시스템 구현하기

영상 타임라인

- 강의 영상 내 아래의 타임라인에서 학습할 수 있습니다.
- 데이터 저장 시스템 구현하기: 02:13:50

학습 목표

- DataService를 활용하여 유저의 최고 점수를 서버에 저장하고, 재접속 시 이를 불러오는 방법을 이해합니다.
- 클라이언트와 서버 간의 데이터 동기화를 위해 변수의 Sync 속성을 설정하는 이유를 파악합니다.
- OnSyncProperty 이벤트를 활용하여 서버로부터 데이터를 성공적으로 불러온 시점을 정확히 파악하고, 이를 안전하게 사용하는 방법을 이해합니다.

해당 영상 시청 후 수행해 볼 내용

- 새로운 최고 점수값을 직접 기록한 뒤, 월드 재시작 시 정상적으로 이전 점수 정보가 불러와지는지 확인해 봅니다.

서버 측의 최고 점숫값 저장하기

- 매번 점수가 오를 때마다 서버와 통신하는 대신, 게임이 종료되는 시점 등 꼭 필요한 순간에만 데이터를 저장하여 서버 부하를 줄입니다.
- 서버 측에서 관리되는 BestScore 값은 게임 오버(체력 0) 시점에 현재 점수가 기존 최고 점수보다 높을 경우에만 새롭게 갱신됩니다.

서버 측에서 관리하는 BestScore값 갱신하는 방법
GameLogic에서 게임 종료시 호출되는 EndGame메서드

```
[Sync] number bestScore;
[None] number currentScore;

[client] -- 클라이언트 전용 메서드
void EndMainGame()
{
    -- 게임 종료시 호출되는 기능

    -- 현재 점수가 기존 최고 점수보다 높다면
    if self.currentScore > self.bestScore then
        -- 클라이언트(로컬 환경)에서 관리되는 bestScore 점수를 갱신
        self.bestScore = self.currentScore
        -- 서버 측으로도 점수 갱신을 요청
        self:UpdateScore(self.bestScore)

    end
}

[server] -- 서버 전용 메서드
void UpdateScore(number score)
{
    -- 로컬이 아닌 서버에서 관리중인 bestScore값을 갱신
```

```
self.bestScore = score
}
```

DataStorage로 유저 최고점숫값 저장하기

- 서버 측에서 관리되는 데이터 값이 있더라도, 이를 별도로 저장하지 않고 다시 월드에 접속할 경우 유저의 이전 데이터 값은 불러올 수 없습니다.
- 유저의 데이터 또는 게임의 데이터를 저장하기 위해서는 DataStorage 서비스를 활용합니다.
- DataStorage의 기능은 HandleUserLeaveEvent, HandleUserEnterEvent 이벤트에서 활용하는 것을 권장합니다.

DataStorage를 활용한 유저 및 게임 데이터 저장
GameLogic의 SaveScore메서드

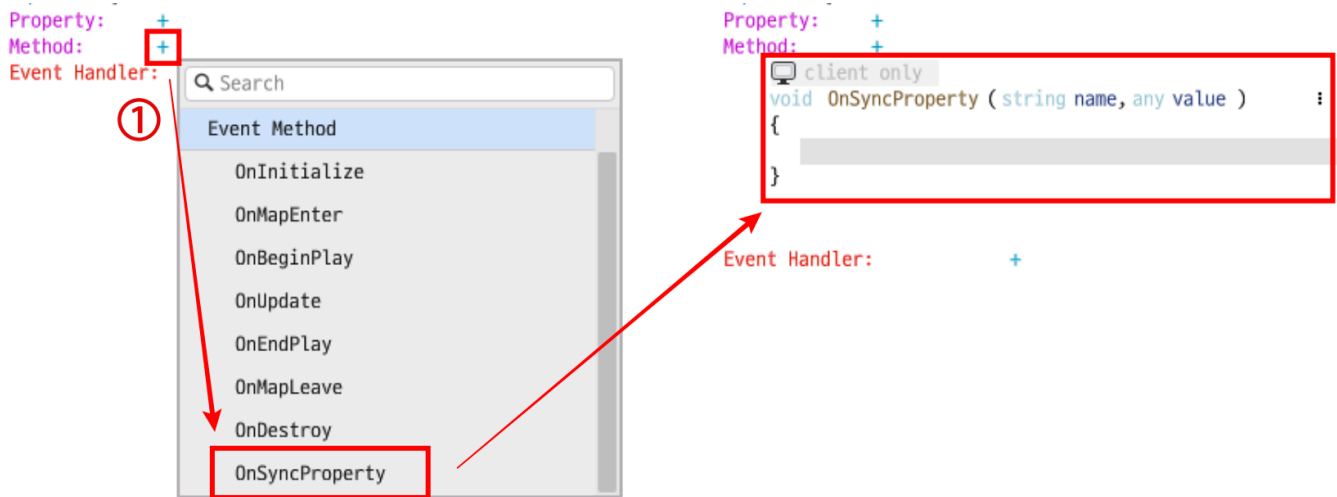
```
HandleUserLeaveEvent(UserLeaveEvent event)
{
    self:SaveScore(self.bestScore)
}

[server] -- 서버 전용 메서드
void SaveScore(number score)
{
    local data = _DataStorageService:GetGlobalDataStorage("MyData")
    local errorCode = data:SetAndWait("BestScore", tostring(score))
}
```

최고 점숫값 불러오기

- 데이터 저장과 마찬가지로, 월드에 처음 진입할 때 한 번만 데이터를 요청하여 불필요한 서버 통신을 방지합니다.
- 만약, 플레이어가 월드에 접속했을 때 이전에 기록되어 있던 최고 점숫값이 있다면 해당 점수값을 불러옵니다.

DataStorage와 OnSyncProperty를 활용한 이전 점수값 불러오기



1. 스크립트의 Method 탭에서 '+' 버튼 우클릭 후 OnSyncProperty를 통해 메서드를 생성할 수 있습니다.

- 유저의 데이터 또는 게임의 데이터를 불러오기 위해서는 DataStorage 서비스를 활용합니다.
- Sync로 설정된 변수는 OnSyncProperty 메서드를 통해 서버로부터 값이 동기화 완료된 시점을 정확히 감지할 수 있습니다.

DataStorage를 활용한 유저 및 게임 데이터 불러오기
GameLogic의 LoadData메서드

[Sync] number bestScore

```
HandleUserEnterEvent(UserEnterEvent event)
{
    self:LoadData()
}
```

[server] -- 서버 전용 메서드
void LoadData()

```
{
    -- 데이터스토리지 서비스를 활용해 'MyData'라는 데이터 스토리지를 가져옵니다.
    local data = _DataStorageService:GetGlobalDataStorage("MyData")

    -- 해당 데이터 스토리지 내에서 'BestScore' 키에 해당하는 Value 값을 가져옵니다.
    local errorCode,value = data:GetAndWait("BestScore")

    -- 임시 점수값 변수를 등록합니다.
    local tempScore

    -- 만약 저장된 데이터가 없다면
    if value == nil then
        -- 임시 점수값을 0으로 설정합니다.
        tempScore = 0

    -- 만약 저장된 데이터가 있다면
    else
        -- 임시 점수값을 기존 점수값으로 설정합니다.
        tempScore = tonumber(value)
    end
end
```

```

-- bestScore값을 갱신합니다.
-- Sync로 선언된 프로퍼티로써 값 변경이 일어날때,
-- 클라이언트쪽에서도 값이 동기화되며 OnSyncProperty 메서드가 자동으로 호출됩니다.
self.bestScore = tonumber(tempScore)
}

[client only]
void OnSyncProperty(string name, any value)
{
    -- 변수(프로퍼티)의 변경이 감지되었을 때

    -- 변경된 변수명이 "bestScore" 라면
    if name == "bestScore" then
        -- UIManagerLogic에게 메인 타이틀 화면의 최고 점수 텍스트 갱신을 요청합니다.
        _UIManagerLogic:UpdateTitleBestScore(tonumber(value))
    end
}

```

최소 구현 기준

- 플레이어가 게임에서 새로운 최고 점수를 달성하고 월드를 완전히 종료한 뒤 재접속했을 때, 타이틀 화면에 이전의 최고 점수가 0으로 초기화되지 않고 정상적으로 표기되어야 합니다.
- 서버로부터 점수 데이터 동기화가 완료되지 않은 시점에 UI 텍스트가 먼저 갱신되어 빈 값이나 에러가 표기되지 않도록, OnSyncProperty를 통한 시점 제어가 완벽히 동작해야 합니다.

응용 및 확장 방향

- bestScore뿐만 아니라 누적 재화, 캐릭터 목록, 진행한 스테이지 정보 등 다양한 데이터를 묶어 저장하는 시스템으로 확장해 볼 수 있습니다.